

Practice Exercise

This document provides a list of exercises to be practiced by learners. Please raise feedback in Talent Next, should you have any queries.

Skill	Java Microservices
Proficiency	S1
Document Type	Lab Practice Exercises
Author	L & D
Current Version	3.1
Current Version Date	6-March-2024
Status	Active

Document Control

Version	Change Date	Change Description	Changed By
1.0	18-July-2022	Baseline version	Manpreet Singh Bindra
2.0	23-Sep-2022	Added the problem statements for the topics like multiple profiles, actuator, Ribbon, Feign and Hystrix Dashboard.	Manpreet Singh Bindra
3.0	1-June-2023	Added the Spring Initializer (start.spring.io) to all the problem statements. Added the new problem statements for the topics like refresh strategies, @RefreshScope, Spring Cloud Bus, Hystrix with Ribbon & Feign, Hystrix Dashboard, Resilience4J, Sleuth and Zipkin, Netflix Zuul, Spring Cloud Gateway.	Manpreet Singh Bindra
3.1	15-March-2024	Updated the document as per the Spring Boot 3.x.x, Spring Cloud 2023.x.x, and JDK 17 version.	Manpreet Singh Bindra

Contents

Practice Exercise	1
Document Control.....	2
Problem Statement 1: Create MySQL Table.....	4
Problem Statement 2: Create Spring Boot Microservice with Spring Data JPA	5
Problem Statement 3: Working with Multiple Profiles	8
Problem Statement 4: Configuring production-ready features using Actuator	9
Problem Statement 5: Setting up Spring Cloud Config Server with Git Integration.....	10
Problem Statement 6: Setting up Spring Cloud Config Client	14
Problem Statement 7: Changing the application name for the Spring Cloud Config Client	15
Problem Statement 8: Spring Cloud Config Client Refresh Strategies.....	16
Problem Statement 9: Enabling Auto-Refresh of Configuration.....	17
Problem Statement 10: Dynamically manage and refreshable configuration with Spring Cloud Bus	19
Problem Statement 11: Configure Microservices with Eureka Service Registry Server	24
Problem Statement 12: Enable Dynamic Registration to eBookStore Microservice	27
Problem Statement 13: Enable Dynamic Discovery to eBookStore Consumer Microservice using Ribbon	29
Problem Statement 14: Stabilize the performance in your microservices using Hystrix with Ribbon Client	32
Problem Statement 15: Configuring Circuit Breaker: Hystrix Dashboard to Ribbon Client.....	34
Problem Statement 16: Enable Dynamic Discovery to eBookStore Consumer Microservice using Feign.....	37
Problem Statement 17: Stabilize the performance in your microservices using Hystrix with Feign Client	41
Problem Statement 18: Configuring Circuit Breaker: Hystrix Dashboard to Feign Client	43
Problem Statement 19: Spring Cloud Circuit Breaker, Adding Fault Tolerance to Your Ribbon Client Applications.....	46
Problem Statement 20: Configuring Circuit Breaker and Retry with Spring Cloud Resilience4J to Your Ribbon Client Applications	48
Problem Statement 21: Spring Cloud Circuit Breaker, Adding Fault Tolerance to Your Feign Client Applications.....	51
Problem Statement 22: Configuring Circuit Breaker and Retry with Spring Cloud Resilience4J to Your Feign Client Application.....	53
Problem Statement 23: Monitoring your Ribbon Client Application with Sleuth and Zipkin.....	58
Problem Statement 24: Implementing an API Gateway with Netflix Zuul.....	64
Problem Statement 25: Implementing Spring Cloud Gateway in Microservices	70

Note: Every Problem Statement start on a new page

Problem Statement 1: Create MySQL Table

Create a table **Book_Details** with the following attributes:

- id
- book_title
- book_publisher
- book_isbn
- book_number_of_pages
- book_year

Where “id” will be the primary key.

Add few records inside the table:

Title	ISBN	Page	Year
.NET Core in Action	978-1-61729-427-3	288	2018
.NET Development Using the Compiler API	978-1-484221-10-5	176	2016
.Net Framework 4.5 Expert Programming Cookbook	978-1-84968-742-3	276	2013
.NET Framework Essentials, 2nd Edition	978-0-596-00302-9	320	2002
.NET IL Assembler	978-1-4302-6761-4	492	2014
.NET Standard 2.0 Cookbook	978-1-78883-466-7	394	2018
.NET Test Automation Recipes	978-1-59059-663-0	408	2006
10 LED Projects for Geeks	978-1-59327-825-0	240	2018
101 Design Ingredients to Solve Big Tech Problems	978-1-93778-532-1	298	2013
101 Excel 2013 Tips, Tricks and Timesavers	978-1-118-64218-4	312	2013
101 UX Principles	978-1-78883-736-1	414	2018
101 Windows Phone 7 Apps	978-0-672-33552-5	1152	2011
20 Easy Raspberry Pi Projects	978-1-59327-843-4	288	2018
20 Recipes for Programming MVC 3	978-1-4493-0986-2	122	2011
20 Recipes for Programming PhoneGap	978-1-4493-1954-0	78	2012
21 Recipes for Mining Twitter	978-1-4493-0316-7	72	2011
21st Century C	978-1-4493-2714-9	296	2012
21st Century C, 2nd Edition	978-1-49190-389-6	408	2014
21st Century Robot	978-1-44933-821-3	278	2014
25 Recipes for Getting Started with R	978-1-4493-0323-5	50	2011

Problem Statement 2: Create Spring Boot Microservice with Spring Data JPA

Mphasis got a requirement to create an eBookStore portal, wherein multiple users access the books details. We need to design the API for the application so we can achieve the interoperability feature in the application.

Technology stack:

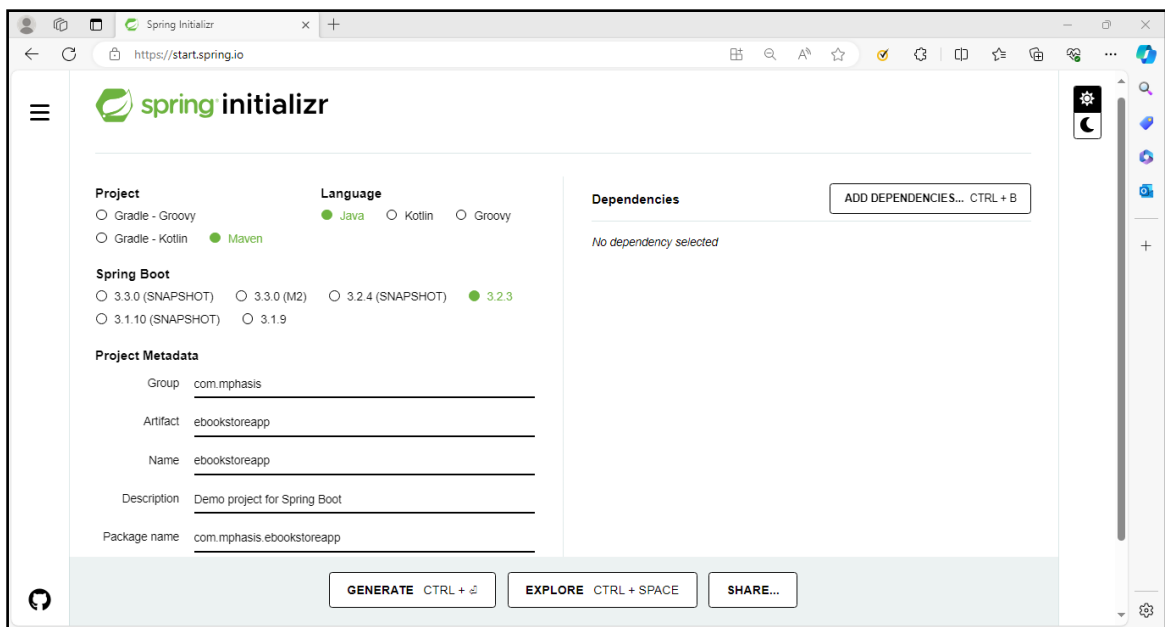
- Spring Web, Spring Data JPA, Spring Boot Actuator
- H2 Database

Building a RESTful web application where CRUD operations to be carried out on entities.

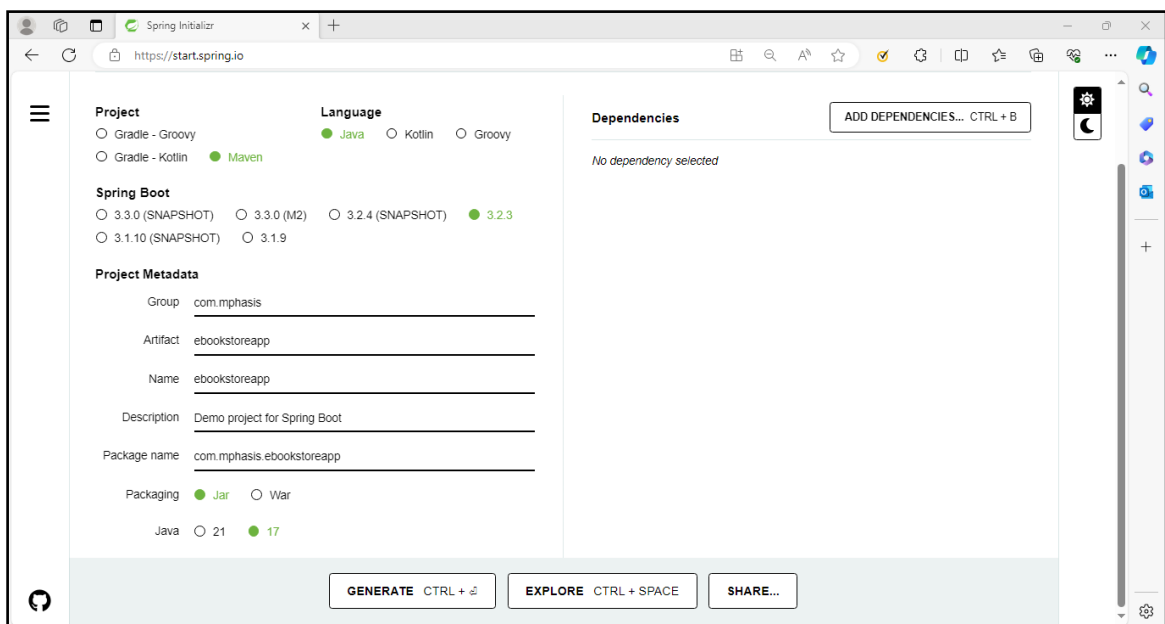
Here will have multiple layers into the application:

1. Create a new Project for **ebookstoreapp** using the **Spring Initializr** (start.spring.io).

Refer the below screenshots:

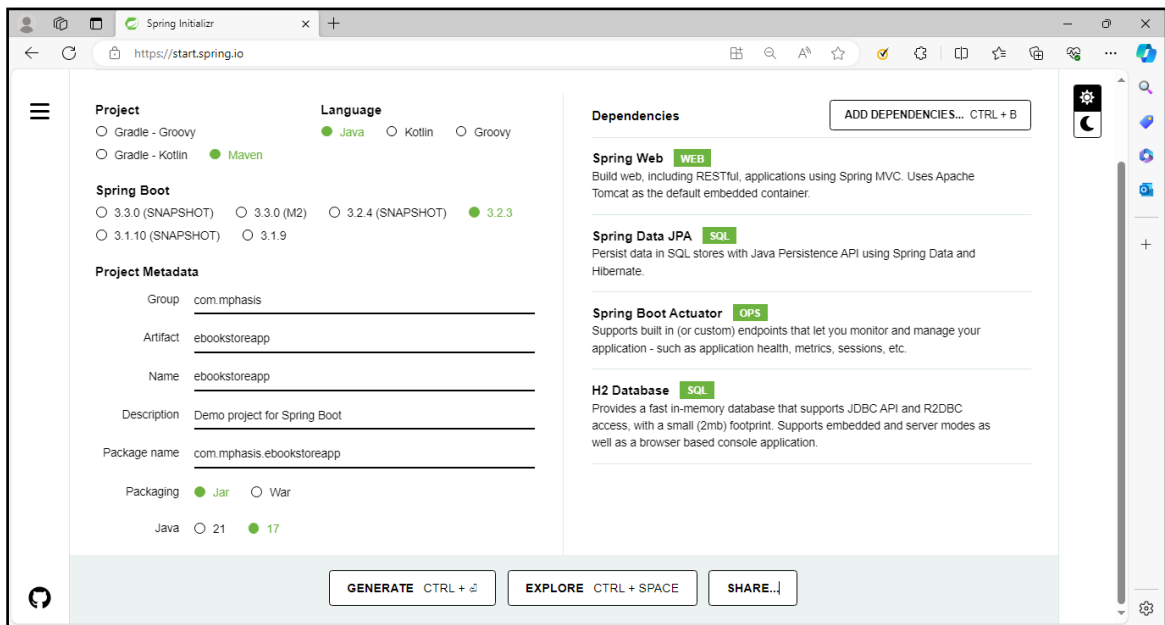


A screenshot of the Spring Initializr web application. The interface is divided into several sections:
- **Project**: Radio buttons for Gradle - Groovy, Gradle - Kotlin, and Maven (selected).
- **Language**: Radio buttons for Java (selected), Kotlin, and Groovy.
- **Spring Boot**: Radio buttons for 3.3.0 (SNAPSHOT), 3.3.0 (M2), 3.2.4 (SNAPSHOT), 3.2.3 (selected), 3.1.10 (SNAPSHOT), and 3.1.9.
- **Project Metadata**: Fields for Group (com.mphasis), Artifact (ebookstoreapp), Name (ebookstoreapp), Description (Demo project for Spring Boot), and Package name (com.mphasis.ebookstoreapp).
- **Dependencies**: A section with a button 'ADD DEPENDENCIES... CTRL + B' and the text 'No dependency selected'.
- At the bottom, there are three buttons: 'GENERATE CTRL + G', 'EXPLORE CTRL + SPACE', and 'SHARE...'.
The browser address bar shows 'https://start.spring.io'.



A second screenshot of the Spring Initializr web application, showing additional options.
- **Project**: Same as the first screenshot.
- **Language**: Same as the first screenshot.
- **Spring Boot**: Same as the first screenshot.
- **Project Metadata**: Same as the first screenshot.
- **Packaging**: Radio buttons for Jar (selected) and War.
- **Java**: Radio buttons for 21 and 17 (selected).
- **Dependencies**: Same as the first screenshot.
- At the bottom, there are three buttons: 'GENERATE CTRL + G', 'EXPLORE CTRL + SPACE', and 'SHARE...'.
The browser address bar shows 'https://start.spring.io'.

2. Select the below Dependencies:



The image shows the Spring Initializr web form at <https://start.spring.io>. The form is divided into three main sections: Project, Language, and Dependencies.

Project

- Language: ☒ Java, ☐ Kotlin, ☐ Groovy
- Spring Boot: ☐ 3.3.0 (SNAPSHOT), ☐ 3.3.0 (M2), ☐ 3.2.4 (SNAPSHOT), ☒ 3.2.3, ☐ 3.1.10 (SNAPSHOT), ☐ 3.1.9
- Project Metadata:
 - Group:
 - Artifact:
 - Name:
 - Description:
 - Package name:
 - Packaging: ☒ Jar, ☐ War
 - Java: ☐ 21, ☒ 17

Dependencies

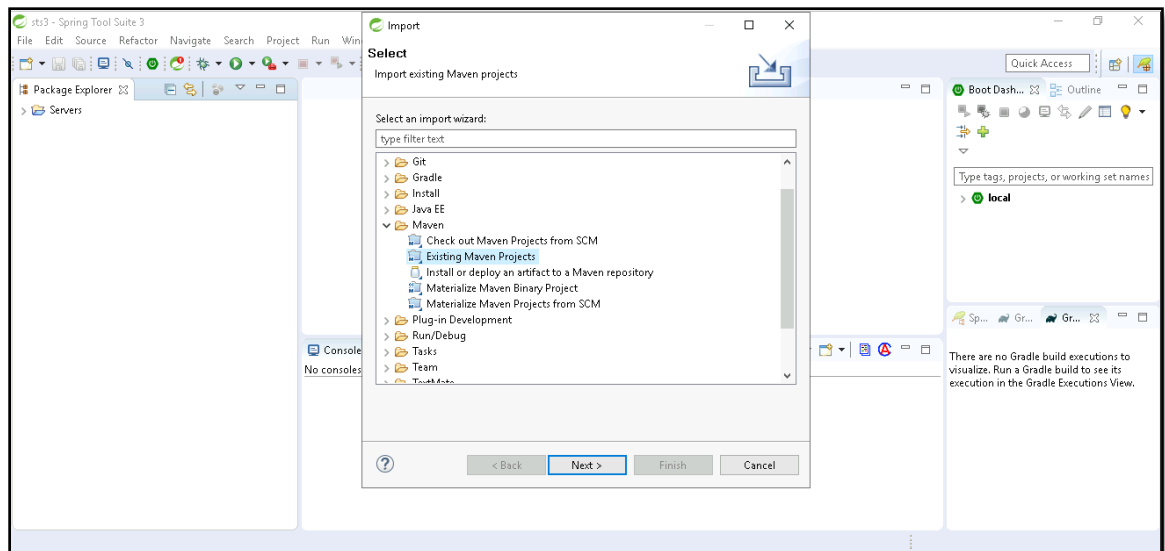
ADD DEPENDENCIES... CTRL + B

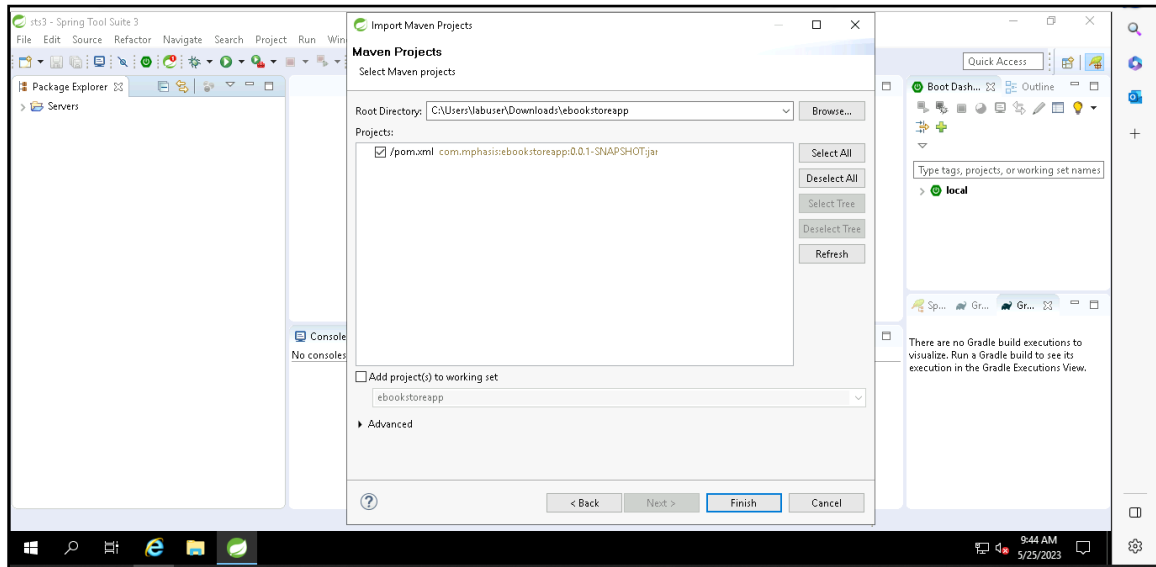
- Spring Web** WEB: Build web, including RESTful, applications using Spring MVC. Uses Apache Tomcat as the default embedded container.
- Spring Data JPA** SQL: Persist data in SQL stores with Java Persistence API using Spring Data and Hibernate.
- Spring Boot Actuator** OPS: Supports built in (or custom) endpoints that let you monitor and manage your application - such as application health, metrics, sessions, etc.
- H2 Database** SQL: Provides a fast in-memory database that supports JDBC API and R2DBC access, with a small (2mb) footprint. Supports embedded and server modes as well as a browser based console application.

Buttons at the bottom: GENERATE CTRL + G, EXPLORE CTRL + SPACE, SHARE...

3. Click on GENERATE button or CTRL + Enter to create the project structure.

4. Let's import the ebookstoreapp maven project in STS.





5. Will take some time for the project to be imported and the maven dependencies to be downloaded once you click **Finish**.
6. Let's start building our application.
7. Create an Entity: Book
8. Create a BookStoreRepository interface and will make use of Spring Data JPA (JpaRepository)
 - a. Will inherit few behaviors:
 - Add the Book details.
 - Update the Book details.
 - Will have findAll books method.
 - Will have findByIdByBookId method.
 - Delete book by id method.
 - b. Need to add few behaviors using JPA Query Methods:
 - Will have findByBookTitle, findByBookPublisherLike, findByYear method.
9. Create a BookStoreService class and will expose all these services.
10. Finally, create a BookStoreRestController will have the following Uri's:

URI	METHODS	Description	Format
/books	POST	Add the Book details	JSON
/books	PUT	Update the Book details	JSON
/books	GET	Give all books details	JSON
/books/{book_id}	GET	Give a single book description searched based on id	JSON
/books/{book_id}	DELETE	Delete book by id method	
/books/title/{book_title}	GET	Give all books details searched based on title	JSON
/books/publisher/{book_publisher}	GET	Give all books details searched based on publisher	JSON
/books?year={book_year}	GET	Give all books details searched based on year	JSON

11. Running on a web server tier (using tomcat).

Problem Statement 3: Working with Multiple Profiles

Now the team planned to deploy the application in the multiple stages with multiple profiles, using the H2 and MySQL databases.

Technology stack:

- Spring Boot Actuator
- Spring Web
- Spring Data JPA
- MySQL Database
- H2 Database

Refer the RESTful web application created in the Problem Statement 2.

Here will have multiple layers into the application:

1. No changes in the source code of the application.
2. Create the below properties files:
 - a. application.properties file for setting the profile if required or use command line.
 - b. application-dev.properties file with the H2 configuration.
 - c. application-prod.properties file with the MySQL configuration.
3. Execute the application by switching the profile to **Prod/Dev** using command line/application.properties files.

```
1) java -Dspring.profiles.active=dev -jar ebookstoreapp-1.0.0-SNAPSHOT.jar  
2) java -Dspring.profiles.active=prod -jar ebookstoreapp-1.0.0-SNAPSHOT.jar
```


Problem Statement 4: Configuring production-ready features using Actuator

Modify the previous application to add the capability of Actuator in your Spring Boot application, you can monitor, gather metrics, understand traffic, and manage your application by invoking different HTTP endpoints exposed by Spring Boot Actuator.

Technology stack:

- Spring Actuator

Refer the RESTful web application updated in the Problem Statement 3.

Initialization steps are mentioned below:

1. To enable Spring Boot Actuator, we just need to add the *spring-boot-starter-actuator* dependency to our package manager.

```
<dependency>
  <groupId>org.springframework.boot</groupId>
  <artifactId>spring-boot-starter-actuator</artifactId>
</dependency>
```

2. Add the property into application.properties:

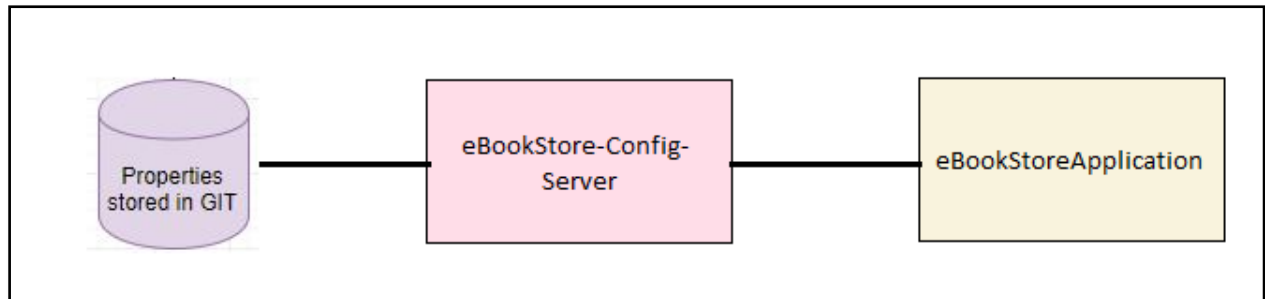
```
management.endpoints.web.exposure.include=*
```

3. Try accessing some available endpoints:

ENDPOINTS	USAGE
/actuator/metrics	To view the application metrics such as memory used, memory free, threads, classes, system uptime etc.
/actuator/env	To view the list of Environment variables used in the application.
/actuator/beans	To view the Spring beans and its types, scopes, and dependency.
/actuator/health	To view the application health
/actuator/info	To view the information about the Spring Boot application.
/actuator/trace	To view the list of Traces of your Rest endpoints.
/actuator/configprops	Allows us to fetch all @ConfigurationProperties beans.
/actuator/heapdump	Builds and returns a heap dump from the JVM used by our application.
/actuator/threaddumps	Dumps the thread information of the underlying JVM.

Problem Statement 5: Setting up Spring Cloud Config Server with Git Integration

Now in the era of distributed system. The company intends to have a centralized place to manage external attributes for applications across all settings. Bind to the Config Server to configure the Spring Environment with remote property sources.



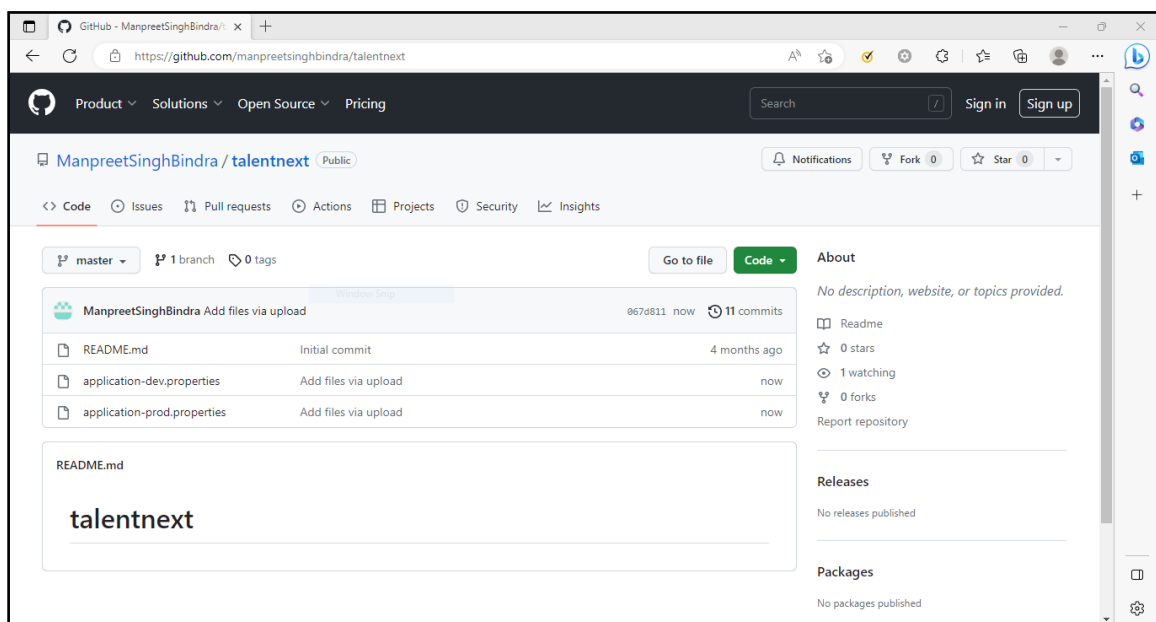
Technology stack:

- Spring Cloud Config Server

Steps for Spring Cloud Config Server:

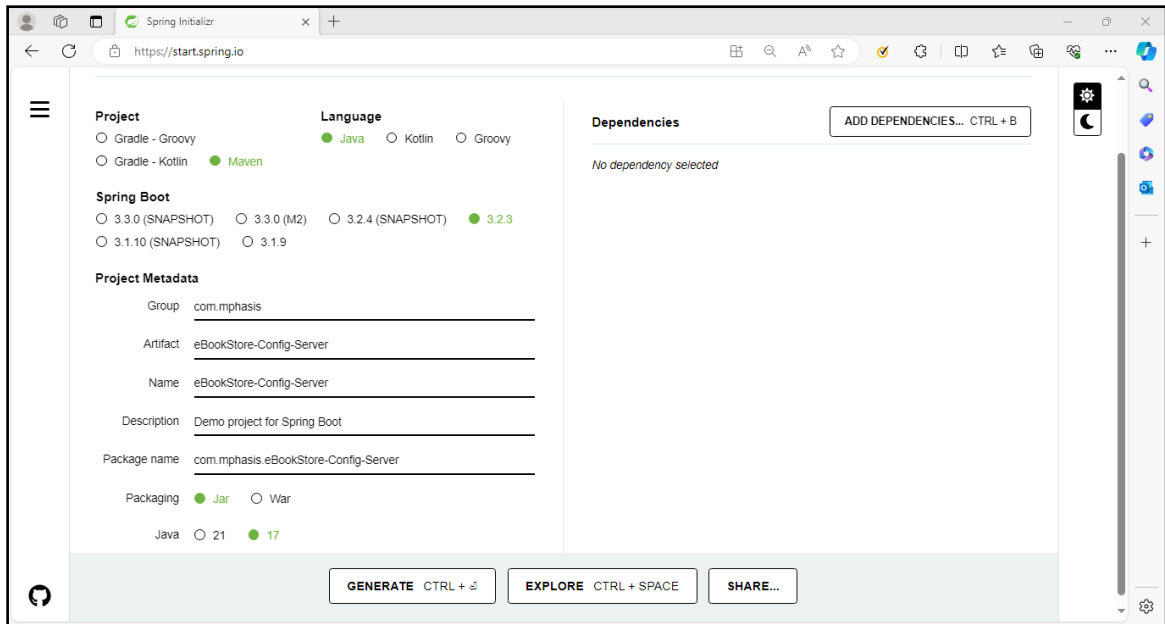
1. Create a **git repository** and add the files `application-dev.properties` and `application-prod.properties`.

Refer the below screenshots:

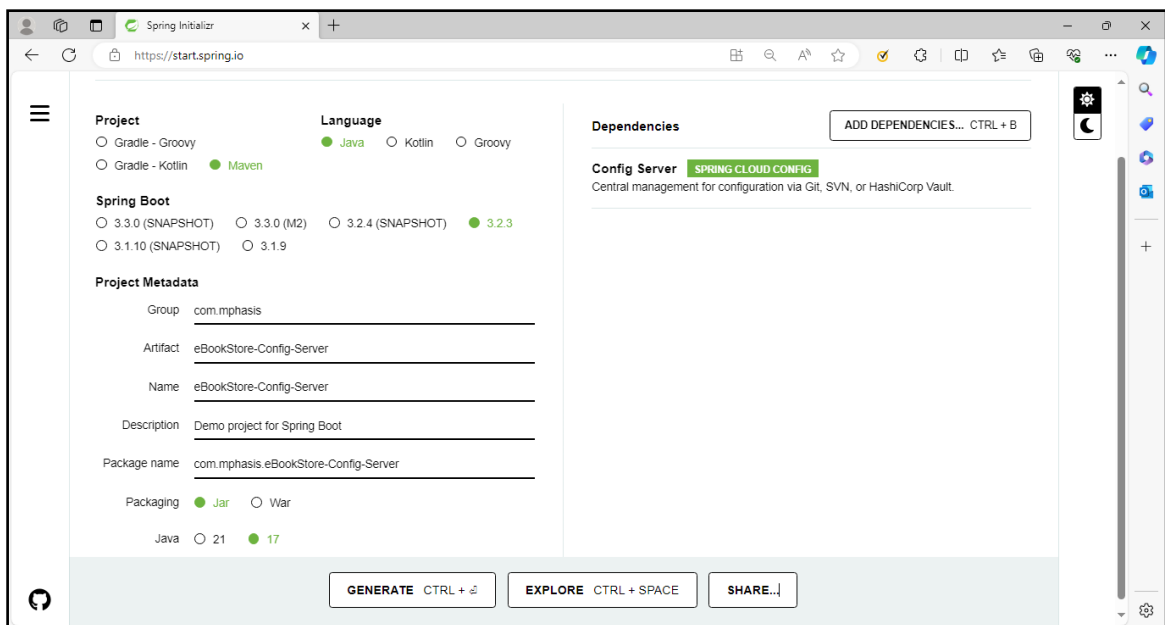


2. Create a new Project for **eBookStore-Config-Server** using the **Spring Initializr** (start.spring.io).

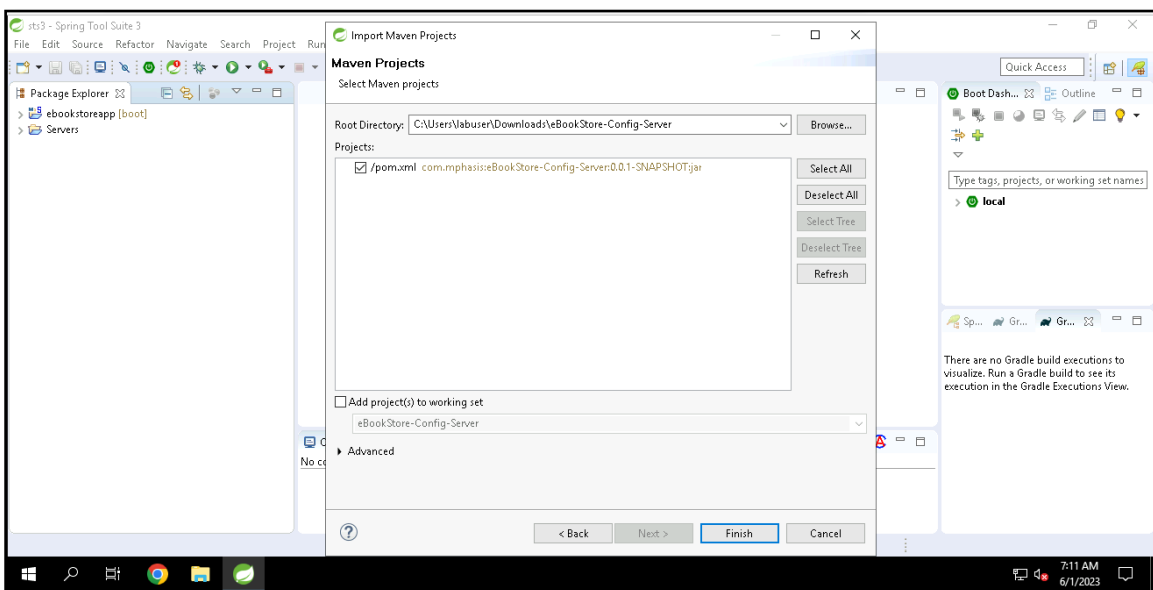
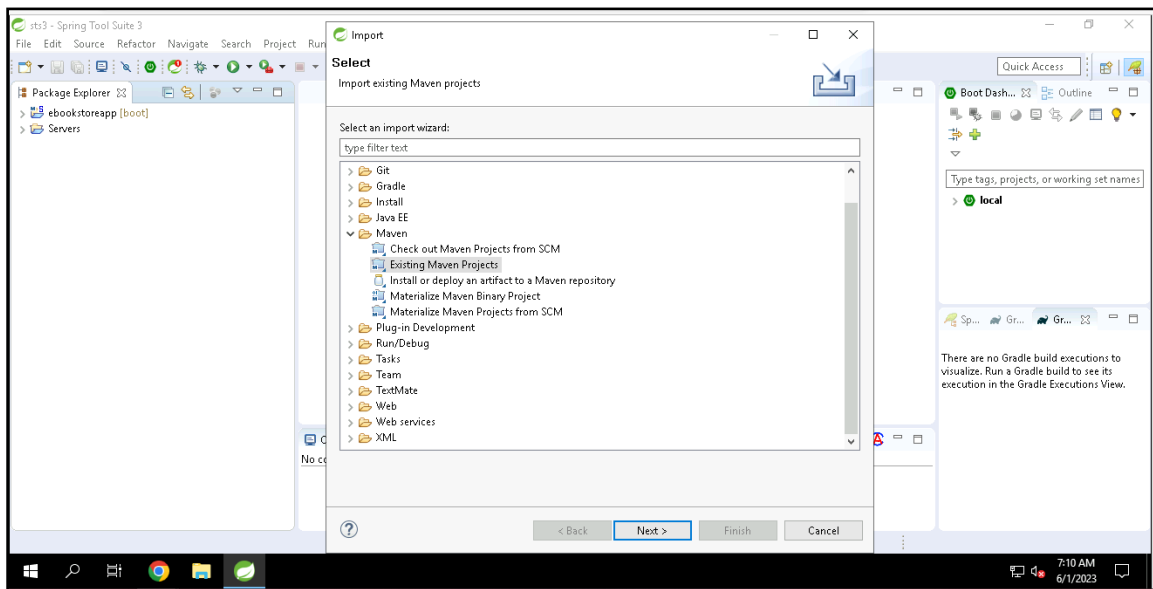
Refer the below screenshots:



3. Select the below Dependencies:



- Let's import the eBookStore-Config-Server maven project in STS.



- Will take some time for the project to be imported and the maven dependencies to be downloaded once you click **Finish**.
- In the Application class, Add **@EnableConfigServer** annotation.
- Specify the server.port = 8888, git uri, username and password in the application.properties files for the eBookStore-Config-Server application to be linked to a Git Repository.



- Start the application.

9. Use the syntax listed below to test the eBookStore-Config-Server application:

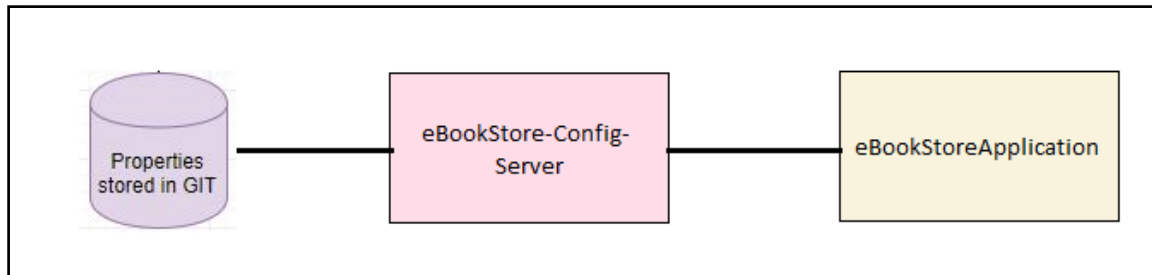
```
http://localhost:8888/{name}/{profile}/{label}
```

10. Test the eBookStore-Config-Server application with several Profiles:

```
-----  
application-dev.properties:  
http://localhost:8888/application/dev/master  
  
application-prod.properties:  
http://localhost:8888/application/prod/master  
-----
```

Problem Statement 6: Setting up Spring Cloud Config Client

The "eBookStore" application will make use of an independent database. The Config-Server will be used to manage database configuration information. This application is not just a website in the Microservice architecture; it is a service that provides information to other applications.



Technology stack:

- Spring Cloud Config Client

Steps for Spring Cloud Config Client:

1. Refer the **ebookstoreapp** created in the problem statement – 4 and add the **Config Client and MySQL** dependencies to the application.
2. Include the application name and Config Server uri in the application.properties files. For the eBookStore application to be linked to eBookStore-Config-Server.

```
application.properties
1
2 spring.main.allow-bean-definition-overriding=true
3
4 #Before - Spring Boot 2.4 Version
5 #spring.cloud.config.uri=http://localhost:8888
6
7 #After - Spring Boot 2.4 Version
8 spring.config.import=optional:configserver:http://localhost:8888
9 spring.cloud.config.label=master
10 spring.application.name=application
```

3. Start the eBookStore-Config-Server.
4. After that, launch the ebookstoreapp application using VM arguments to configure the profiles.

VM Arguments:

```
-Dspring.profiles.active=dev
-Dspring.profiles.active=prod
```

Problem Statement 7: Changing the application name for the Spring Cloud Config Client

As the product evolves, we may build numerous microservices with a centralized location to manage external properties for applications across all environments. So, assign an application name to each microservice.

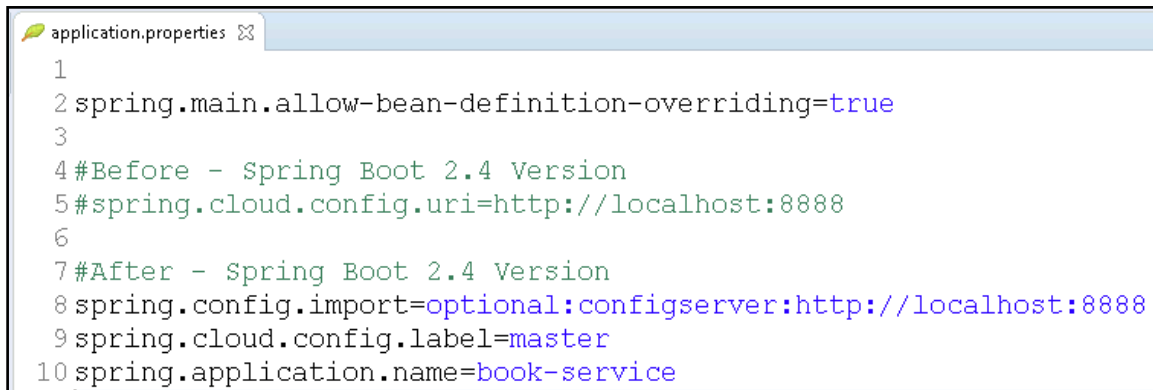
Steps for Spring Cloud Config Client:

1. Rename the properties files as follows:
 - a. application-dev.properties to book-service-dev.properties
 - b. application-prod.properties to book-service-prod.properties
2. Verify that the eBookStore-Config-Server is running.
3. Test the eBookStore-Config-Server application with the following Profiles:

```
book-service-dev.properties
http://localhost:8888/book-service/dev/master

book-service-prod.properties
http://localhost:8888/book-service/prod/master
```

4. Refer the **ebookstoreapp** developed in the problem statement – 6 and add the following points to it:
 - a. In application.properties files, add the **spring.application.name=book-service** property.
 - b. Add the **spring.cloud.config.label=master** property pointing to master branch.



```
application.properties
1
2 spring.main.allow-bean-definition-overriding=true
3
4 #Before - Spring Boot 2.4 Version
5 #spring.cloud.config.uri=http://localhost:8888
6
7 #After - Spring Boot 2.4 Version
8 spring.config.import=optional:configserver:http://localhost:8888
9 spring.cloud.config.label=master
10 spring.application.name=book-service
```

5. After that, launch the ebookstoreapp application using VM arguments to configure the profiles.

```
VM Arguments:
-Dspring.profiles.active=dev
-Dspring.profiles.active=prod
```

Problem Statement 8: Spring Cloud Config Client Refresh Strategies

The `/actuator/refresh` endpoint is used for the first way of property refreshment. This endpoint is exposed in configuration clients, a call to it just refreshes the client to which the request is made.

Steps for Spring Cloud Config Client:

1. Verify that the `eBookStore-Config-Server` and `ebookstoreapp` are both running.
2. Refer the `ebookstoreapp` developed in the problem statement – 7, check the value of `"spring.datasource.hikari.maximum-pool-size"` property when your VM arguments are set to prod profile.

```
"spring.datasource.hikari.maximum-pool-size": "10",
```

3. Assume that we have changed the maximum-pool-size in properties file in the git repository to 20 and want to reflect this change in our application at runtime.

```
"spring.datasource.hikari.maximum-pool-size": "20",
```

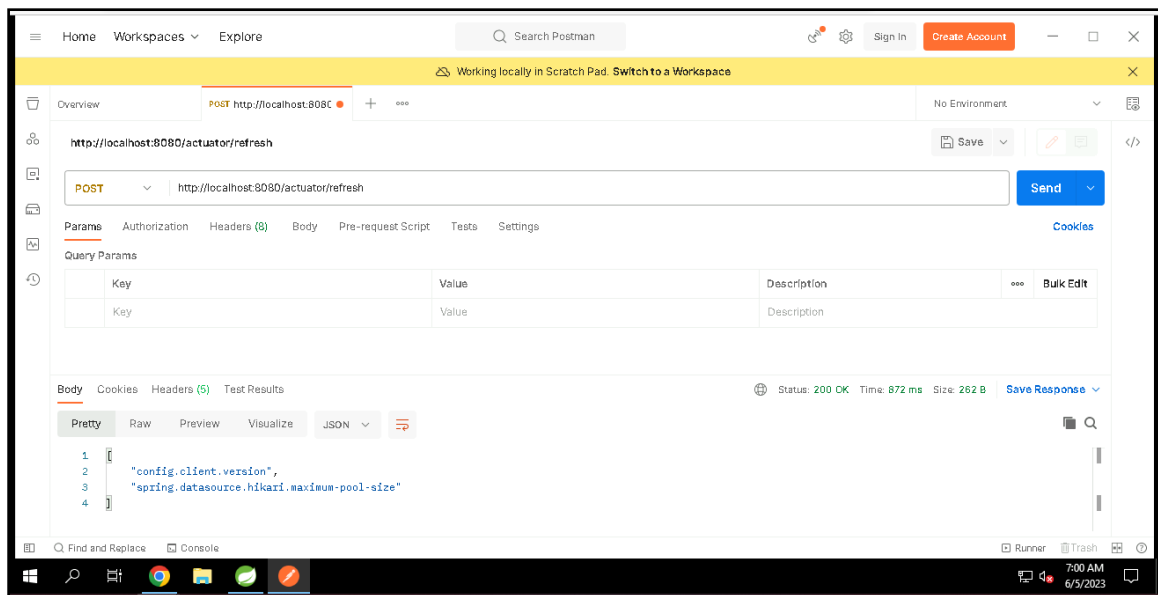
4. Next, we make a rest call to the `/actuator/refresh` endpoint exposed by the config client application.

Either using the curl command:

```
curl -H "Content-Type: application/json" -d {} -X POST http://localhost:8080/actuator/refresh
```

or

Using the Postman tools:



5. With this call, the RefreshEndpoint in the config client prepares the config server URL and performs a call to the config server to retrieve the most recent version of the properties.
6. With the up-to-date properties, the property sources in the Environment are refreshed.

Problem Statement 9: Enabling Auto-Refresh of Configuration

The attributes bounded with **@Value** in beans with the annotation **@RefreshScope** are refreshed immediately after reloading property sources via the **/actuator/refresh** endpoint.

@RefreshScope helps in providing a new scope for defining the bean. It refreshes the property sources as well as the bean.

Steps for Spring Cloud Config Client Refresh Strategies:

1. Create a new `ValueRefreshConfigBean` class for apply the theme to your application.

```
@Component
@RefreshScope
public class ValueRefreshConfigBean {

    private String color;

    public ValueRefreshConfigBean(
        @Value("${application.theme.color}") String color) {

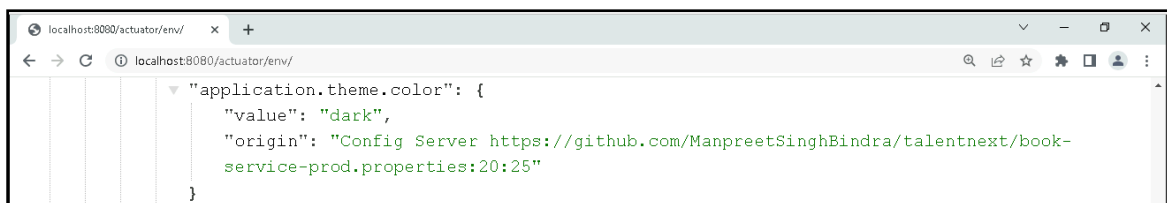
        this.color = color;
    }

    public String getColor() {
        return color;
    }
}
```

2. Include the following custom property to `book-service-prod.properties` file:

```
"application.theme.color": "dark"
```

3. Start the `ebookstoreapp` application.
4. Verify the current environment variables.



5. Assume that we have changed the application theme in properties files in the git repository and want to reflect these changes in our application at runtime.
6. Next, we make a rest call to the **/actuator/refresh** endpoint exposed by the config client application.

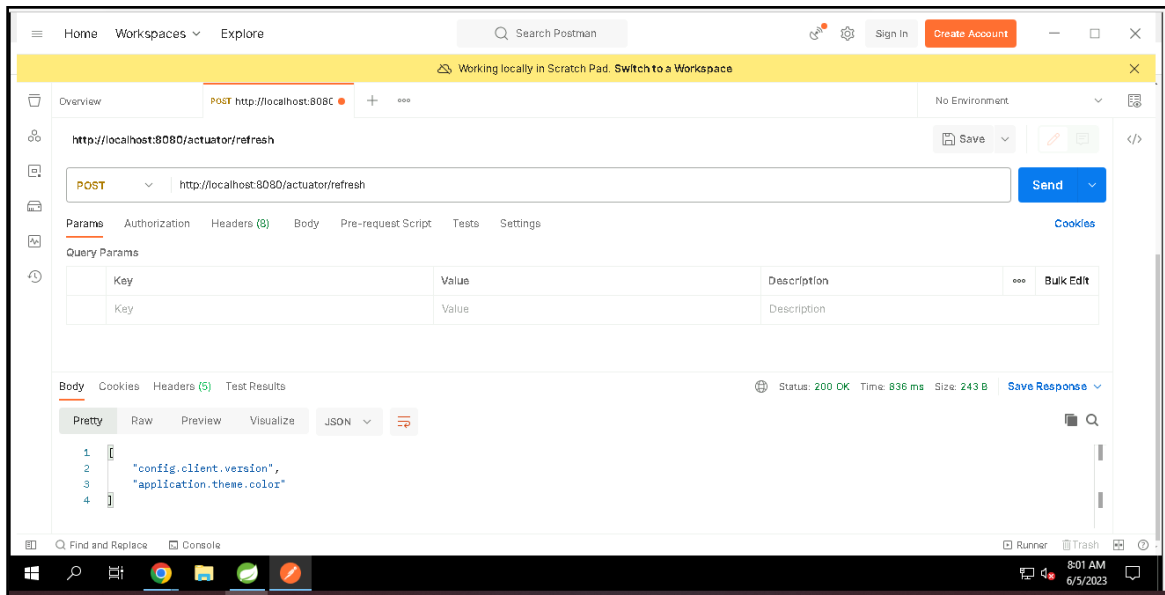
Java Microservices – S1 – Practice Exercise

Either using the curl command:

`curl -H "Content-Type: application/json" -d {} -X POST http://localhost:8080/actuator/refresh`

or

Using the Postman tools:



Problem Statement 10: Dynamically manage and refreshable configuration with Spring Cloud Bus

Spring Cloud Bus provides feature to refresh configurations automatically across multiple instances of the required modules. This could be useful when multiple micro-services are up and running. This could be achieved by connecting all micro-services via a single message broker (for example, RabbitMQ). Whenever an instance gets refreshed, this event is subscribed to all the micro-services listening to the broker and they also get refreshed. The endpoint `/actuator/busrefresh` can be used to refresh any single instance at any time.

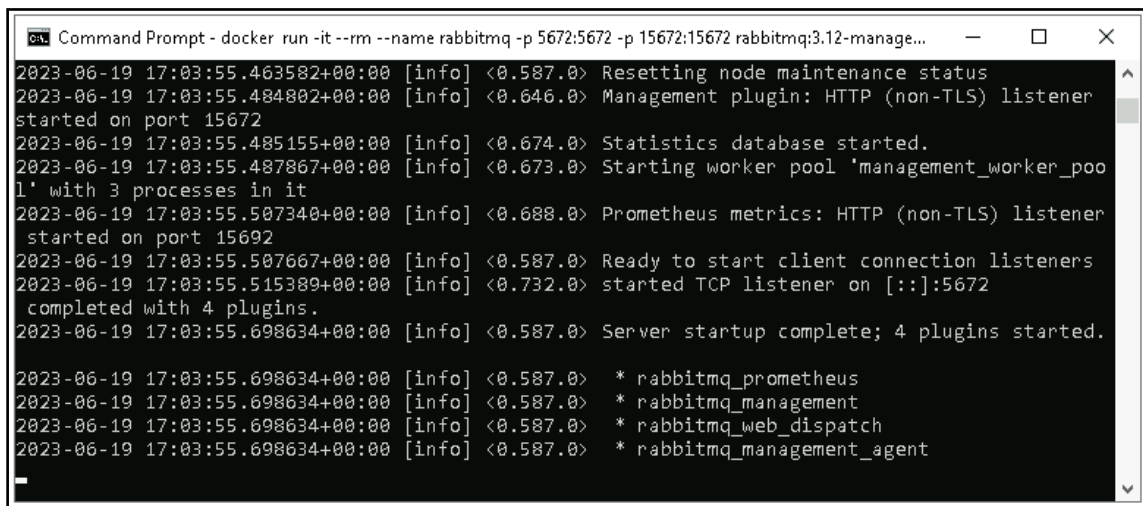
Technology stack:

- Spring Cloud Bus - AMQP

Steps for Spring Cloud Config Client Refresh Strategies:

1. To Download and Run Rabbit MQ, go to <https://rabbitmq.com>.
2. Click on Download + Installation button.
3. Ensure the Docker is started on your machine.
4. Open the command prompt and execute the below docker command:

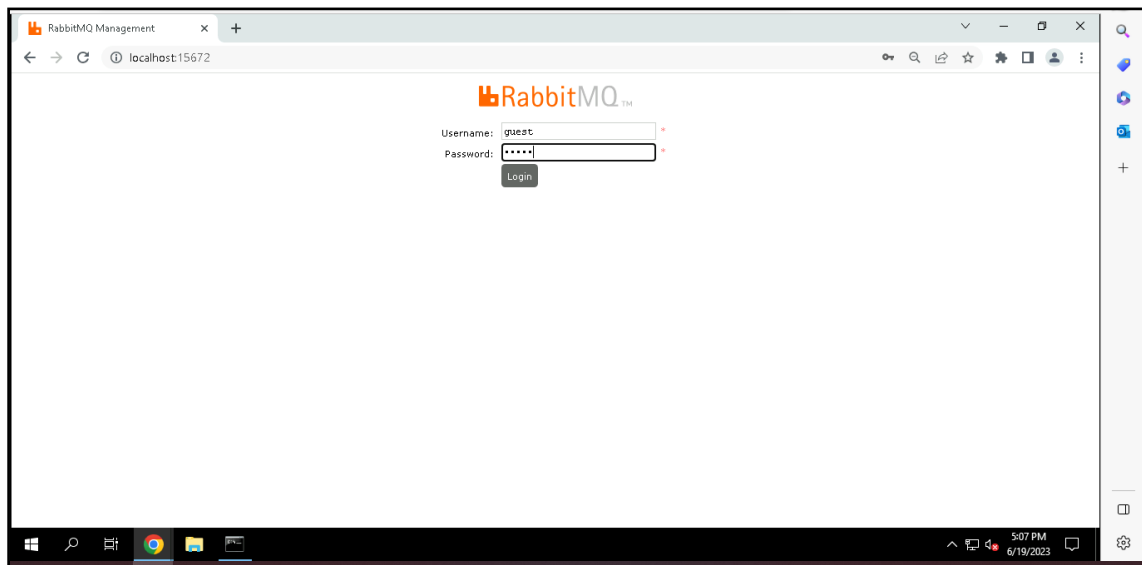
`"docker run -it --rm --name rabbitmq -p 5672:5672 -p 15672:15672 rabbitmq:3.12-management"`



```
Command Prompt - docker run -it --rm --name rabbitmq -p 5672:5672 -p 15672:15672 rabbitmq:3.12-manage...
2023-06-19 17:03:55.463582+00:00 [info] <0.587.0> Resetting node maintenance status
2023-06-19 17:03:55.484802+00:00 [info] <0.646.0> Management plugin: HTTP (non-TLS) listener
started on port 15672
2023-06-19 17:03:55.485155+00:00 [info] <0.674.0> Statistics database started.
2023-06-19 17:03:55.487867+00:00 [info] <0.673.0> Starting worker pool 'management_worker_poo
l' with 3 processes in it
2023-06-19 17:03:55.507340+00:00 [info] <0.688.0> Prometheus metrics: HTTP (non-TLS) listener
started on port 15692
2023-06-19 17:03:55.507667+00:00 [info] <0.587.0> Ready to start client connection listeners
2023-06-19 17:03:55.515389+00:00 [info] <0.732.0> started TCP listener on [::]:5672
completed with 4 plugins.
2023-06-19 17:03:55.698634+00:00 [info] <0.587.0> Server startup complete; 4 plugins started.

2023-06-19 17:03:55.698634+00:00 [info] <0.587.0> * rabbitmq_prometheus
2023-06-19 17:03:55.698634+00:00 [info] <0.587.0> * rabbitmq_management
2023-06-19 17:03:55.698634+00:00 [info] <0.587.0> * rabbitmq_web_dispatch
2023-06-19 17:03:55.698634+00:00 [info] <0.587.0> * rabbitmq_management_agent
```

5. Access the RabbitMQ dashboard via <http://localhost:15672>



6. The default username and password of RabbitMQ is guest.
7. Enhance the **eBookStore-Config-Server** application created in the Problem Statement – 5.
8. Add the Spring Cloud Bus and Actuator dependencies.

```
<dependency>
  <groupId>org.springframework.cloud</groupId>
  <artifactId>spring-cloud-starter-bus-amqp</artifactId>
</dependency>

<dependency>
  <groupId>org.springframework.boot</groupId>
  <artifactId>spring-boot-starter-actuator</artifactId>
</dependency>
```

9. To enable the **/busrefresh** URL endpoint, add the below properties to application.properties:

```
application.properties
1
2 server.port=8888
3 spring.cloud.config.server.git.uri=https://github.com/ManpreetSinghBindra/talentnext
4 #spring.cloud.config.server.git.username=
5 #spring.cloud.config.server.git.password=
6 #spring.cloud.config.server.git.clone-on-start=true
7
8 management.endpoints.web.exposure.include=busrefresh
9
10 spring.rabbitmq.host=localhost
11 spring.rabbitmq.port=5672
12 spring.rabbitmq.username=guest
13 spring.rabbitmq.password=guest
14
```

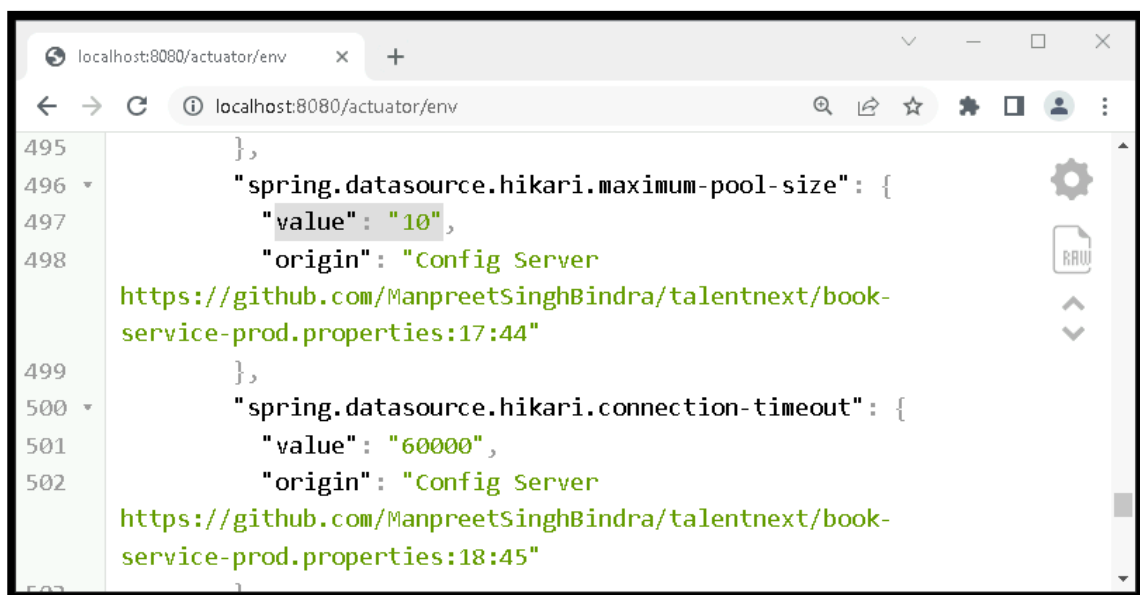
10. Refer the **ebookstoreapp** developed in the problem statement – 9, add the **Spring Cloud Bus** dependency.

```
<dependency>
  <groupId>org.springframework.cloud</groupId>
  <artifactId>spring-cloud-starter-bus-amqp</artifactId>
</dependency>
```

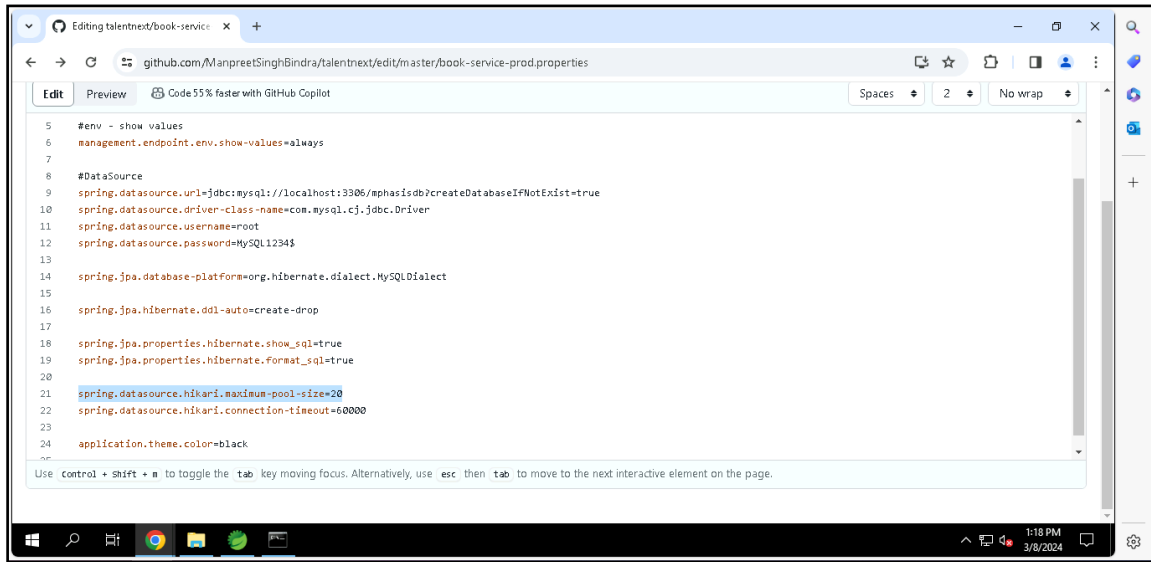
11. Similarly go to each microservices, add the following properties in application.properties file:

```
application.properties
1
2 spring.main.allow-bean-definition-overriding=true
3
4 #Before - Spring Boot 2.4 Version
5 #spring.cloud.config.uri=http://localhost:8888
6
7 #After - Spring Boot 2.4 Version
8 spring.config.import=optional:configserver:http://localhost:8888
9 spring.cloud.config.label=master
10 spring.application.name=book-service
11
12 spring.rabbitmq.host=localhost
13 spring.rabbitmq.port=5672
14 spring.rabbitmq.username=guest
15 spring.rabbitmq.password=guest
16
```

12. Execute the **eBookStore-Config-Server** and **ebookstoreapp** application with **prod** profile.
13. Observe the value of “**spring.datasource.hikari.maximum-pool-size**” attribute which is remotely fetched during the startup of the application using the actuator **env** endpoint.



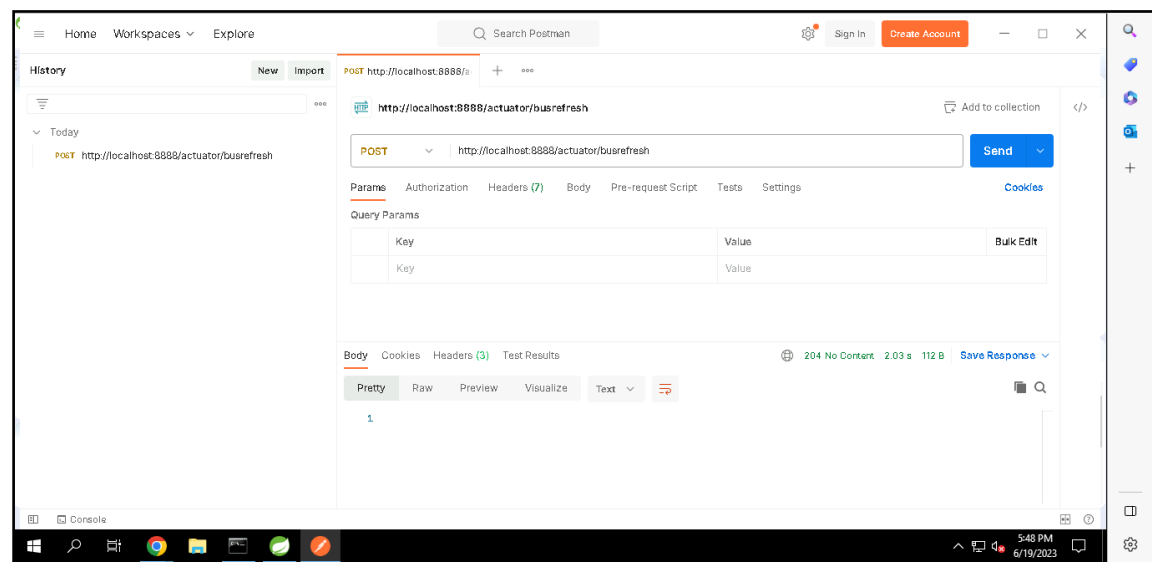
14. Now go to remote git repository and update the value in the “**book-service-prod.properties**”:



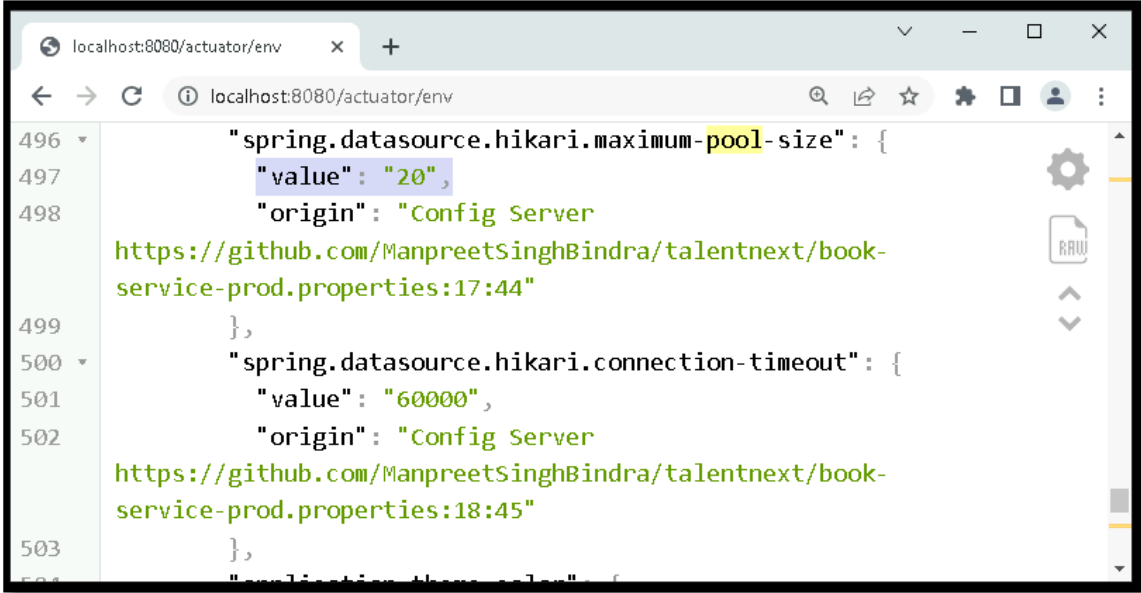
```
5 #env - show values
6 management.endpoint.env.show-values=always
7
8 #DataSource
9 spring.datasource.url=jdbc:mysql://localhost:3306/mphasisdb?createDatabaseIfNotExist=true
10 spring.datasource.driver-class-name=com.mysql.cj.jdbc.Driver
11 spring.datasource.username=root
12 spring.datasource.password=MySQL1234$
13
14 spring.jpa.database-platform=org.hibernate.dialect.MySQLDialect
15
16 spring.jpa.hibernate.ddl-auto=create-drop
17
18 spring.jpa.properties.hibernate.show_sql=true
19 spring.jpa.properties.hibernate.format_sql=true
20
21 spring.datasource.hikari.maximum-pool-size=20
22 spring.datasource.hikari.connection-timeout=60000
23
24 application.theme.color=black
```

15. Click Commit Changes.

16. Let's use the **/actuator/busrefresh** endpoint at Config Server to push the configuration while they are running. The microservices to get these updates do not need to be restart.



17. Once again review the value of “**spring.datasource.hikari.maximum-pool-size**” attribute which is updated while all applications are running.



```
496   "spring.datasource.hikari.maximum-pool-size": {  
497     "value": "20",  
498     "origin": "Config Server  
https://github.com/ManpreetsinghBindra/talentnext/book-  
service-prod.properties:17:44"  
499   },  
500   "spring.datasource.hikari.connection-timeout": {  
501     "value": "60000",  
502     "origin": "Config Server  
https://github.com/ManpreetsinghBindra/talentnext/book-  
service-prod.properties:18:45"  
503   },  
504   "spring.datasource.hikari.maximum-pool-size": {
```

Problem Statement 11: Configure Microservices with Eureka Service Registry Server

The "eBookStore" application instance will expose a remote API such as HTTP/REST at a particular location (host and port). To overcome the challenge of dynamically changing service instances and their locations. The code deployers intended to create a service registry, which is a database containing information about services, their instances, and their locations.

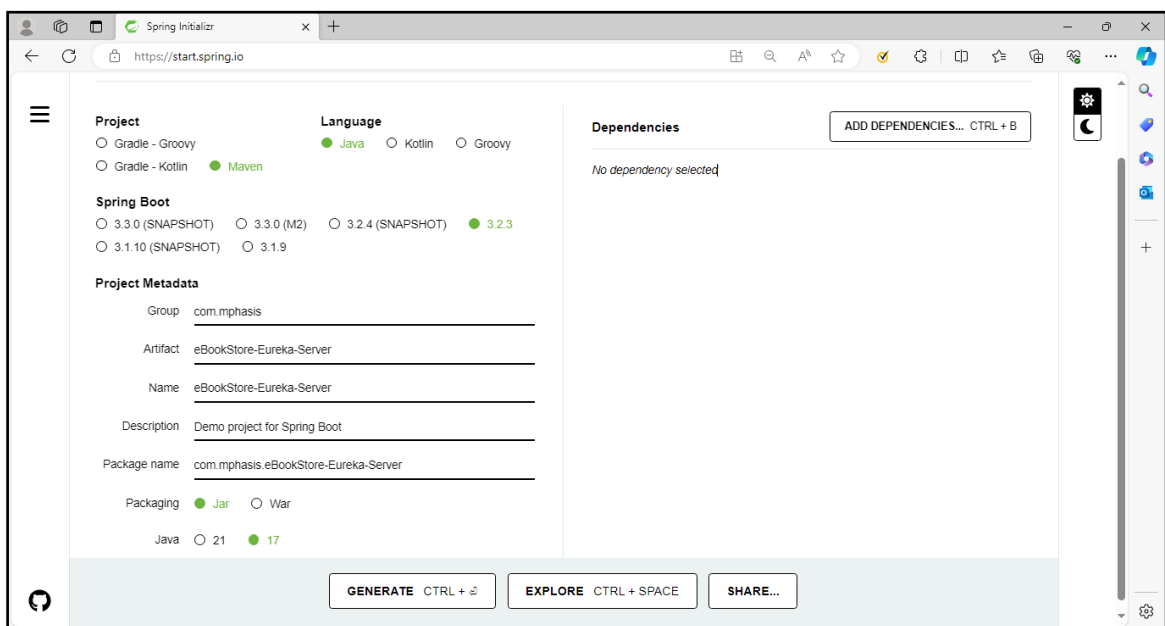
Technology stack:

- Spring Cloud Eureka Server

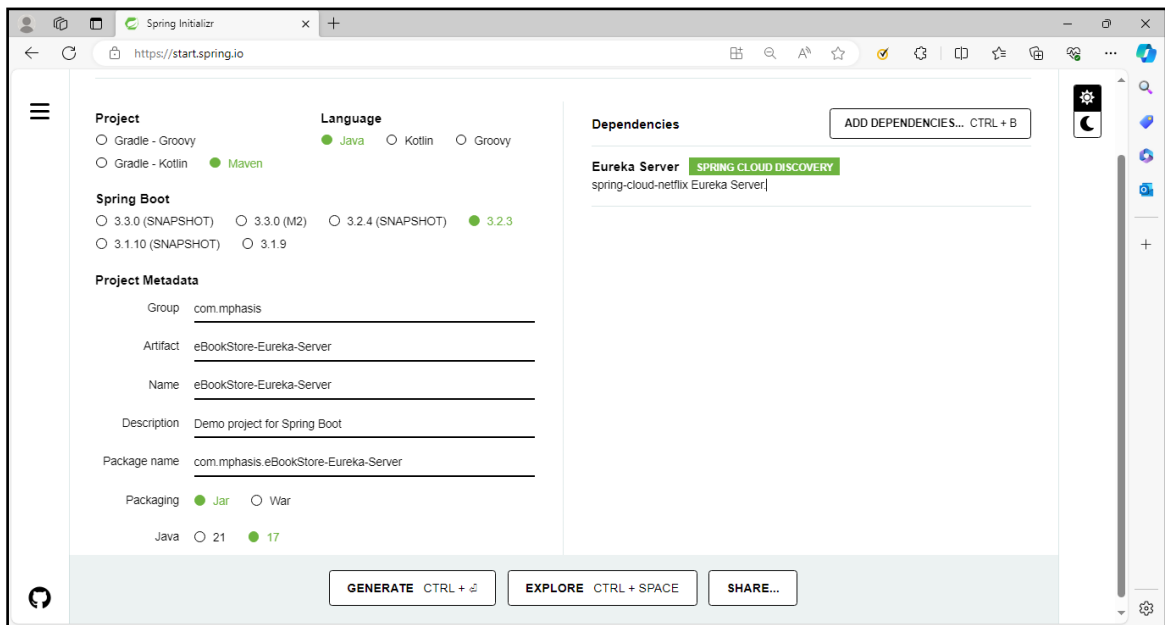
Steps for Spring Cloud Config Server:

1. Create a new Project for **eBookStore-Eureka-Server** using the **Spring Initializr** (start.spring.io).

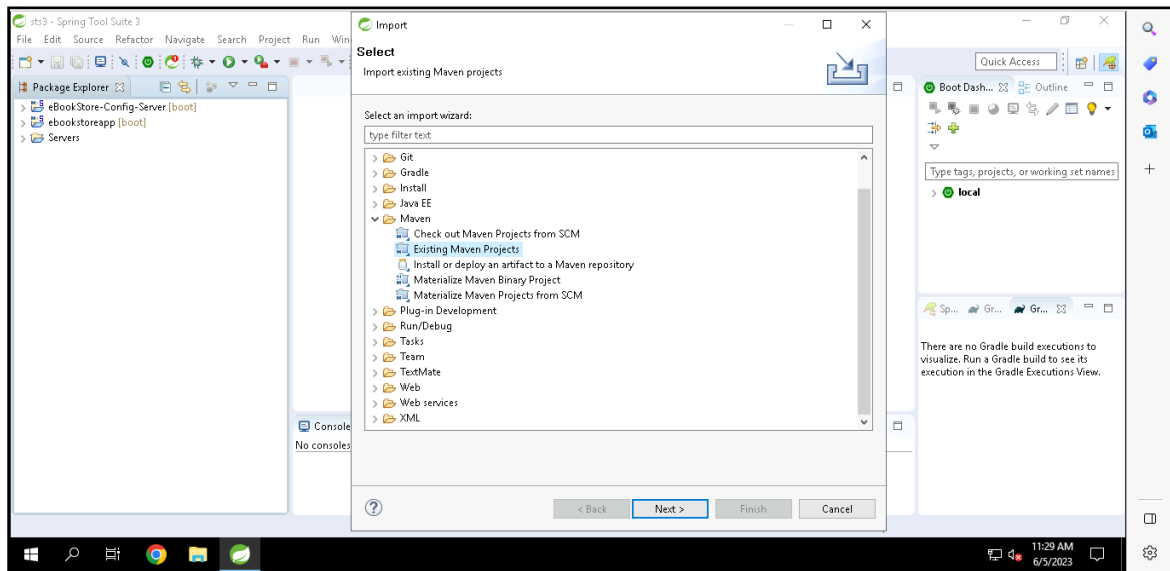
Refer the below screenshots:

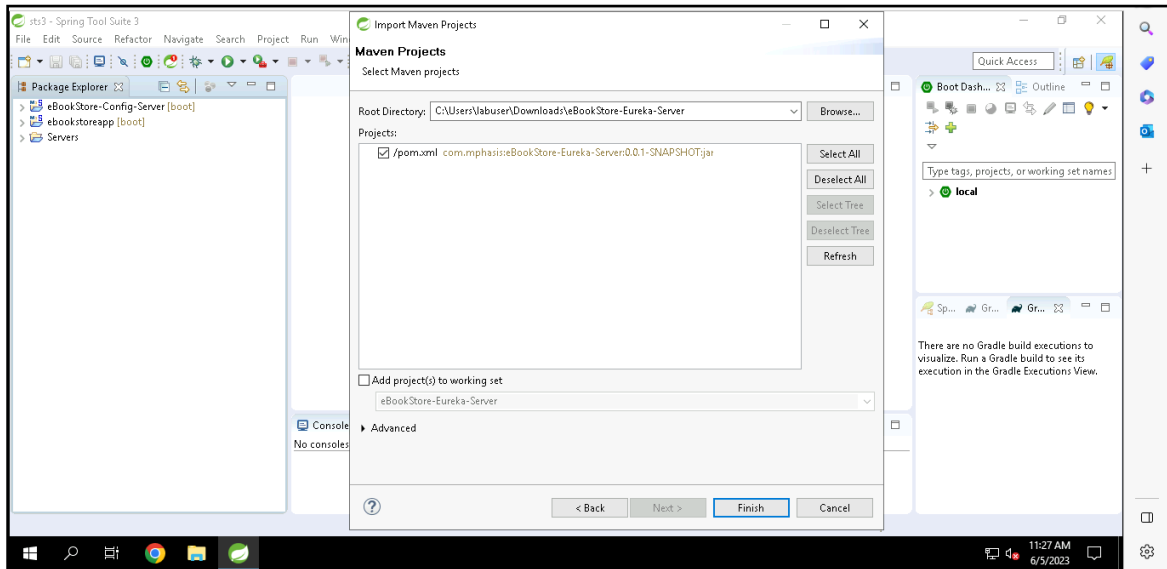


2. Select the below Dependencies:



3. Let's import the eBookStore-Eureka-Server maven project in STS.

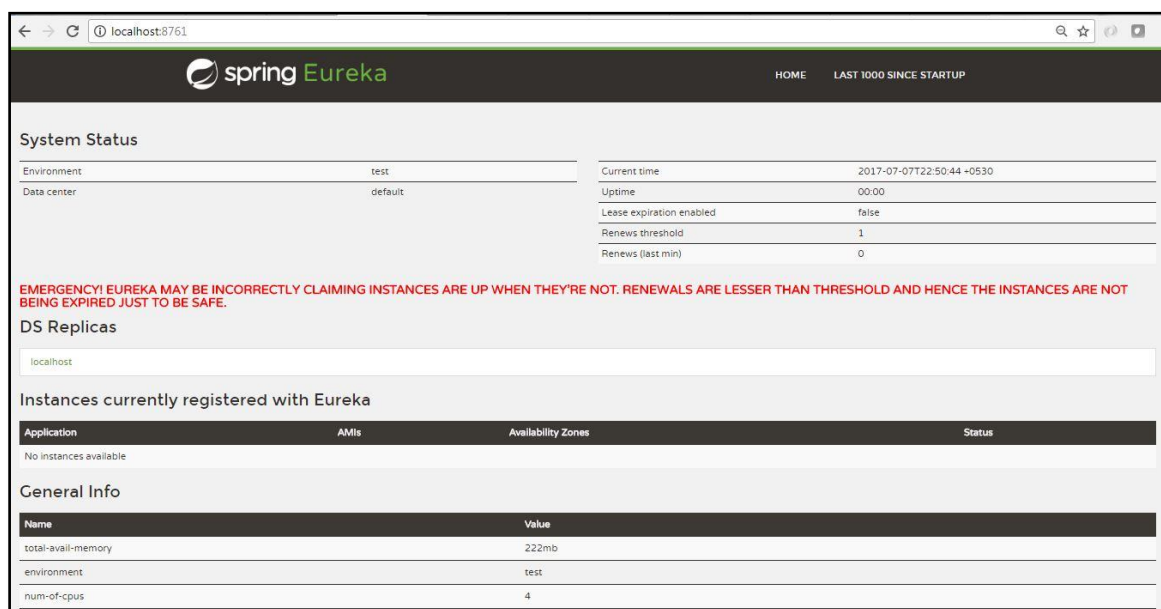




4. Will take some time for the project to be imported and the maven dependencies to be downloaded once you click **Finish**.
5. In the Application class, Add **@EnableEurekaServer** annotation.
6. Ensure the server is running on 8761.

```
application.properties
1 server.port=8761
2 eureka.client.register-with-eureka=false
3 eureka.client.fetch-registry=false
```

7. Start the application.
8. Verify the Eureka Server: <http://localhost:8761/>



Problem Statement 12: Enable Dynamic Registration to eBookStore Microservice

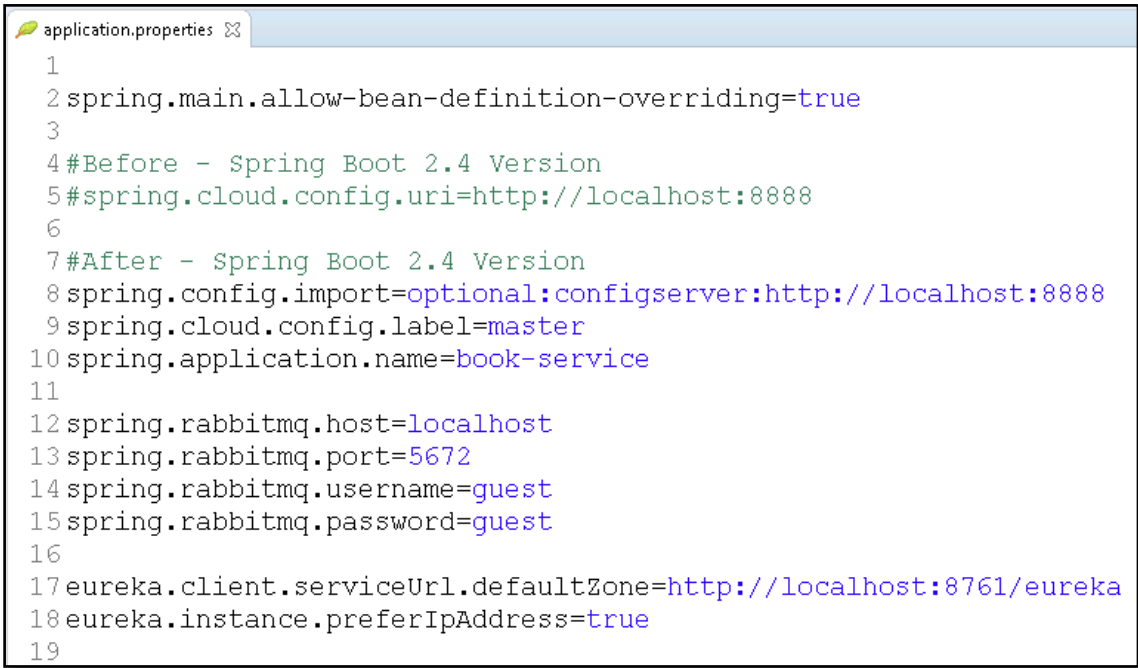
Now we will configure the ebookstoreapp service to register with Spring Cloud Eureka Server.

Technology stack:

- Spring Cloud Eureka Client

Steps for Spring Cloud Config Client:

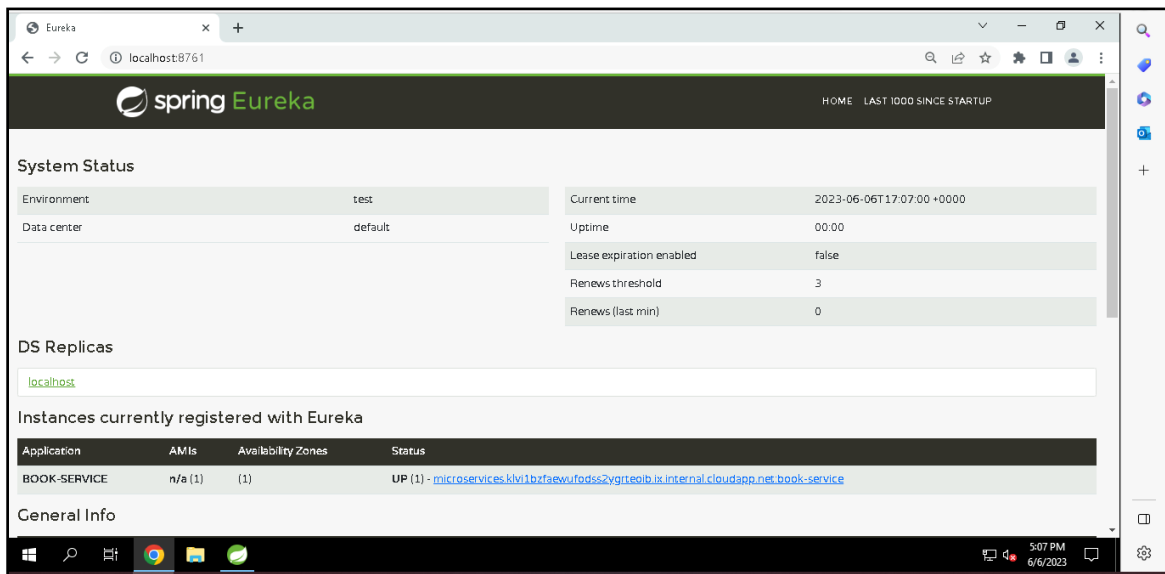
1. Refer the **ebookstoreapp** developed in the problem statement – 10 and add the **Eureka Client** starter to the application.
2. Include the **application name** and **eureka.client.serviceUrl.defaultZone** in the application.properties files. For the eBookStore application to dynamically register to eBookStore-Eureka-Server.

A screenshot of a code editor showing the 'application.properties' file. The file contains 19 lines of configuration. Line 1 is empty. Line 2 sets 'spring.main.allow-bean-definition-overriding=true'. Line 3 is empty. Line 4 is a comment '#Before - Spring Boot 2.4 Version'. Line 5 sets 'spring.cloud.config.uri=http://localhost:8888'. Line 6 is empty. Line 7 is a comment '#After - Spring Boot 2.4 Version'. Line 8 sets 'spring.config.import=optional:configserver:http://localhost:8888'. Line 9 sets 'spring.cloud.config.label=master'. Line 10 sets 'spring.application.name=book-service'. Line 11 is empty. Line 12 sets 'spring.rabbitmq.host=localhost'. Line 13 sets 'spring.rabbitmq.port=5672'. Line 14 sets 'spring.rabbitmq.username=guest'. Line 15 sets 'spring.rabbitmq.password=guest'. Line 16 is empty. Line 17 sets 'eureka.client.serviceUrl.defaultZone=http://localhost:8761/eureka'. Line 18 sets 'eureka.instance.preferIpAddress=true'. Line 19 is empty.

```
1
2 spring.main.allow-bean-definition-overriding=true
3
4 #Before - Spring Boot 2.4 Version
5 spring.cloud.config.uri=http://localhost:8888
6
7 #After - Spring Boot 2.4 Version
8 spring.config.import=optional:configserver:http://localhost:8888
9 spring.cloud.config.label=master
10 spring.application.name=book-service
11
12 spring.rabbitmq.host=localhost
13 spring.rabbitmq.port=5672
14 spring.rabbitmq.username=guest
15 spring.rabbitmq.password=guest
16
17 eureka.client.serviceUrl.defaultZone=http://localhost:8761/eureka
18 eureka.instance.preferIpAddress=true
19
```

3. In the Application class, Add **@EnableDiscoveryClient** annotation.
4. Ensure the RabbitMQ docker container, Config Server, Eureka Server and ebookstoreapp microservice is running.

5. Again, verify the Eureka Server: <http://localhost:8761/>



Problem Statement 13: Enable Dynamic Discovery to eBookStore Consumer Microservice using Ribbon

Based on the business needs, we need to design another API that will make a request to a service. As stated in the previous problem statement, the services are registered in the Service Registry. During the design process, the client (consumer) obtains the location of a service instance by querying a Service Registry, which is aware of the locations of all service instances.

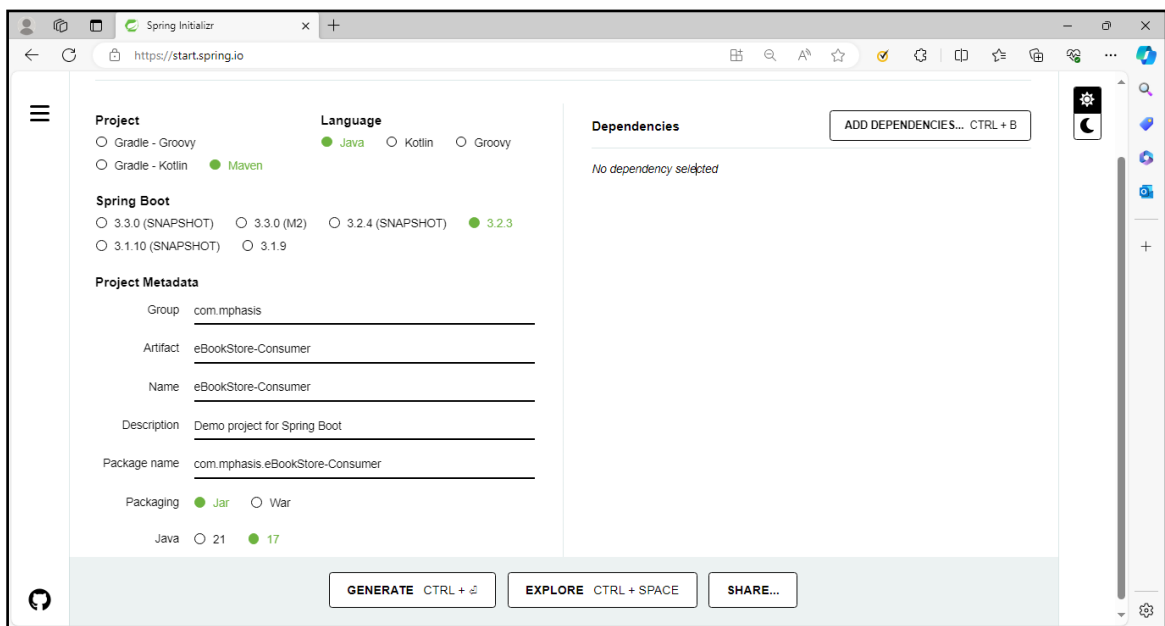
Technology stack:

- Spring Cloud Eureka Discovery Client (/Ribbon)
- Web
- Actuator

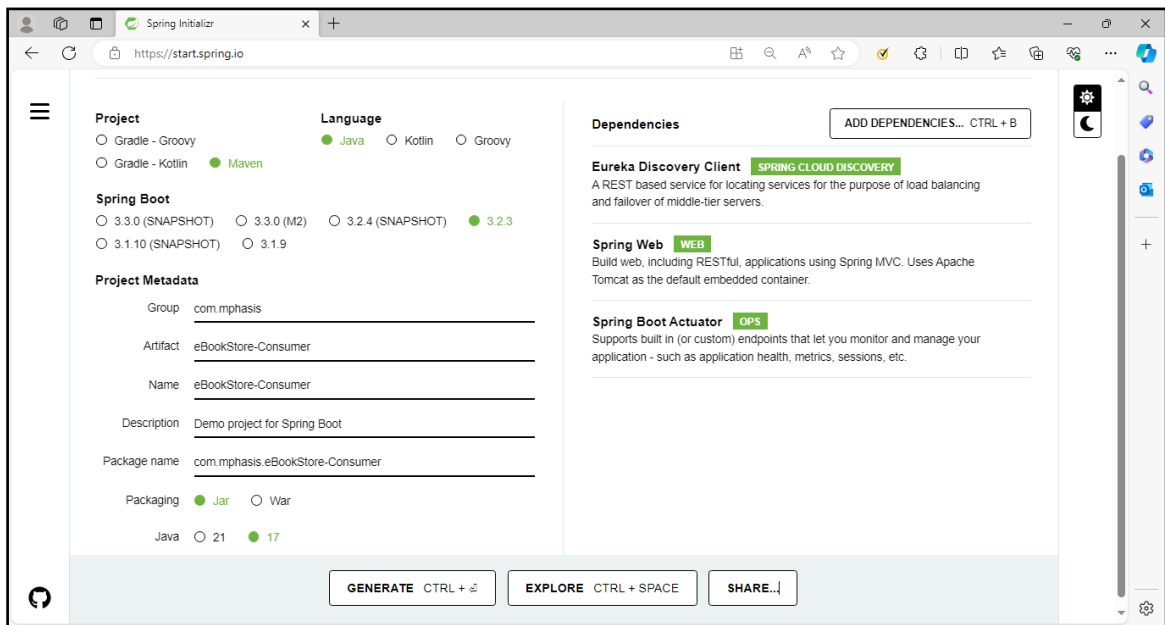
Steps for Client-Side Load Balancing with Ribbon:

1. Create a new Project for **eBookStore-Consumer** using the **Spring Initializr** (start.spring.io).

Refer the below screenshots:



2. Select the below Dependencies:

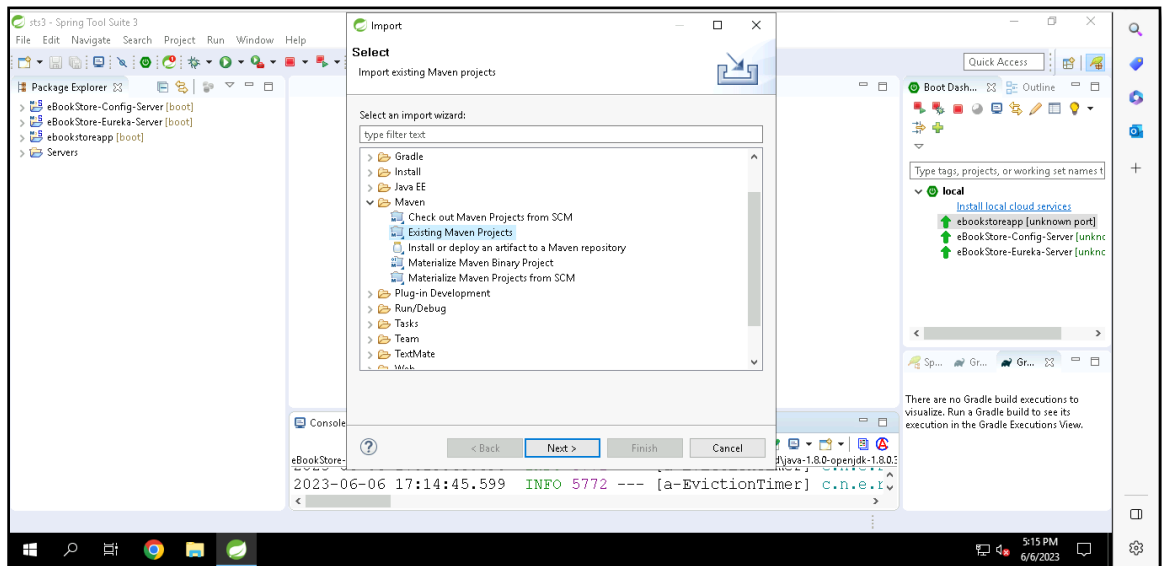


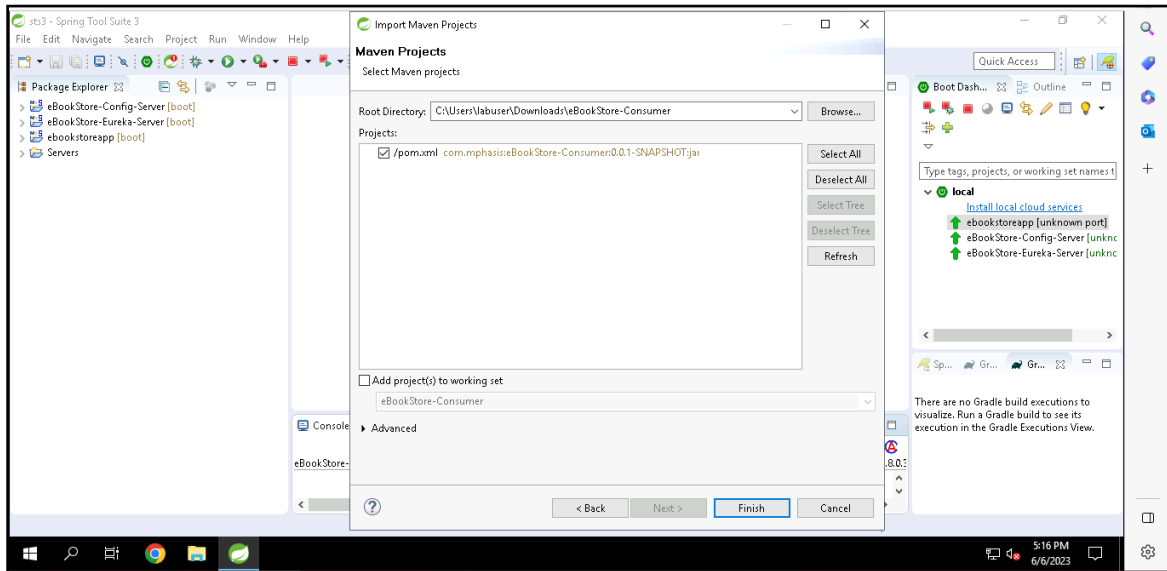
The screenshot shows the Spring Initializr web form at <https://start.spring.io>. The form is configured with the following settings:

- Project:** ☐ Gradle - Groovy, ☒ Gradle - Kotlin, ☒ Maven
- Language:** ☒ Java, ☐ Kotlin, ☐ Groovy
- Spring Boot:** ☐ 3.3.0 (SNAPSHOT), ☐ 3.3.0 (M2), ☐ 3.2.4 (SNAPSHOT), ☒ 3.2.3, ☐ 3.1.10 (SNAPSHOT), ☐ 3.1.9
- Project Metadata:**
 - Group: com.mphasis
 - Artifact: eBookStore-Consumer
 - Name: eBookStore-Consumer
 - Description: Demo project for Spring Boot
 - Package name: com.mphasis.eBookStore-Consumer
 - Packaging: ☒ Jar, ☐ War
 - Java: ☐ 21, ☒ 17
- Dependencies:**
 - Eureka Discovery Client** (SPRING CLOUD DISCOVERY): A REST based service for locating services for the purpose of load balancing and failover of middle-tier servers.
 - Spring Web** (WEB): Build web, including RESTful, applications using Spring MVC. Uses Apache Tomcat as the default embedded container.
 - Spring Boot Actuator** (OPS): Supports built in (or custom) endpoints that let you monitor and manage your application - such as application health, metrics, sessions, etc.

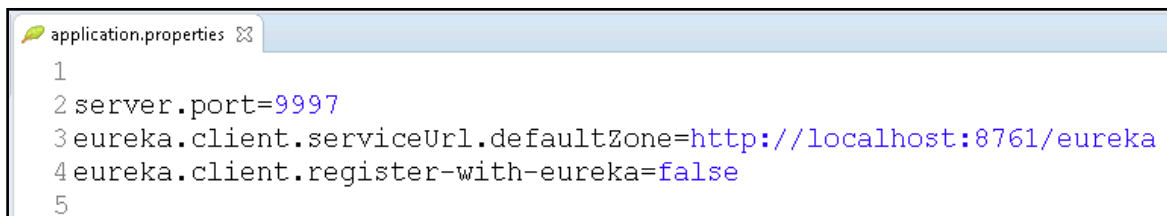
Buttons at the bottom: , ,

3. Let's import the eBookStore-Consumer maven project in STS.

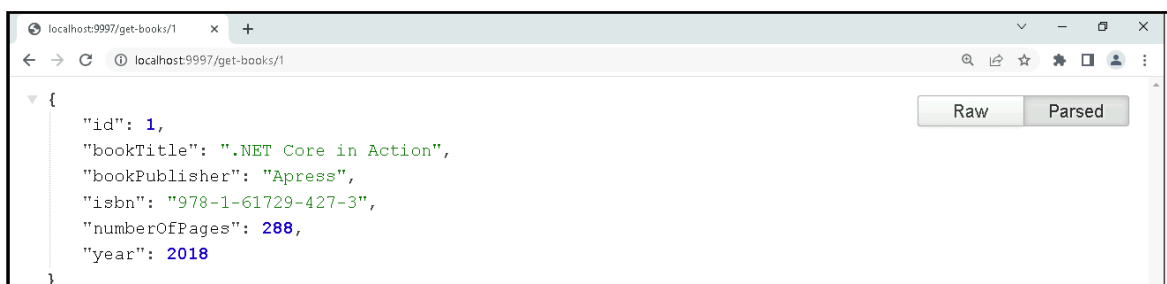




- Will take some time for the project to be imported and the maven dependencies to be downloaded once you click **Finish**.
- Include the **eureka.client.serviceUrl.defaultZone** in the application.properties files. For the eBookStore-Consumer application to dynamic discover the locations of all service instances.
- Ensure the server is running on 9997.



- In RegistryAwareConfig class, create a **RestTemplate** bean annotated with **@LoadBalanced** annotation.
- Create a BookConsumerRestController for consuming the books API (i.e., <http://book-service/books/1> URL) using the **RestTemplate** class.
- Start the eBookStore-Consumer.
- Test the consumer: <http://localhost:9997/get-books/1>



Problem Statement 14: Stabilize the performance in your microservices using Hystrix with Ribbon Client

You have applied the Microservice architecture. Services sometimes collaborate when handling requests. There is always the possibility that one service will synchronously invoke another that is either **unavailable** or has such **high latency** that it is unusable. While waiting for the other service to respond, the caller may use valuable resources such as threads. This might lead to resource exhaustion, which would make the calling service unable to handle other requests. Failure of one service could potentially cascade affect other services throughout the application.

Let us look at how to use Hystrix to prevent a service failure from cascading to other services.

Technology stack:

- Spring Cloud Netflix Hystrix

Implementation approach for Spring Cloud Circuit Breaker:

1. Refer the **eBookStore-Consumer** created in the problem statement – 13 and create a copy of the project with the new name - **eBookStore-Consumer-Hystrix**.
2. Let's downgrade the version of Spring Boot and Spring Cloud in pom.xml for Hystrix implementation.
 - Spring Boot: 2.3.10.RELEASE
 - Spring Cloud: Hoxton.SR11

```
<parent>
  <groupId>org.springframework.boot</groupId>
  <artifactId>spring-boot-starter-parent</artifactId>
  <version>2.3.10.RELEASE</version>
  <relativePath/> <!-- lookup parent from repository -->
</parent>
<groupId>com.mphasis</groupId>
<artifactId>eBookStore-Consumer-Feign</artifactId>
<version>0.0.1-SNAPSHOT</version>
<name>eBookStore-Consumer-Feign</name>
<description>Demo project for Spring Boot Feign Consumer</description>
<properties>
  <java.version>1.8</java.version>
  <spring-cloud.version>Hoxton.SR11</spring-cloud.version>
</properties>
<dependencies>
```

3. Add the **Spring Cloud Netflix Hystrix** starter to the application.

```
<dependency>
  <groupId>org.springframework.cloud</groupId>
  <artifactId>spring-cloud-starter-netflix-hystrix</artifactId>
</dependency>
```


4. Enable the actuator endpoints in the application.properties files.

```
application.properties
1
2 server.port=9995
3 eureka.client.serviceUrl.defaultZone=http://localhost:8761/eureka
4 eureka.client.register-with-eureka=false
5
6 management.endpoints.web.exposure.include=*
7
```

5. Create a **BookService** class and implement your fallback methods and **RestTemplate** call i.e., your business logic. Should be annotated with **@Component/@Service**.
6. Add the **@HystrixCommand(fallbackMethod="fallbackForGetBookById")** annotation on the **getBookById** method of **BookService** class.
7. Now, inside the **BookConsumerRestController** class, you can Autowired the **BookService**.
8. Add **@EnableCircuitBreaker** in the Application class.
9. Start the eBookStore-Consumer-Hystrix
10. Test the consumer: <http://localhost:9995/get-books/1>



```
{
  "id": 1,
  "bookTitle": ".NET Core in Action",
  "bookPublisher": "Apress",
  "isbn": "978-1-61729-427-3",
  "numberOfPages": 288,
  "year": 2018
}
```

11. Just to verify the hystrix is working or not, stop running the ebookstoreapp and refresh the browser. You will find the calls are delegated to fallback methods.

Problem Statement 15: Configuring Circuit Breaker: Hystrix Dashboard to Ribbon Client

One of the primary advantages of Hystrix is the collection of metrics about each HystrixCommand. The Hystrix Dashboard efficiently displays the status of each circuit breaker. Allow us to enable monitoring capabilities in microservices.

Technology stack:

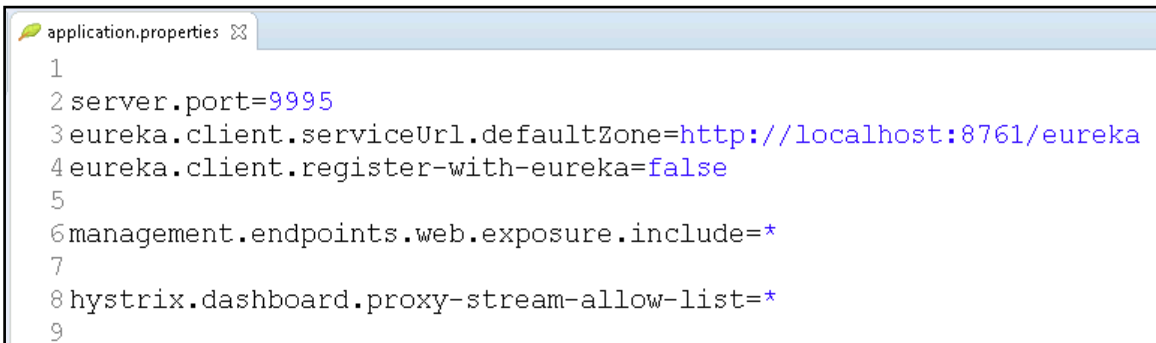
- Spring Cloud Netflix Hystrix Dashboard

Implementation approach for Spring Cloud Circuit Breaker:

1. Refer the **eBookStore-Consumer-Hystrix** created in the problem statement – 14 and add the **Spring Cloud Netflix Hystrix Dashboard** starter to the application.

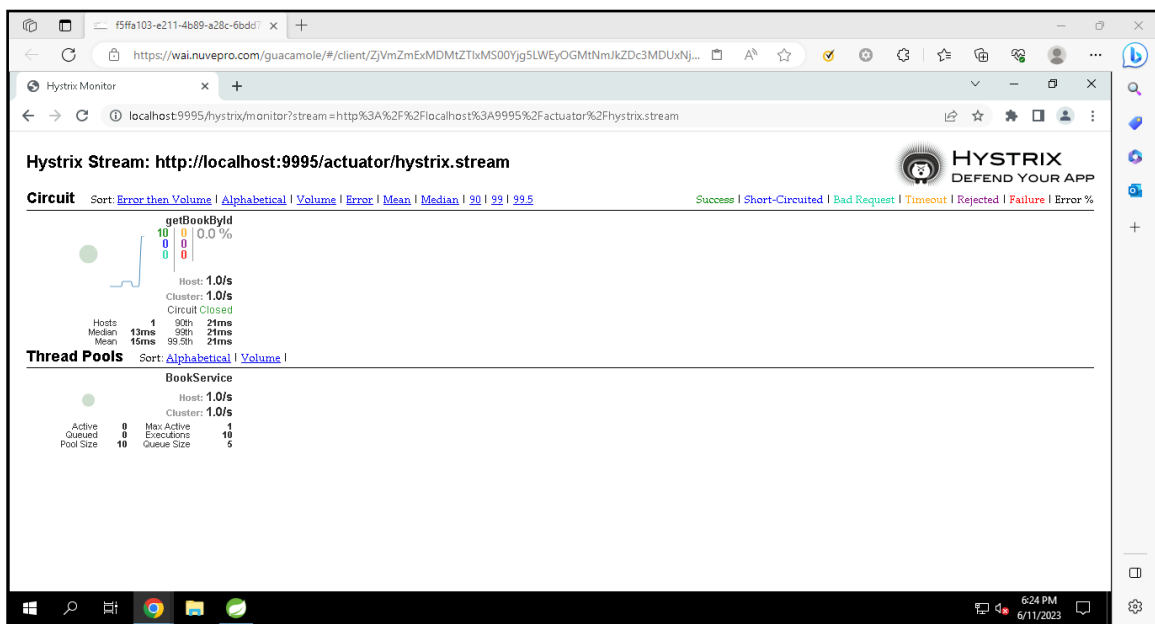
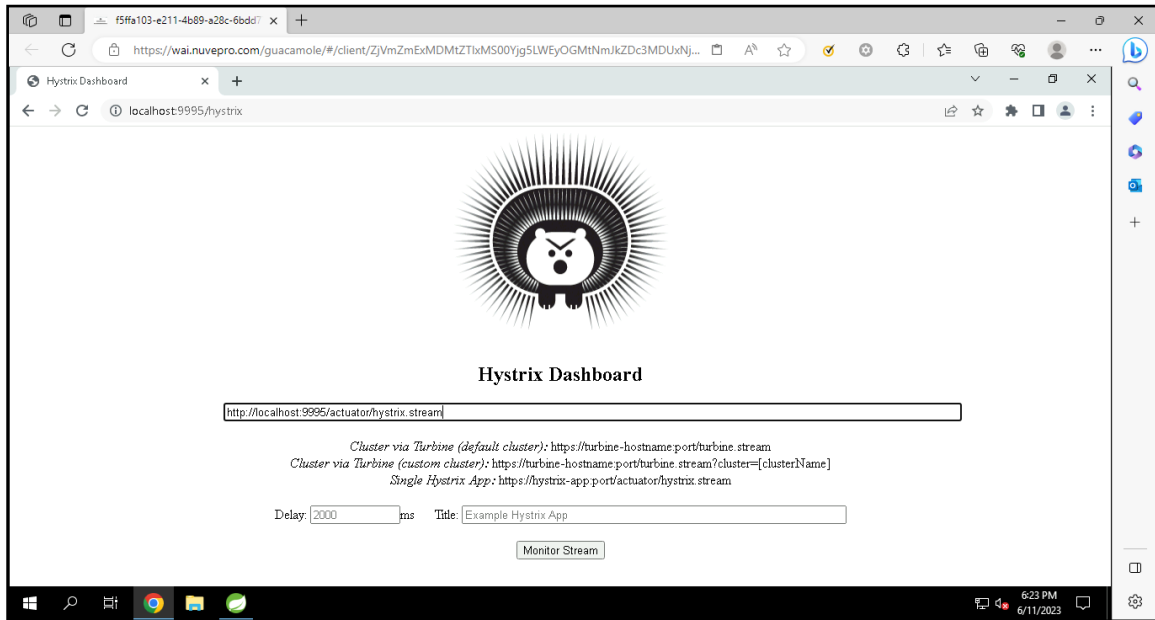
```
<dependency>
  <groupId>org.springframework.cloud</groupId>
  <artifactId>spring-cloud-starter-netflix-hystrix</artifactId>
</dependency>
<dependency>
  <groupId>org.springframework.cloud</groupId>
  <artifactId>spring-cloud-starter-netflix-hystrix-dashboard</artifactId>
</dependency>
```

2. Add **@EnableHystrixDashboard** in the Application class.
3. Enable the **hystrix.dashboard.proxy-stream-allow-list** in the application.properties files.

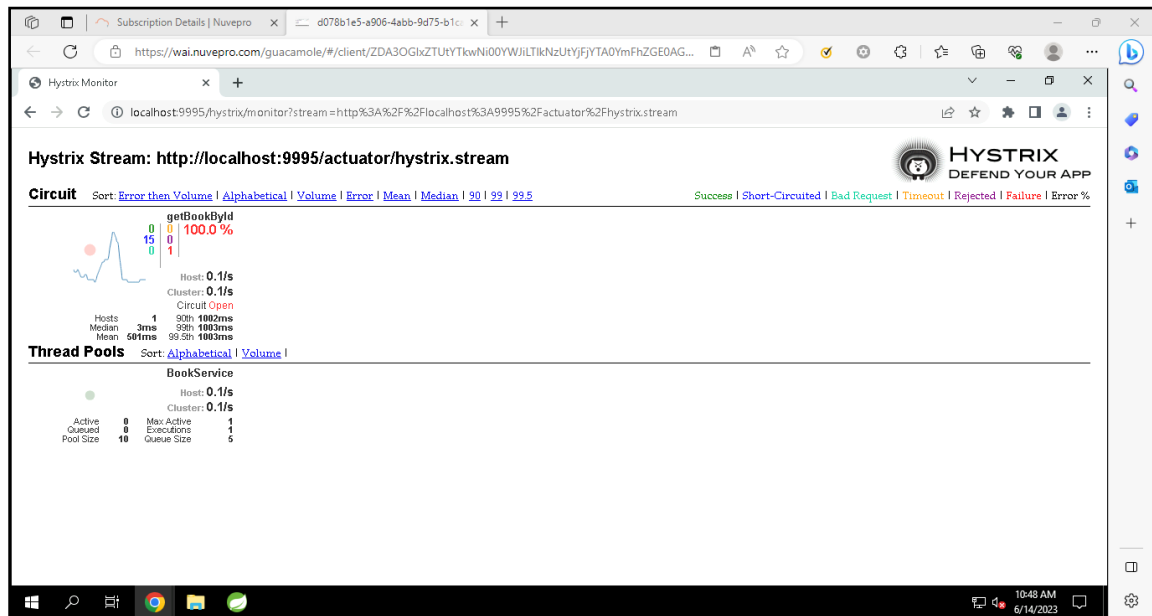


```
application.properties
1
2 server.port=9995
3 eureka.client.serviceUrl.defaultZone=http://localhost:8761/eureka
4 eureka.client.register-with-eureka=false
5
6 management.endpoints.web.exposure.include=*
7
8 hystrix.dashboard.proxy-stream-allow-list=*
9
```

4. After we have done restarting the application, we will point a browser at <http://localhost:9995/hystrix>, input the metrics URL of a Hystrix stream (i.e., <http://localhost:9995/actuator/hystrix.stream>) and begin monitoring.



Java Microservices – S1 – Practice Exercise



Problem Statement 16: Enable Dynamic Discovery to eBookStore Consumer Microservice using Feign

Let us use Feign to simplify the process of designing the consumer API.

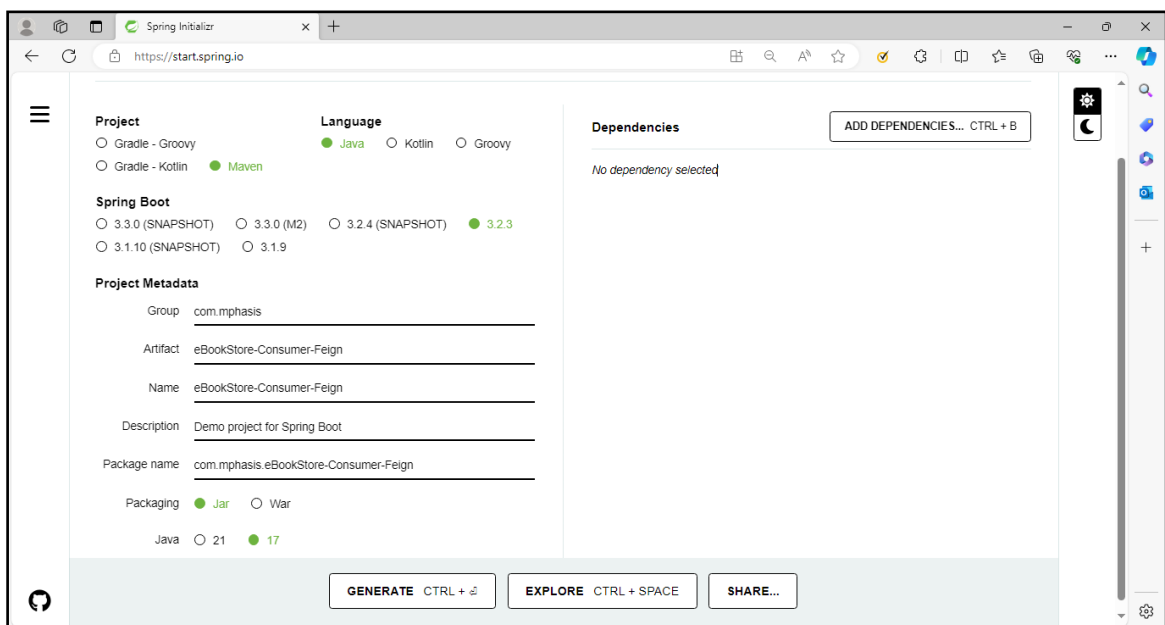
Technology stack:

- Spring Cloud Eureka Discovery Client (/Ribbon)
- OpenFeign
- Spring Web
- Spring Boot Actuator

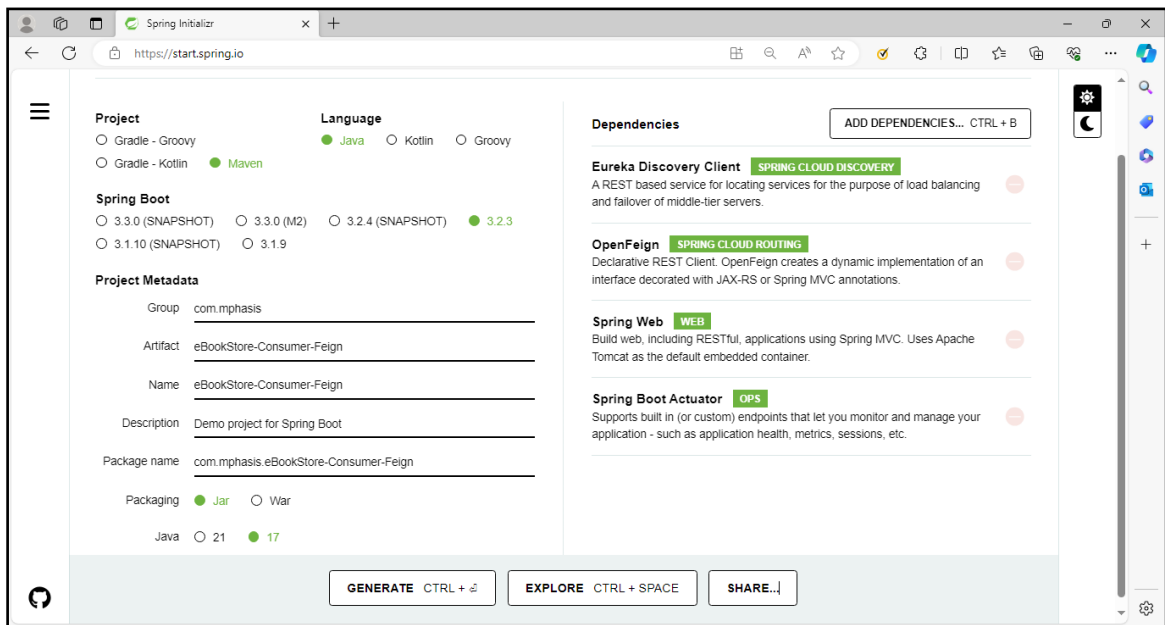
Steps for eBookStore consumer using Feign:

1. Create a new Project for **eBookStore-Consumer-Feign** using the **Spring Initializr** (start.spring.io).

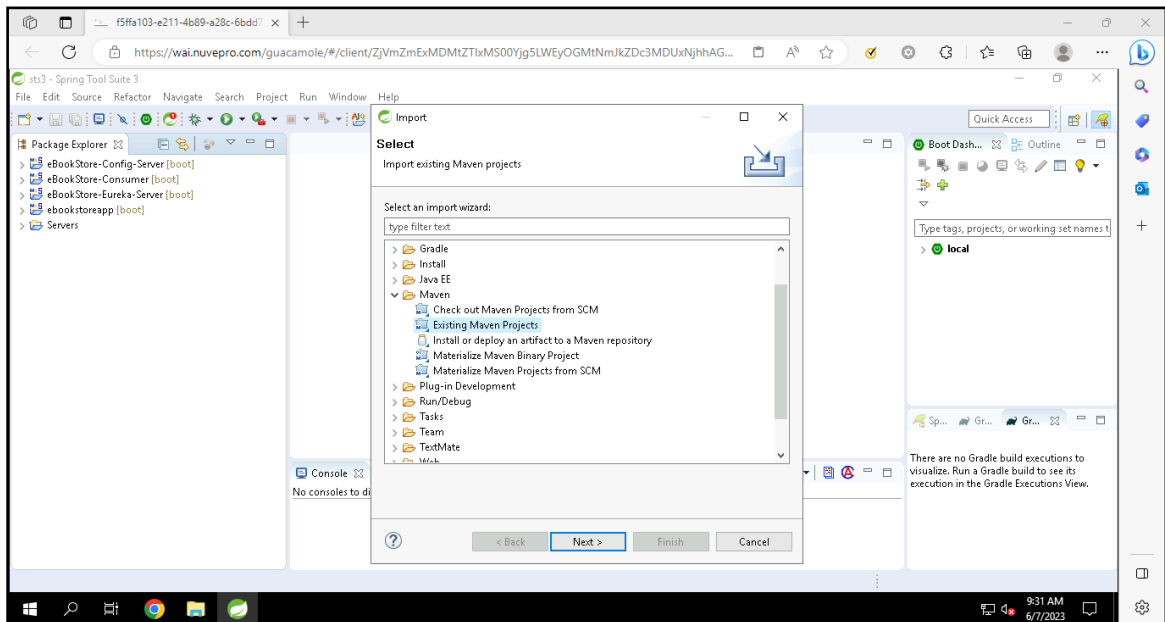
Refer the below screenshots:

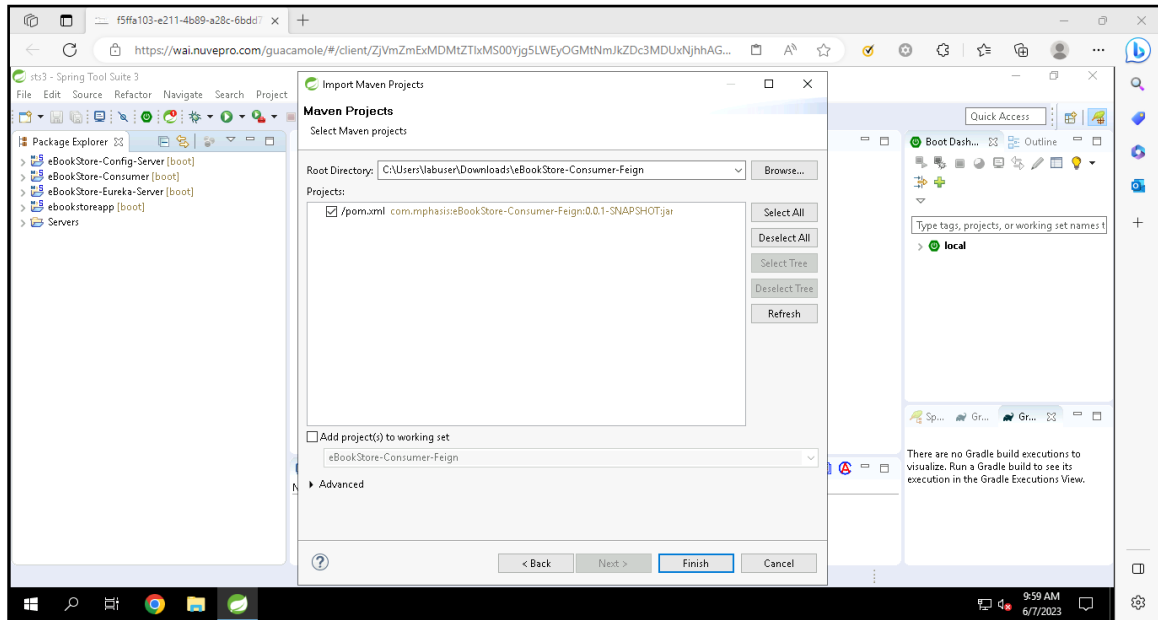


2. Select the below Dependencies:



3. Let's import the eBookStore-Consumer-Feign maven project in STS.





- Will take some time for the project to be imported and the maven dependencies to be downloaded once you click **Finish**.
- Include the **eureka.client.serviceUrl.defaultZone** in the application.properties files. For the eBookStore-Consumer-Feign application to dynamic discover the locations of all service instances.
- Ensure the server is running on 9998.

```
application.properties
1
2 server.port=9998
3 eureka.client.serviceUrl.defaultZone=http://localhost:8761/eureka
4 eureka.client.register-with-eureka=false
5
```

- Create a BookServiceProxy Interface using **@FeignClient** annotation.

```
@FeignClient("book-service")
public interface BookServiceProxy {

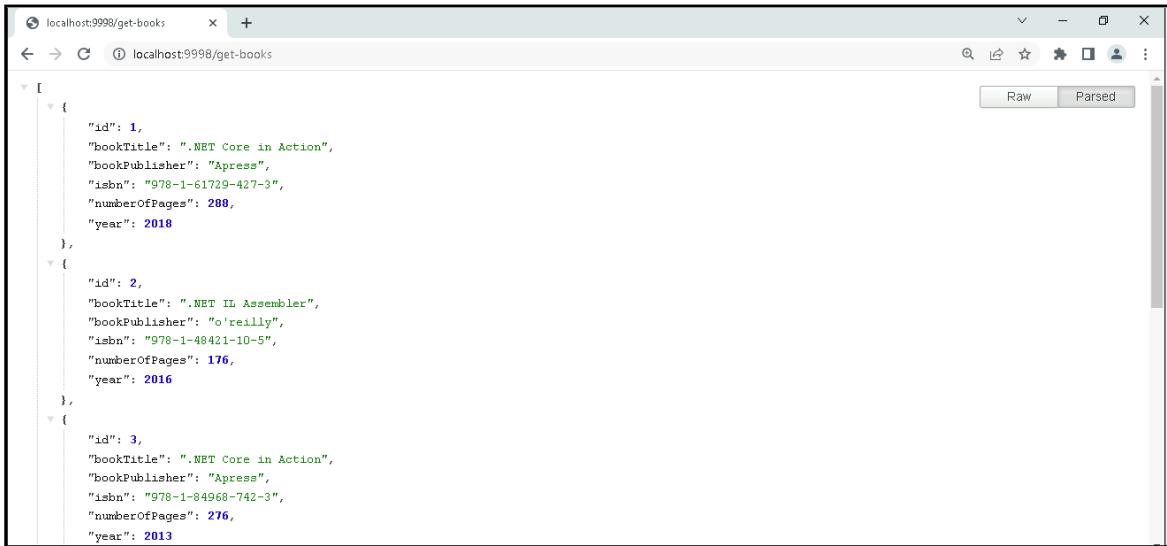
    @GetMapping(value = "/books/{id}", produces = {MediaType.APPLICATION_JSON_VALUE})
    public Book getBookById(@PathVariable("id") Integer id);

    @GetMapping(value = "/books", produces = {MediaType.APPLICATION_JSON_VALUE})
    public List<Book> getAllBooks();

}
```

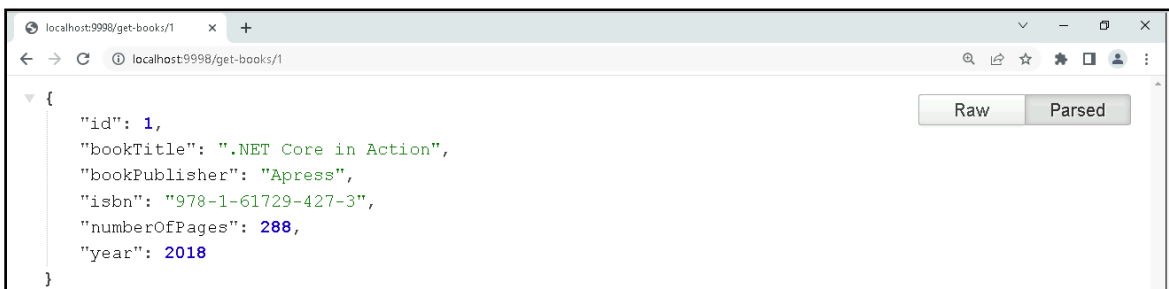
- Create a BookConsumerRestController for consuming the books API (i.e., <http://book-service/books> and <http://book-service/books/1> URL) using the **BookServiceProxy** interface.
- Add **@EnableFeignClients** in the Application class.
- Start the eBookStore-Consumer-Feign.

11. Test the consumer: <http://localhost:9998/get-books>



```
[{"id": 1, "bookTitle": ".NET Core in Action", "bookPublisher": "Apress", "isbn": "978-1-61729-427-3", "numberOfPages": 288, "year": 2018}, {"id": 2, "bookTitle": ".NET IL Assembler", "bookPublisher": "o'reilly", "isbn": "978-1-48421-10-5", "numberOfPages": 176, "year": 2016}, {"id": 3, "bookTitle": ".NET Core in Action", "bookPublisher": "Apress", "isbn": "978-1-84968-742-3", "numberOfPages": 276, "year": 2013}]
```

12. Test the consumer: <http://localhost:9998/get-books/1>



```
{ "id": 1, "bookTitle": ".NET Core in Action", "bookPublisher": "Apress", "isbn": "978-1-61729-427-3", "numberOfPages": 288, "year": 2018 }
```


Problem Statement 17: Stabilize the performance in your microservices using Hystrix with Feign Client

You have applied the Microservice architecture. Services sometimes collaborate when handling requests. There is always the possibility that one service will synchronously invoke another that is either **unavailable** or has such **high latency** that it is unusable. While waiting for the other service to respond, the caller may use valuable resources such as threads. This might lead to resource exhaustion, which would make the calling service unable to handle other requests. Failure of one service could potentially cascade affect other services throughout the application.

Let us look at how to use Hystrix to prevent a service failure from cascading to other services.

Technology stack:

- Spring Cloud Netflix Hystrix

Implementation approach for Spring Cloud Circuit Breaker:

1. Refer the **eBookStore-Consumer-Feign** created in the problem statement – 16 and create a copy of the project with the new name - **eBookStore-Consumer-Feign-Hystrix**.
2. Let's downgrade the version of Spring Boot and Spring Cloud in pom.xml for Hystrix implementation.
 - Spring Boot: 2.3.10.RELEASE
 - Spring Cloud: Hoxton.SR11

```
<parent>
  <groupId>org.springframework.boot</groupId>
  <artifactId>spring-boot-starter-parent</artifactId>
  <version>2.3.10.RELEASE</version>
  <relativePath/> <!-- lookup parent from repository -->
</parent>
<groupId>com.mphasis</groupId>
<artifactId>eBookStore-Consumer-Feign</artifactId>
<version>0.0.1-SNAPSHOT</version>
<name>eBookStore-Consumer-Feign</name>
<description>Demo project for Spring Boot Feign Consumer</description>
<properties>
  <java.version>1.8</java.version>
  <spring-cloud.version>Hoxton.SR11</spring-cloud.version>
</properties>
<dependencies>
```

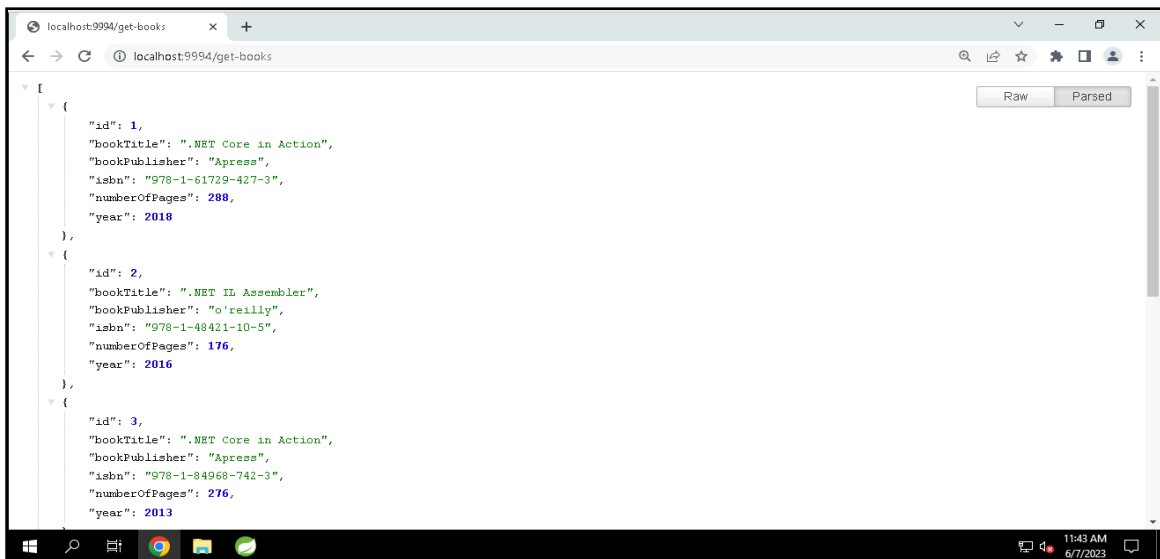
3. Add the **Spring Cloud Netflix Hystrix** starter to the application.

```
<dependency>
  <groupId>org.springframework.cloud</groupId>
  <artifactId>spring-cloud-starter-netflix-hystrix</artifactId>
</dependency>
```

4. Enable the **feign.hystrix.enabled** and all actuator endpoints in the application.properties files.

```
application.properties
1
2 server.port=9994
3 eureka.client.serviceUrl.defaultZone=http://localhost:8761/eureka
4 eureka.client.register-with-eureka=false
5
6 #Enable Actuator
7 management.endpoints.web.exposure.include=*
8
9 feign.hystrix.enabled=true
10
```

5. Create a **BookServiceFallback** class and implement your fallback methods i.e., your business logic. Should be annotated with **@Component/@Service**.
6. Add the fallback class to BookServiceProxy Interface using **@FeignClient(name = "book-service", fallback=BookServiceFallback.class)** annotation.
7. Add **@EnableFeignClients** and **@EnableCircuitBreaker** in the Application class.
8. Start the eBookStore-Consumer-Feign-Hystrix.
9. Test the consumer: <http://localhost:9994/get-books>



```
[{"id": 1, "bookTitle": ".NET Core in Action", "bookPublisher": "Apress", "isbn": "978-1-61729-427-3", "numberOfPages": 288, "year": 2018}, {"id": 2, "bookTitle": ".NET IL Assembler", "bookPublisher": "O'Reilly", "isbn": "978-1-48421-10-5", "numberOfPages": 176, "year": 2016}, {"id": 3, "bookTitle": ".NET Core in Action", "bookPublisher": "Apress", "isbn": "978-1-84968-742-3", "numberOfPages": 276, "year": 2013}]
```

10. Test the consumer: <http://localhost:9994/get-books/1>



```
{"id": 1, "bookTitle": ".NET Core in Action", "bookPublisher": "Apress", "isbn": "978-1-61729-427-3", "numberOfPages": 288, "year": 2018}
```

11. Just to verify the hystrix is working or not, stop running the ebookstoreapp and refresh the browser. You will find the calls are delegated to fallback methods.

Problem Statement 18: Configuring Circuit Breaker: Hystrix Dashboard to Feign Client

One of the primary advantages of Hystrix is the collection of metrics about each HystrixCommand. The Hystrix Dashboard efficiently displays the status of each circuit breaker. Allow us to enable monitoring capabilities in microservices.

Technology stack:

- Spring Cloud Netflix Hystrix Dashboard

Implementation approach for Spring Cloud Circuit Breaker:

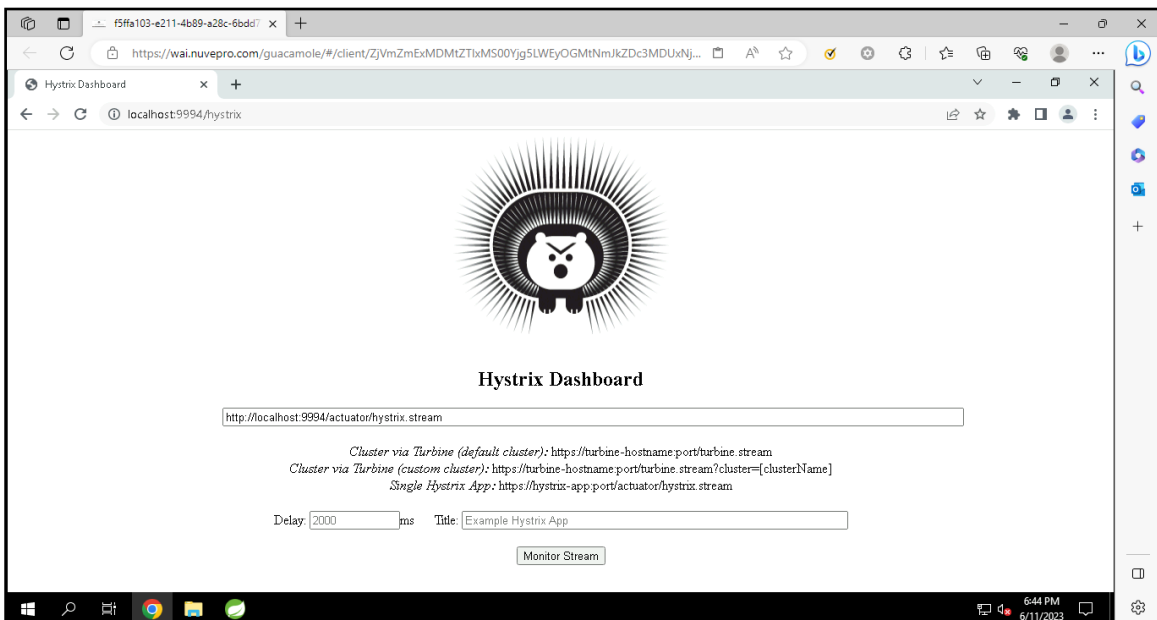
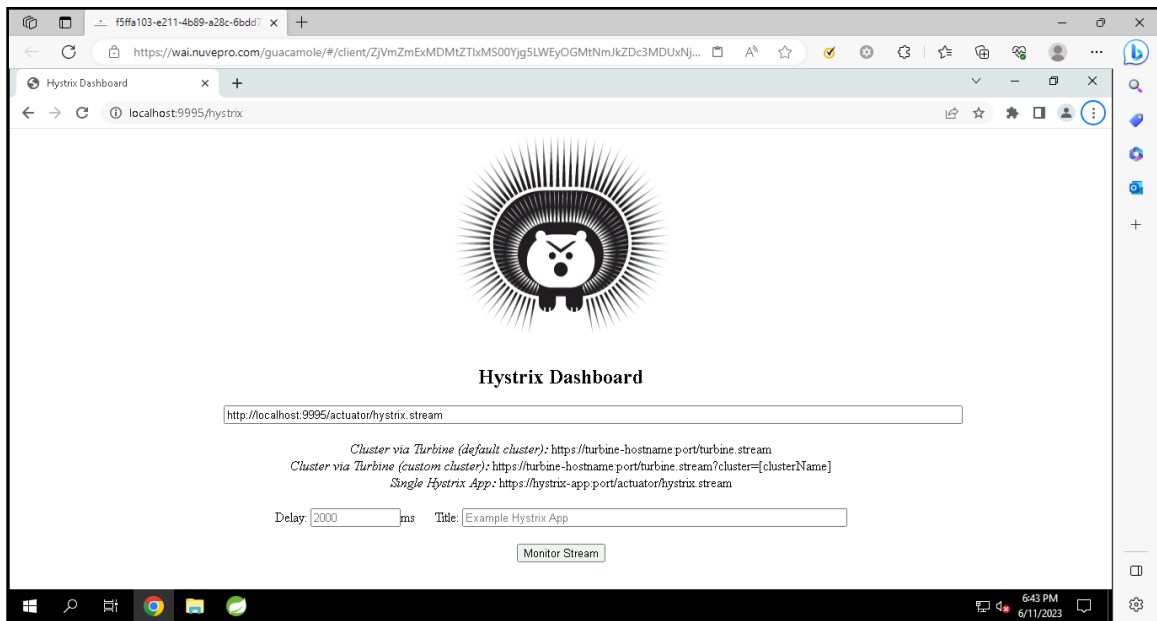
1. Refer the **eBookStore-Consumer-Feign-Hystrix** created in the problem statement – 17 and add the **Spring Cloud Netflix Hystrix Dashboard** starter to the application.

```
<dependency>
  <groupId>org.springframework.cloud</groupId>
  <artifactId>spring-cloud-starter-netflix-hystrix</artifactId>
</dependency>
<dependency>
  <groupId>org.springframework.cloud</groupId>
  <artifactId>spring-cloud-starter-netflix-hystrix-dashboard</artifactId>
</dependency>
```

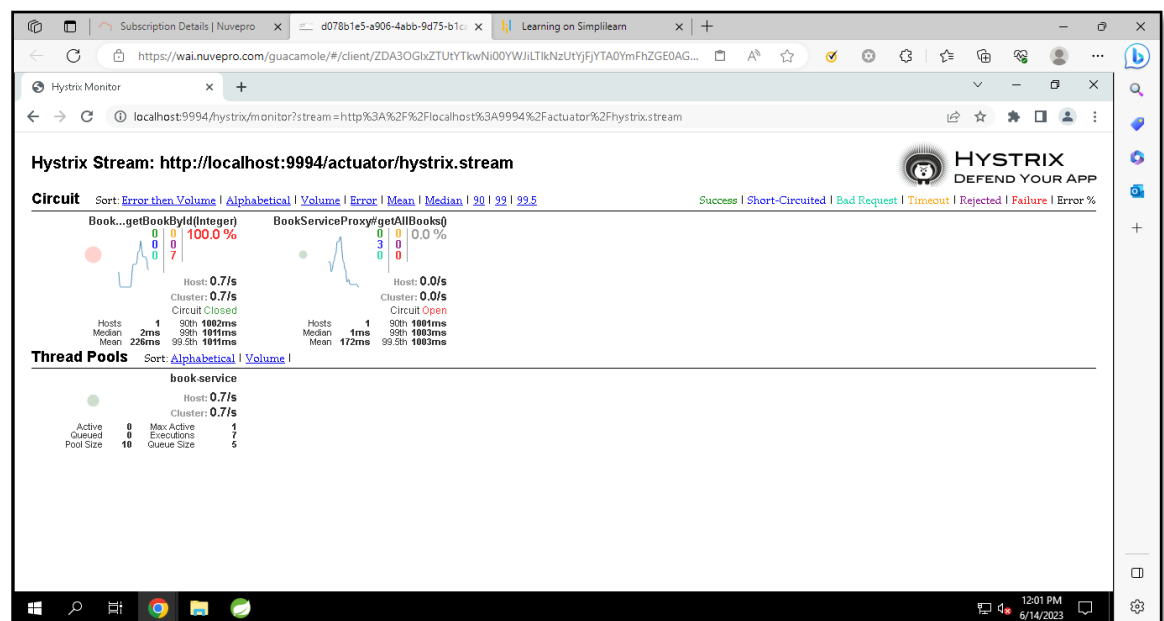
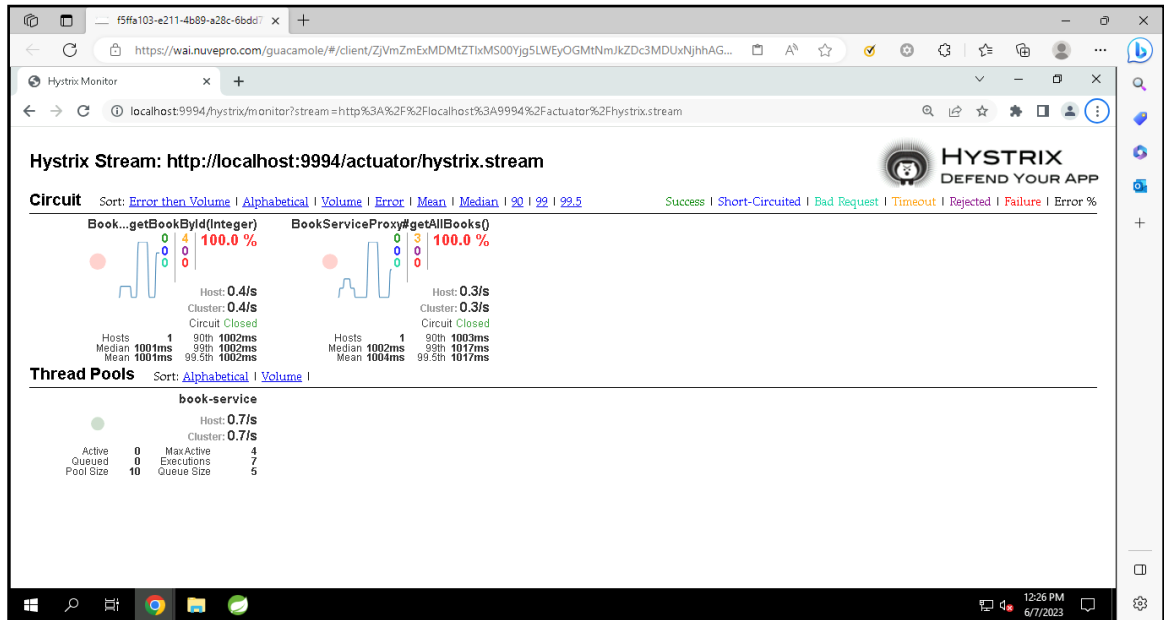
2. Add **@EnableHystrixDashboard** in the Application class.
3. Enable the **hystrix.dashboard.proxy-stream-allow-list** in the application.properties files.

```
application.properties
1
2 server.port=9994
3 eureka.client.serviceUrl.defaultZone=http://localhost:8761/eureka
4 eureka.client.register-with-eureka=false
5
6 #Enable Actuator
7 management.endpoints.web.exposure.include=*
8
9 feign.hystrix.enabled=true
10
11 hystrix.dashboard.proxy-stream-allow-list=*
12
```

- After we have done restarting the application, we will point a browser at <http://localhost:9994/hystrix>, input the metrics URL of a Hystrix stream (i.e., <http://localhost:9994/actuator/hystrix.stream>) and begin monitoring.



Java Microservices – S1 – Practice Exercise



Problem Statement 19: Spring Cloud Circuit Breaker, Adding Fault Tolerance to Your Ribbon Client Applications

In the new era, organization are planning to adopt an abstraction for the different circuit breaker implementations. It provides a consistent API to use in your applications allowing the developer to select the circuit breaker implementation that best meets your app's needs.

Let us look at how to use Resilience4j with Ribbon Client to prevent a service failure from cascading to other services.

Technology stack:

- Spring Cloud Circuit Breaker - Resilience4j

Implementation approach for Spring Cloud Circuit Breaker:

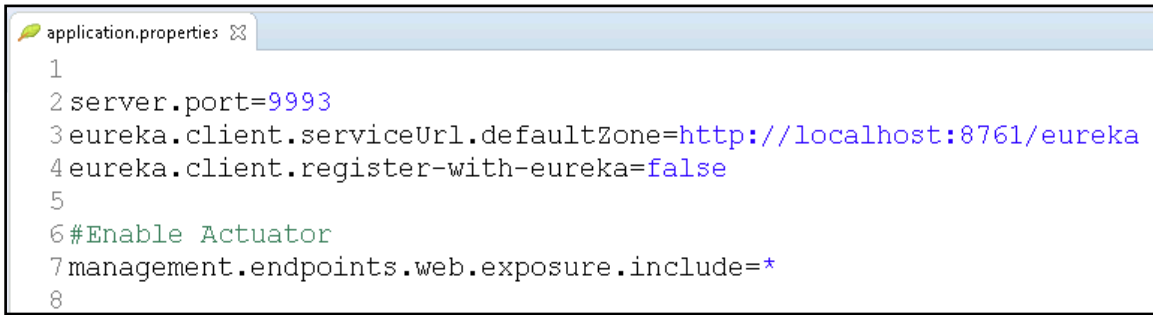
1. Refer the **eBookStore-Consumer** created in the problem statement – 13 and create a copy of the project with the new name - **eBookStore-Consumer-Resilience4J**.
2. Let's use the latest version of Spring Boot and Spring Cloud in pom.xml for Resilience4J implementation.
 - Spring Boot: 3.2.3
 - Spring Cloud: 2023.0.0

```
<parent>
  <groupId>org.springframework.boot</groupId>
  <artifactId>spring-boot-starter-parent</artifactId>
  <version>3.2.3</version>
  <relativePath /> <!-- lookup parent from repository -->
</parent>
<groupId>com.mphasis</groupId>
<artifactId>eBookStore-Consumer</artifactId>
<version>0.0.1-SNAPSHOT</version>
<name>eBookStore-Consumer</name>
<description>Demo project for Spring Boot Consumer</description>
<properties>
  <java.version>17</java.version>
  <spring-cloud.version>2023.0.0</spring-cloud.version>
</properties>
```

- Add the below **Spring Cloud Circuit Breaker - Resilience4J** starter to the application.

```
<dependency>
  <groupId>org.springframework.boot</groupId>
  <artifactId>spring-boot-starter-actuator</artifactId>
</dependency>
<dependency>
  <groupId>org.springframework.boot</groupId>
  <artifactId>spring-boot-starter-aop</artifactId>
</dependency>
<dependency>
  <groupId>org.springframework.cloud</groupId>
  <artifactId>spring-cloud-starter-circuitbreaker-resilience4j</artifactId>
</dependency>
```

3. Enable the actuator endpoints in the application.properties files.



```
1
2 server.port=9993
3 eureka.client.serviceUrl.defaultZone=http://localhost:8761/eureka
4 eureka.client.register-with-eureka=false
5
6 #Enable Actuator
7 management.endpoints.web.exposure.include=*
8
```

4. Create a **BookService** class and implement your fallback methods and RestTemplate call i.e., your business logic.
5. Add the **@CircuitBreaker(name="book-service", fallbackMethod="fallbackForGetBookById")** annotation on the **getBookById** method of **BookService** class.
6. Now, inside the **BookConsumerRestController** class, you can Autowired the **BookService**.
7. Start the eBookStore-Consumer-Resilience4J microservice.
8. Test the consumer: <http://localhost:9993/get-books/2>



```
{
  "id": 2,
  "bookTitle": ".NET IL Assembler",
  "bookPublisher": "o'reilly",
  "isbn": "978-1-48421-10-5",
  "numberOfPages": 176,
  "year": 2016
}
```

9. Just to verify the Resilience4J is working or not, stop running the ebookstoreapp and refresh the browser. You will find the calls are delegated to fallback methods.

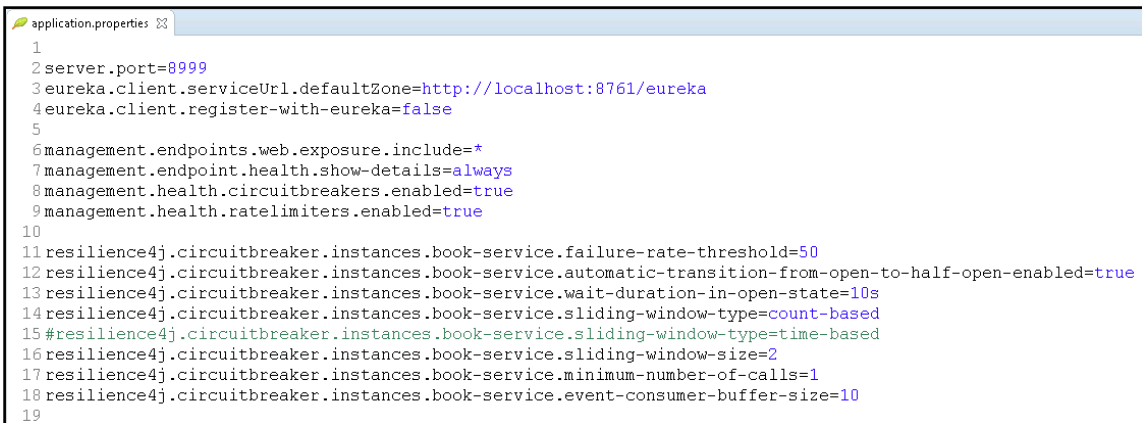
Problem Statement 20: Configuring Circuit Breaker and Retry with Spring Cloud Resilience4J to Your Ribbon Client Applications

With the growing number of services, services may need to communicate with other servers synchronously, making them dependent on the upstream service. Any issues encountered while communicating with upstream services will be propagated to the downstream services.

To prevent services from such problems, we can use some of the patterns to protect the service. So, today, we'll look at two of them: the Circuit Breaker and the Retry mechanism.

Implementation approach for Circuit Breaker and Retry mechanism:

1. Refer the **eBookStore-Consumer-Resilience4J** created in the problem statement – 19.
2. Let's add the configuration for the circuit breaker and health endpoints via the Spring Boot Actuator in the application.properties file:

A screenshot of a code editor showing the application.properties file. The file contains 19 lines of configuration for a Spring Boot application. It sets the server port to 8999, Eureka client details, management endpoints, and Resilience4J circuit breaker settings for the book-service.

```
1
2server.port=8999
3eureka.client.serviceUrl.defaultZone=http://localhost:8761/eureka
4eureka.client.register-with-eureka=false
5
6management.endpoints.web.exposure.include=*
7management.endpoint.health.show-details=always
8management.health.circuitbreakers.enabled=true
9management.health.ratelimiters.enabled=true
10
11resilience4j.circuitbreaker.instances.book-service.failure-rate-threshold=50
12resilience4j.circuitbreaker.instances.book-service.automatic-transition-from-open-to-half-open-enabled=true
13resilience4j.circuitbreaker.instances.book-service.wait-duration-in-open-state=10s
14resilience4j.circuitbreaker.instances.book-service.sliding-window-type=count-based
15#resilience4j.circuitbreaker.instances.book-service.sliding-window-type=time-based
16resilience4j.circuitbreaker.instances.book-service.sliding-window-size=2
17resilience4j.circuitbreaker.instances.book-service.minimum-number-of-calls=1
18resilience4j.circuitbreaker.instances.book-service.event-consumer-buffer-size=10
19
```

3. Ensure the RabbitMQ docker container, Config Server, Eureka Server and ebookstoreapp microservice is running.
4. Start the eBookStore-Consumer-Resilience4J microservice.
5. Test the consumer: <http://localhost:8999/get-books/3>

A screenshot of a web browser window showing the response to a GET request at localhost:8999/get-books/3. The response is a JSON object containing book details.

```
1 // 20230623121338
2 // http://localhost:8999/get-books/3
3
4 {
5   "id": 3,
6   "bookTitle": "101 UX Principles",
7   "bookPublisher": "Apress",
8   "isbn": "978-1-78883-736-1",
9   "page": 414,
10  "year": 2013
11 }
```

6. Access the endpoint: <http://localhost:8999/actuator/circuitbreakerevents> which will list by default the latest 100 emitted events of all the CircuitBreaker instances.


```
localhost:8999/actuator/circuitbreakerevents
1 // 20230623122014
2 // http://localhost:8999/actuator/circuitbreakerevents
3
4 {
5   "circuitBreakerEvents": [
6   ]
7 }
8 }
```

```
localhost:8999/actuator/circuitbreakerevents
1 // 20230623122110
2 // http://localhost:8999/actuator/circuitbreakerevents
3
4 {
5   "circuitBreakerEvents": [
6   {
7     "circuitBreakerName": "book-service",
8     "type": "SUCCESS",
9     "creationTime": "2023-06-23T12:21:07.811Z[UTC]",
10    "errorMessage": null,
11    "durationInMs": 160,
12    "stateTransition": null
13  }
14 ]
15 }
```

7. Just to verify the Resilience4J Circuit Breaker mechanism is working or not, stop running the ebookstoreapp and refresh the browser. You will find the calls are delegated to fallback methods. Also, review the Circuit Breaker Events.
8. Test the consumer – Refresh Thrice: <http://localhost:8999/get-books/3>.
9. Observe the different types of **STATE_TRANSITION** via Circuit Breaker Events.
10. Start the ebookstoreapp, wait for 90 secs and then check the circuit breaker events – state transition.
11. Let's configure the Resilience4J Retry mechanism to our spring boot application properties files.

```
20 resilience4j.retry.instances.book-service.max-attempts=3
21 resilience4j.retry.instances.book-service.wait-duration=2s
```

12. Now, it may happen that retrying after a fixed time could cause the upstream service to further not respond (probably it's already overwhelmed with many requests). In this case, we can provide an exponential back-off mechanism. Let's add this config.

```
23 resilience4j.retry.instances.book-service.enable-exponential-backoff=true
24 resilience4j.retry.instances.book-service.exponential-backoff-multiplier=5
```

13. Also change the default order of aspects, by giving the retry module a higher priority than circuit breaker.

```
26 resilience4j.circuitbreaker.circuit-breaker-aspect-order=1
27 resilience4j.retry.retry-aspect-order=2
```

14. The overall configuration of Circuit Breaker and Retry mechanism.

```
10
11 resilience4j.circuitbreaker.circuit-breaker-assert-order=1
12
13 resilience4j.circuitbreaker.instances.book-service.failure-rate-threshold=50
14 resilience4j.circuitbreaker.instances.book-service.automatic-transition-from-open-to-half-open-enabled=true
15 resilience4j.circuitbreaker.instances.book-service.wait-duration-in-open-state=10s
16 resilience4j.circuitbreaker.instances.book-service.sliding-window-type=count-based
17 #resilience4j.circuitbreaker.instances.book-service.sliding-window-type=time-based
18 resilience4j.circuitbreaker.instances.book-service.sliding-window-size=2
19 resilience4j.circuitbreaker.instances.book-service.minimum-number-of-calls=1
20 resilience4j.circuitbreaker.instances.book-service.event-consumer-buffer-size=10
21
22 resilience4j.retry.retry-assert-order=2
23
24 resilience4j.retry.instances.book-service.max-attempts=3
25 resilience4j.retry.instances.book-service.wait-duration=2s
26
27 resilience4j.retry.instances.book-service.enable-exponential-backoff=true
28 resilience4j.retry.instances.book-service.exponential-backoff-multiplier=5
29
```

15. Add the **@Retry** annotation to the BookService class.



```
14 @Service(value = "bookService")
15 public class BookService {}
16
17 @Autowired
18 private RestTemplate restTemplate;
19
20 @Retry(name = "book-service")
21 @CircuitBreaker(name="book-service", fallbackMethod = "fallbackForGetBookById")
22 public Book getBookById(@PathVariable("id") Integer id) {
23     Book book = null;
24     try {
25         book = restTemplate.getForObject("http://book-service/books/" + id, Book.class);
26     } catch (Exception ex) {
27         ex.printStackTrace();
28         throw ex;
29     }
30     return book;
31 }
32
33 public Book fallbackForGetBookById(Integer id, Exception ex) {
34     System.out.println("Exception raised with message :==>" + ex.getMessage());
35     return new Book(id, ".Net Core in Action", "Apress", "978-61729-427-3", 288, 2018);
36 }
```

16. Restart the eBookStore-Consumer-Resilience4J microservice.

17. Test the consumer: <http://localhost:8999/get-books/3>



```
1 // 20230623121338
2 // http://localhost:8999/get-books/3
3
4 {
5   "id": 3,
6   "bookTitle": "101 UX Principles",
7   "bookPublisher": "Apress",
8   "isbn": "978-1-78883-736-1",
9   "page": 414,
10  "year": 2013
11 }
```

18. Just to verify the Retry mechanism is working or not, stop running the ebookstoreapp and refresh the browser. You will find 3 times the exception stack trace and compare the timestamp of each request.

Problem Statement 21: Spring Cloud Circuit Breaker, Adding Fault Tolerance to Your Feign Client Applications

Let's see how Resilience4j with Feign Client can be used to prevent a service failure from cascading to other services.

Technology stack:

- Spring Cloud Circuit Breaker - Resilience4J

Implementation approach for Spring Cloud Circuit Breaker:

1. Refer the **eBookStore-Consumer-Feign** created in the problem statement – 16 and create a copy of the project with the new name - **eBookStore-Consumer-Feign-Resilience4J**.
2. Let's use the latest version of Spring Boot and Spring Cloud in pom.xml for Resilience4J implementation.
 - Spring Boot: 3.2.3
 - Spring Cloud: 2023.0.0

```
<parent>
  <groupId>org.springframework.boot</groupId>
  <artifactId>spring-boot-starter-parent</artifactId>
  <version>3.2.3</version>
  <relativePath /> <!-- lookup parent from repository -->
</parent>
<groupId>com.mphasis</groupId>
<artifactId>eBookStore-Consumer-Feign</artifactId>
<version>0.0.1-SNAPSHOT</version>
<name>eBookStore-Consumer-Feign</name>
<description>Demo project for Spring Boot Feign Consumer</description>
<properties>
  <java.version>17</java.version>
  <spring-cloud.version>2023.0.0</spring-cloud.version>
</properties>
```

3. Add the below **Spring Cloud Circuit Breaker - Resilience4J** starter to the application.

```
<dependency>
  <groupId>org.springframework.boot</groupId>
  <artifactId>spring-boot-starter-actuator</artifactId>
</dependency>
<dependency>
  <groupId>org.springframework.boot</groupId>
  <artifactId>spring-boot-starter-aop</artifactId>
</dependency>
<dependency>
  <groupId>org.springframework.cloud</groupId>
  <artifactId>spring-cloud-starter-circuitbreaker-resilience4j</artifactId>
</dependency>
```

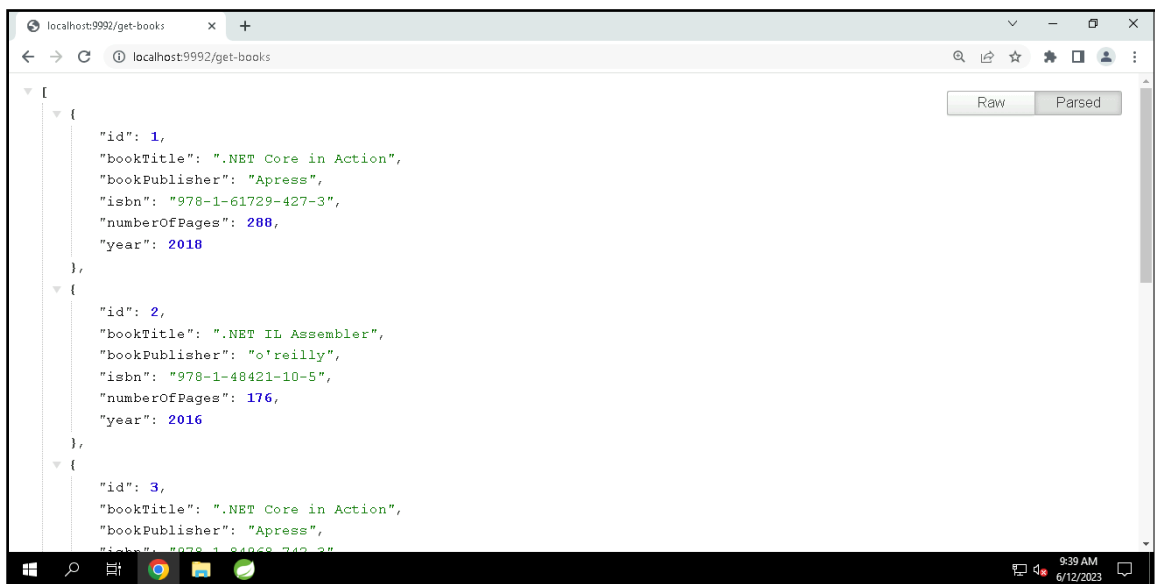
4. Enable the **spring.cloud.openfeign.circuitbreaker.enabled=true** and all actuator endpoints in the application.properties files.

```
server.port=9992
eureka.client.serviceUrl.defaultZone=http://localhost:8761/eureka
eureka.client.register-with-eureka=false

#Enable Actuator
management.endpoints.web.exposure.include=*

#From Spring Boot 3.2.x
spring.cloud.openfeign.circuitbreaker.enabled=true
```

5. Create a **BookServiceFallback** class and implement your fallback methods i.e., your business logic. Should be annotated with **@Component/@Service**.
6. Add the fallback class to BookServiceProxy Interface using **@FeignClient(name = "book-service", fallback=BookServiceFallback.class)** annotation.
7. Add **@EnableFeignClients** in the Application class.
8. Start the eBookStore-Consumer-Feign-Resilience4J.
9. Test the consumer: <http://localhost:9992/get-books>



10. Test the consumer: <http://localhost:9992/get-books/2>



11. Just to verify the Resilience4J is working or not, stop running the ebookstoreapp and refresh the browser. You will find the calls are delegated to fallback methods.

Problem Statement 22: Configuring Circuit Breaker and Retry with Spring Cloud Resilience4J to Your Feign Client Application

Let's implement the Circuit Breaker and Retry mechanism to your Feign Client Application in a similar manner.

Implementation approach for Circuit Breaker and Retry mechanism:

1. Refer the **eBookStore-Consumer-Feign-Resilience4J** created in the problem statement – 21.
2. Let's add the configuration for the circuit breaker and health endpoints via the Spring Boot Actuator in the application.properties file:

```
application.properties
1
2 server.port=8998
3 eureka.client.serviceUrl.defaultZone=http://localhost:8761/eureka
4 eureka.client.register-with-eureka=false
5
6 spring.cloud.openfeign.circuitbreaker.enabled=true
7
8 #Enable Actuator
9 management.endpoints.web.exposure.include=*
10 management.endpoint.health.show-details=always
11 management.health.circuitbreakers.enabled=true
12 management.health.ratelimiters.enabled=true
13
14 resilience4j.circuitbreaker.instances.book-service.failure-rate-threshold=50
15 resilience4j.circuitbreaker.instances.book-service.automatic-transition-from-open-to-half-open-enabled=true
16 resilience4j.circuitbreaker.instances.book-service.wait-duration-in-open-state=10s
17 resilience4j.circuitbreaker.instances.book-service.sliding-window-type=count-based
18 #resilience4j.circuitbreaker.instances.book-service.sliding-window-type=time-based
19 resilience4j.circuitbreaker.instances.book-service.sliding-window-size=2
20 resilience4j.circuitbreaker.instances.book-service.minimum-number-of-calls=1
21 resilience4j.circuitbreaker.instances.book-service.event-consumer-buffer-size=10
22
```

3. Add the **@CircuitBreaker** annotation to the **BookServiceProxy**. If you want to find the state-transition in /actuator/circuitbreakerevents do the below changes to BookServiceProxy.

```
BookServiceProxy.java
14
15 @FeignClient(name="book-service")
16 public interface BookServiceProxy {
17
18     @CircuitBreaker(name="book-service", fallbackMethod = "fallbackMethodGetBookById")
19     @GetMapping(value = "/books/{id}", produces = {MediaType.APPLICATION_JSON_VALUE})
20     public Book getBookById(@PathVariable("id") Integer id);
21
22     @CircuitBreaker(name="book-service", fallbackMethod = "fallbackMethodGetAllBooks")
23     @GetMapping(value = "/books", produces = {MediaType.APPLICATION_JSON_VALUE})
24     public List<Book> getAllBooks();
25
26     default Book fallbackMethodGetBookById(Integer id, Throwable cause) {
27         System.out.println("Exception raised with message :==>" + cause.getMessage());
28         return new Book(id, ".Net Core in Action", "Apress", "978-61729-427-3", 288, 2018);
29     }
30
31     default List<Book> fallbackMethodGetAllBooks(Throwable cause) {
32         System.out.println("Exception raised with message :==>" + cause.getMessage());
33         return Arrays.asList();
34     }
35 }
36
```

4. Ensure the RabbitMQ docker container, Config Server, Eureka Server and ebookstoreapp microservice is running.
5. Start the eBookStore-Consumer-Feign-Resilience4J microservice.

6. Test the consumer: <http://localhost:8998/get-books/3>



```
1 // 20230624041343
2 // http://localhost:8998/get-books/3
3
4 {
5   "id": 3,
6   "bookTitle": "101 UX Principles",
7   "bookPublisher": "Apress",
8   "isbn": "978-1-78883-736-1",
9   "page": 414,
10  "year": 2013
11 }
```

7. Test the consumer: <http://localhost:8998/get-books>



```
1 // 20230624041539
2 // http://localhost:8998/get-books
3
4 [
5   {
6     "id": 1,
7     "bookTitle": ".Net Core in Action",
8     "bookPublisher": "Apress",
9     "isbn": "978-61729-427-3",
10    "page": 288,
11    "year": 2018
12   },
13   {
14     "id": 2,
15     "bookTitle": ".Net IL Assembler",
16     "bookPublisher": "Apress",
17     "isbn": "978-1-78883-466-7",
18     "page": 394,
19     "year": 2018
20   }
21 ]
```

8. Access the endpoint: <http://localhost:8998/actuator/circuitbreakerevents> which will list by default the latest 100 emitted events of all the Circuit Breaker instances.



```
4 {
5   "circuitBreakerEvents": [
6     {
7       "circuitBreakerName": "BookServiceProxy#getBookById(Integer)",
8       "type": "SUCCESS",
9       "creationTime": "2023-06-24T04:13:42.985Z[UTC]",
10      "errorMessage": null,
11      "durationInMs": 302,
12      "stateTransition": null
13    },
14    {
15      "circuitBreakerName": "BookServiceProxy#getAllBooks()",
16      "type": "SUCCESS",
17      "creationTime": "2023-06-24T04:15:39.737Z[UTC]",
18      "errorMessage": null,
19      "durationInMs": 162,
20      "stateTransition": null
21    }
22  ]
23 }
```

9. Just to verify the Resilience4J Circuit Breaker mechanism is working or not, stop running the ebookstoreapp and refresh the browser. You will find the calls are delegated to fallback methods.

10. Test the consumer – Refresh Thrice: <http://localhost:8999/get-books/3>.
11. Observe the different types of STATE_TRANSITION via Circuit Breaker Events.
12. Start the ebookstoreapp wait for 90 secs and then check the circuit breaker events – state transition.
13. Let's configure the Resilience4J Retry mechanism to our spring boot application properties files.

```
20 resilience4j.retry.instances.book-service.max-attempts=3
21 resilience4j.retry.instances.book-service.wait-duration=2s
```

14. Now, it may happen that retrying after a fixed time could cause the upstream service to further not respond (probably it's already overwhelmed with many requests). In this case, we can provide an exponential back-off mechanism. Let's add this config.

```
23 resilience4j.retry.instances.book-service.enable-exponential-backoff=true
24 resilience4j.retry.instances.book-service.exponential-backoff-multiplier=5
```

15. Also change the default order of aspects, by giving the retry module a higher priority than circuit breaker.

```
26 resilience4j.circuitbreaker.circuit-breaker-aspect-order=1
27 resilience4j.retry.retry-aspect-order=2
```

16. The overall configuration of Circuit Breaker and Retry mechanism.

```
13
14 resilience4j.circuitbreaker.circuit-breaker-aspect-order=1
15
16 resilience4j.circuitbreaker.instances.book-service.failure-rate-threshold=50
17 resilience4j.circuitbreaker.instances.book-service.automatic-transition-from-open-to-half-open-enabled=true
18 resilience4j.circuitbreaker.instances.book-service.wait-duration-in-open-state=10s
19 resilience4j.circuitbreaker.instances.book-service.sliding-window-type=count-based
20 #resilience4j.circuitbreaker.instances.book-service.sliding-window-type=time-based
21 resilience4j.circuitbreaker.instances.book-service.sliding-window-size=2
22 resilience4j.circuitbreaker.instances.book-service.minimum-number-of-calls=1
23 resilience4j.circuitbreaker.instances.book-service.event-consumer-buffer-size=10
24
25 resilience4j.retry.retry-aspect-order=2
26
27 resilience4j.retry.instances.book-service.max-attempts=3
28 resilience4j.retry.instances.book-service.wait-duration=2s
29
30 resilience4j.retry.instances.book-service.enable-exponential-backoff=true
31 resilience4j.retry.instances.book-service.exponential-backoff-multiplier=5
32
```

17. Add the **@Retry** annotation to the BookServiceProxy interface.

```
BookServiceProxy.java
16 @FeignClient(name="book-service")
17 public interface BookServiceProxy {
18
19     @Retry(name = "book-service")
20     @CircuitBreaker(name="book-service", fallbackMethod = "fallbackMethodGetBookById")
21     @GetMapping(value = "/books/{id}", produces = {MediaType.APPLICATION_JSON_VALUE})
22     public Book getBookById(@PathVariable("id") Integer id);
23
24     @Retry(name = "book-service")
25     @CircuitBreaker(name="book-service", fallbackMethod = "fallbackMethodGetAllBooks")
26     @GetMapping(value = "/books", produces = {MediaType.APPLICATION_JSON_VALUE})
27     public List<Book> getAllBooks();
28
29     default Book fallbackMethodGetBookById(Integer id, Throwable cause) {
30         System.out.println("Exception raised with message :===" + cause.getMessage());
31         return new Book(id, ".Net Core in Action", "Apress", "978-61729-427-3", 288, 2018);
32     }
33
34     default List<Book> fallbackMethodGetAllBooks(Throwable cause) {
35         System.out.println("Exception raised with message :===" + cause.getMessage());
36         return Arrays.asList();
37     }
38 }
```

18. Restart the eBookStore-Consumer-Resilience4J microservice.

19. Test the consumer: <http://localhost:8998/get-books/3>

```
localhost:8998/get-books/3
1 // 20230624041343
2 // http://localhost:8998/get-books/3
3
4 {
5   "id": 3,
6   "bookTitle": "101 UX Principles",
7   "bookPublisher": "Apress",
8   "isbn": "978-1-78883-736-1",
9   "page": 414,
10  "year": 2013
11 }
```

20. Just to verify the Retry mechanism is working or not, stop running the ebookstoreapp and refresh the browser. You will find 3 times the exception stack trace and compare the timestamp of each request.

21. Access the endpoint: <http://localhost:8998/actuator/retries> which will list the names of all the retry instance available.

```
localhost:8998/actuator/retries
1 // 20230624054658
2 // http://localhost:8998/actuator/retries
3
4 {
5   "retries": [
6     "book-service"
7   ]
8 }
```


22. Access the endpoint: <http://localhost:8998/actuator/retryevents> which will list by default the latest 100 retry events in the application.



```
// http://localhost:8998/actuator/retryevents
{
  "retryEvents": [
    {
      "retryName": "book-service",
      "type": "RETRY",
      "creationTime": "2023-06-24T05:40:28.203Z[UTC]",
      "errorMessage": "org.springframework.cloud.client.circuitbreaker.NoFallbackAvailableException: No fallback available.",
      "numberOfAttempts": 1
    },
    {
      "retryName": "book-service",
      "type": "RETRY",
      "creationTime": "2023-06-24T05:40:31.206Z[UTC]",
      "errorMessage": "org.springframework.cloud.client.circuitbreaker.NoFallbackAvailableException: No fallback available.",
      "numberOfAttempts": 2
    }
  ]
}
```



```

    {
      "numberOfAttempts": 1
    },
    {
      "retryName": "book-service",
      "type": "RETRY",
      "creationTime": "2023-06-24T05:40:31.206Z[UTC]",
      "errorMessage": "org.springframework.cloud.client.circuitbreaker.NoFallbackAvailableException: No fallback available.",
      "numberOfAttempts": 2
    },
    {
      "retryName": "book-service",
      "type": "ERROR",
      "creationTime": "2023-06-24T05:40:42.212Z[UTC]",
      "errorMessage": "org.springframework.cloud.client.circuitbreaker.NoFallbackAvailableException: No fallback available.",
      "numberOfAttempts": 3
    }
  ]
}
```

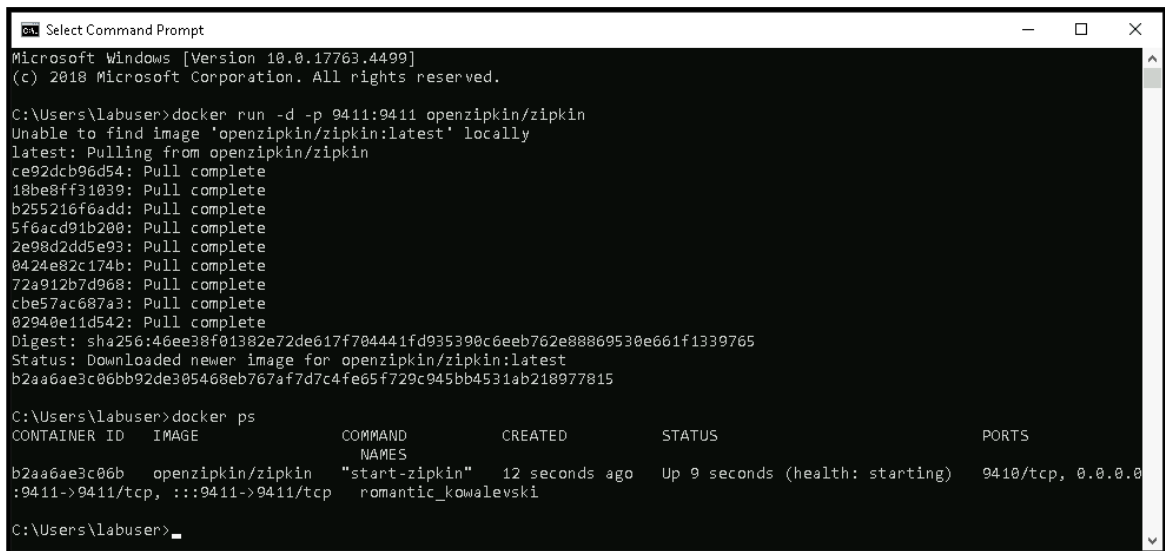
Problem Statement 23: Monitoring your Ribbon Client Application with Sleuth and Zipkin

In a microservices architecture, it is common to have multiple services communicating with each other over the network. This distributed nature of the architecture can make it challenging to diagnose and debug issues when they arise. To address this problem, developers often use generating(Sleuth) and tracing tools(Zipkin) to track the flow of requests between microservices.

Implementation approach to trace HTTP requests using Sleuth and Zipkin:

1. Refer the **eBookStore-Consumer-Hystrix** updated in the problem statement – 15.
2. Ensure the Docker is started on your machine.
3. Starting up Zipkin using Docker Zipkin project. Open the command prompt and execute the below docker command:

`"docker run -d -p 9411:9411 openzipkin/zipkin"`

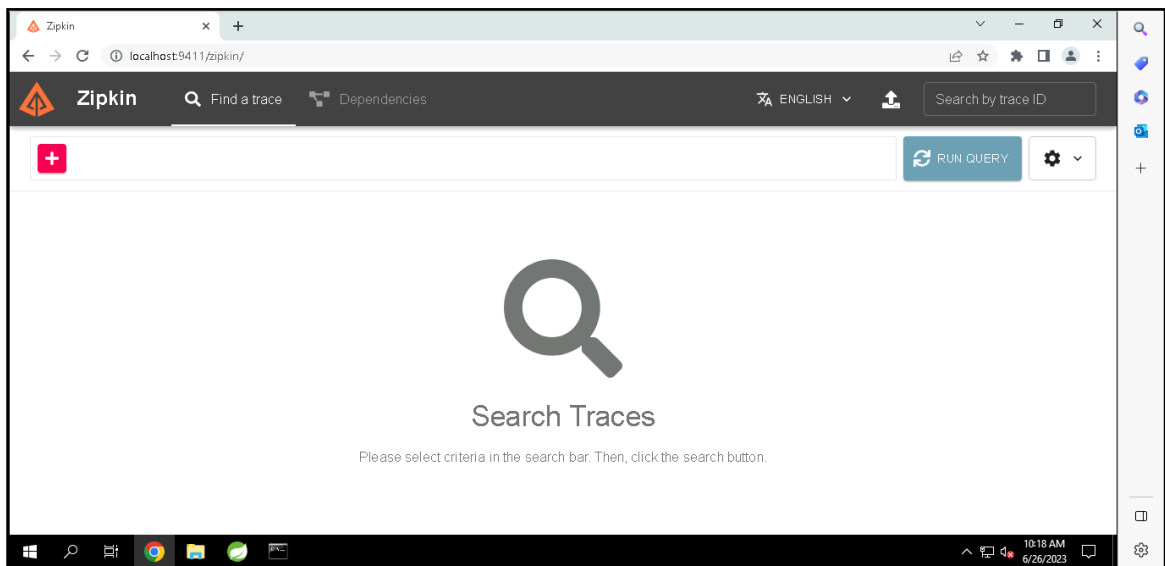


```
Microsoft Windows [Version 10.0.17763.4499]
(c) 2018 Microsoft Corporation. All rights reserved.

C:\Users\labuser>docker run -d -p 9411:9411 openzipkin/zipkin
Unable to find image 'openzipkin/zipkin:latest' locally
latest: Pulling from openzipkin/zipkin
ce92dcb96d54: Pull complete
18be8ff31039: Pull complete
b255216f6add: Pull complete
5f6acd91b200: Pull complete
2e98d2dd5e93: Pull complete
0424e82c174b: Pull complete
72a912b7d968: Pull complete
cbe57ac687a3: Pull complete
02940e11d542: Pull complete
Digest: sha256:46ee38f01382e72de617f704441fd935390c6eeb762e88869530e661f1339765
Status: Downloaded newer image for openzipkin/zipkin:latest
b2aa6ae3c06bb92de305468eb767af7d7c4fe65f729c945bb4531ab218977815

C:\Users\labuser>docker ps
CONTAINER ID   IMAGE             COMMAND                  CREATED        STATUS        PORTS
b2aa6ae3c06b   openzipkin/zipkin "start-zipkin"          12 seconds ago Up 9 seconds (health: starting) 9410/tcp, 0.0.0.0:9411->9411/tcp, :::9411->9411/tcp
romantic_kowalevski
```

4. Once Zipkin is up and running, you can open your web browser and go to <http://localhost:9411/zipkin>.

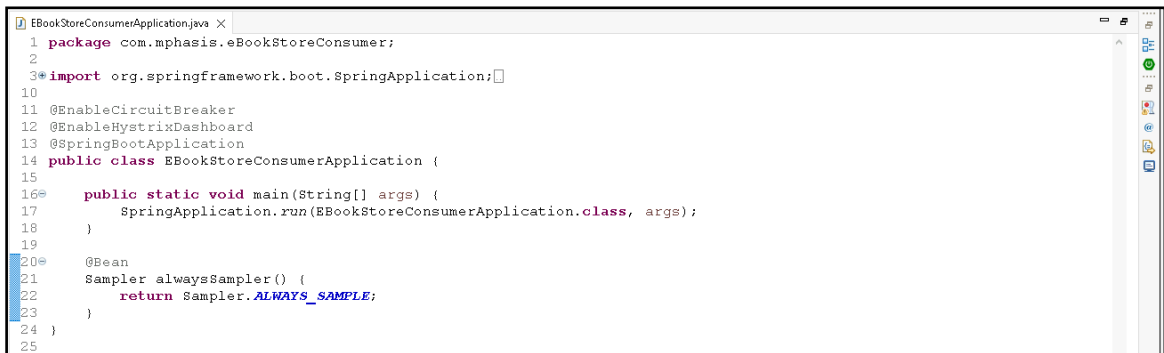


5. Add the Zipkin and Sleuth starter dependency to pom.xml.

```
<dependency>
    <groupId>org.springframework.cloud</groupId>
    <artifactId>spring-cloud-sleuth-zipkin</artifactId>
</dependency>

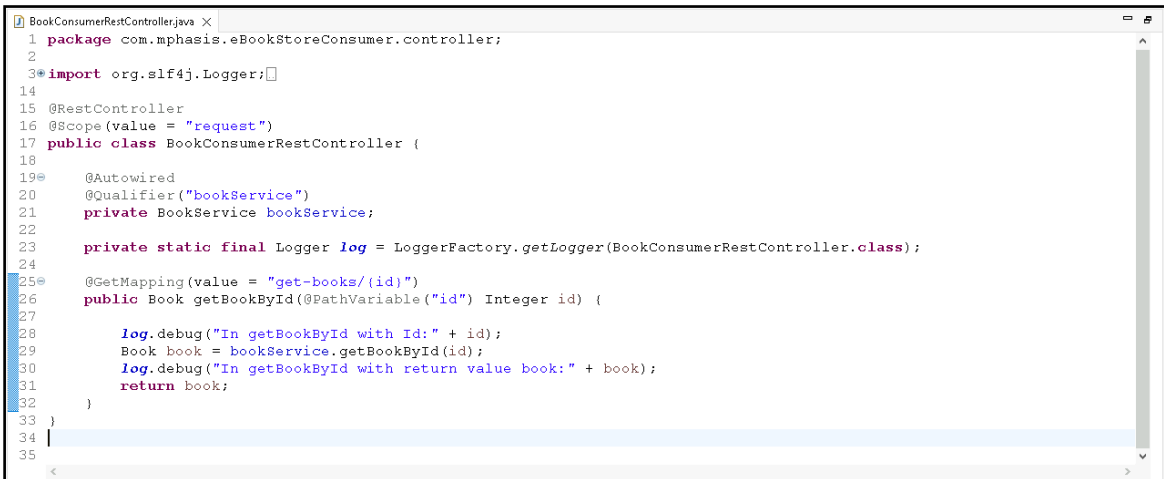
<dependency>
    <groupId>org.springframework.cloud</groupId>
    <artifactId>spring-cloud-starter-sleuth</artifactId>
</dependency>
```

6. Also, we will add `Sampler.ALWAYS_SAMPLE` that traces each action in the Application class.



```
EBookStoreConsumerApplication.java
1 package com.mphasis.eBookStoreConsumer;
2
3 import org.springframework.boot.SpringApplication;
10
11 @EnableCircuitBreaker
12 @EnableHystrixDashboard
13 @SpringBootApplication
14 public class EBookStoreConsumerApplication {
15
16     public static void main(String[] args) {
17         SpringApplication.run(EBookStoreConsumerApplication.class, args);
18     }
19
20     @Bean
21     Sampler alwaysSampler() {
22         return Sampler.ALWAYS_SAMPLE;
23     }
24 }
25
```

7. The `BookConsumerRestController` which exposes an endpoint and invokes one downstream server using the `RestTemplate`. Let's apply the debug logger to the handler methods.



```
BookConsumerRestController.java
1 package com.mphasis.eBookStoreConsumer.controller;
2
3 import org.slf4j.Logger;
14
15 @RestController
16 @Scope(value = "request")
17 public class BookConsumerRestController {
18
19     @Autowired
20     @Qualifier("bookService")
21     private BookService bookService;
22
23     private static final Logger log = LoggerFactory.getLogger(BookConsumerRestController.class);
24
25     @GetMapping(value = "get-books/{id}")
26     public Book getBookById(@PathVariable("id") Integer id) {
27         log.debug("In getBookById with Id: " + id);
28         Book book = bookService.getBookById(id);
29         log.debug("In getBookById with return value book: " + book);
30         return book;
31     }
32 }
33
34
35
```

8. Add the application name and DEBUG logger to the application.properties:

```
application.properties X
1
2 server.port=9995
3 eureka.client.serviceUrl.defaultZone=http://localhost:8761/eureka
4 eureka.client.register-with-eureka=false
5
6 management.endpoints.web.exposure.include=*
7
8 hystrix.dashboard.proxy-stream-allow-list=*
9
10 spring.application.name=eBookStore-Consumer
11 logging.level.com.mphasis.eBookStoreConsumer.controller.BookConsumerRestController=DEBUG
12
```

9. Apply the similar steps to the provider i.e., ebookstoreapp application. Add the Zipkin and Sleuth starter dependency to pom.xml.

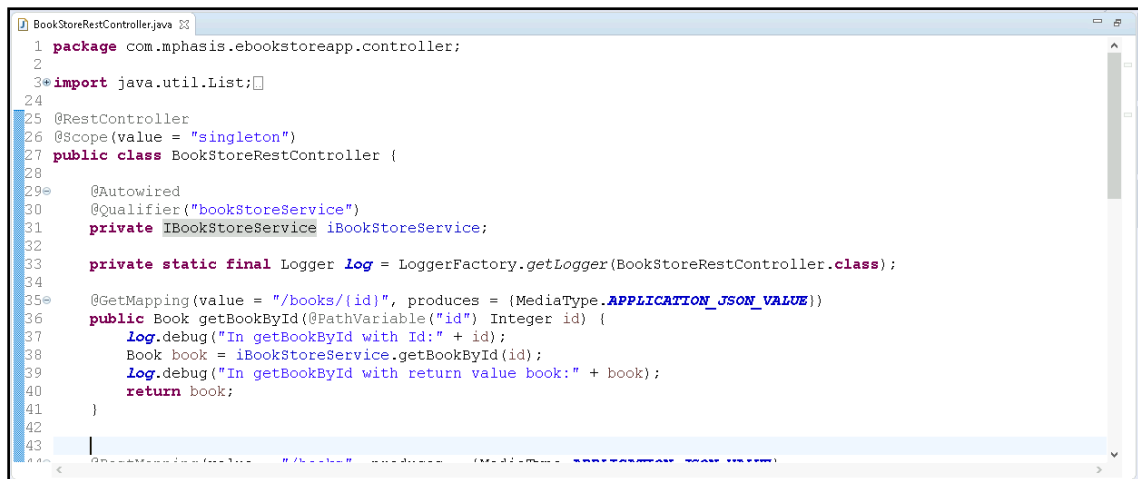
```
<dependency>
    <groupId>org.springframework.cloud</groupId>
    <artifactId>spring-cloud-sleuth-zipkin</artifactId>
</dependency>

<dependency>
    <groupId>org.springframework.cloud</groupId>
    <artifactId>spring-cloud-starter-sleuth</artifactId>
</dependency>
```

10. Also, we will add Sampler.ALWAYS_SAMPLE that traces each action in the Application class.

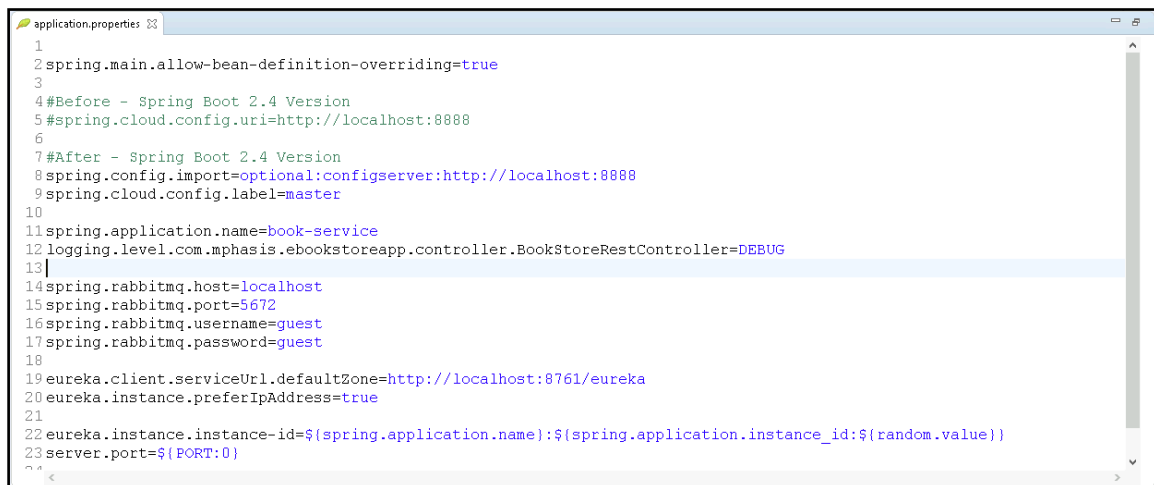
```
EbookstoreappApplication.java X
1 package com.mphasis.ebookstoreapp;
2
3 import org.springframework.beans.factory.annotation.Autowired;
4
15
16 @EnableEurekaClient
17 @SpringBootApplication
18 public class EbookstoreappApplication implements CommandLineRunner {
19
20     @Autowired
21     @Qualifier(value = "bookStoreRepository")
22     private BookStoreRepository bookRepository;
23
24     public static void main(String[] args) {
25         SpringApplication.run(EbookstoreappApplication.class, args);
26     }
27
28     @Bean
29     public Sampler alwaysSampler() {
30         return Sampler.ALWAYS_SAMPLE;
31     }
32 }
```

11. The BookStoreRestController which exposes an GET endpoint (i.e., /books/{id}). Let's apply the debug logger to the handler methods.



```
1 package com.mphasis.ebookstoreapp.controller;
2
3 import java.util.List;
4
25 @RestController
26 @Scope(value = "singleton")
27 public class BookStoreRestController {
28
29     @Autowired
30     @Qualifier("bookStoreService")
31     private IBookStoreService iBookStoreService;
32
33     private static final Logger log = LoggerFactory.getLogger(BookStoreRestController.class);
34
35     @GetMapping(value = "/books/{id}", produces = {MediaType.APPLICATION_JSON_VALUE})
36     public Book getBookById(@PathVariable("id") Integer id) {
37         log.debug("In getBookById with Id:" + id);
38         Book book = iBookStoreService.getBookById(id);
39         log.debug("In getBookById with return value book:" + book);
40         return book;
41     }
42
43 }
```

12. Add the application name and DEBUG logger to the application.properties:



```
1
2 spring.main.allow-bean-definition-overriding=true
3
4 #Before - Spring Boot 2.4 Version
5 spring.cloud.config.uri=http://localhost:8888
6
7 #After - Spring Boot 2.4 Version
8 spring.config.import=optional:configserver:http://localhost:8888
9 spring.cloud.config.label=master
10
11 spring.application.name=book-service
12 logging.level.com.mphasis.ebookstoreapp.controller.BookStoreRestController=DEBUG
13
14 spring.rabbitmq.host=localhost
15 spring.rabbitmq.port=5672
16 spring.rabbitmq.username=guest
17 spring.rabbitmq.password=guest
18
19 eureka.client.serviceUrl.defaultZone=http://localhost:8761/eureka
20 eureka.instance.preferIpAddress=true
21
22 eureka.instance.instance-id=${spring.application.name}:${spring.application.instance_id:${random.value}}
23 server.port=${PORT:0}
```

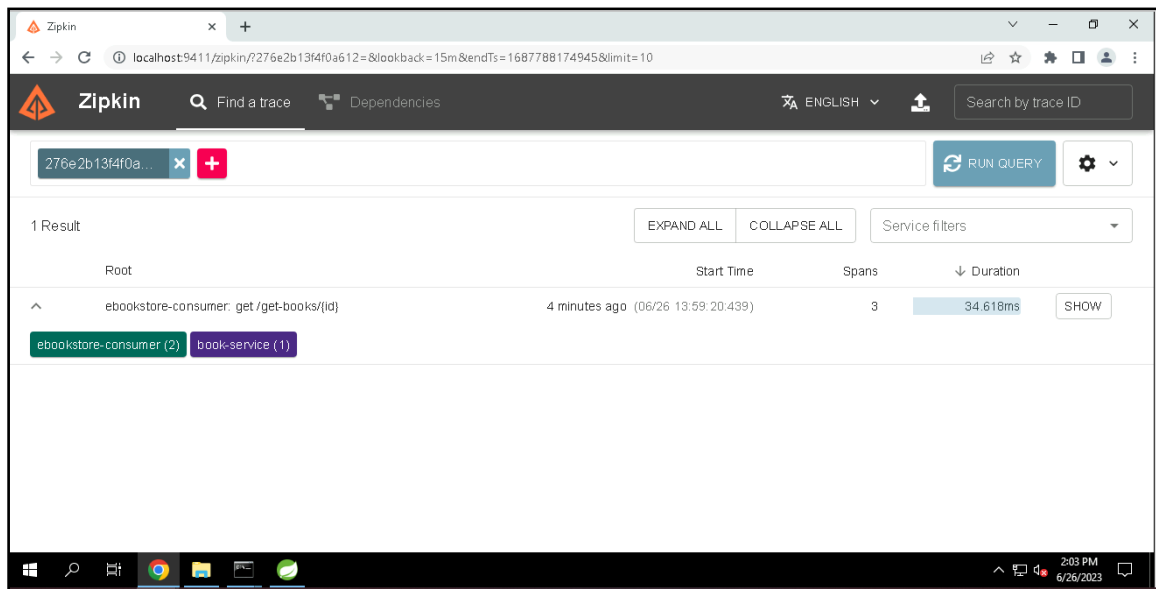
13. Ensure the RabbitMQ docker container, Config Server, Eureka Server and ebookstoreapp microservice is running.
14. Start the eBookStore-Consumer-Hystrix microservice.
15. Test the consumer: <http://localhost:9995/get-books/3>



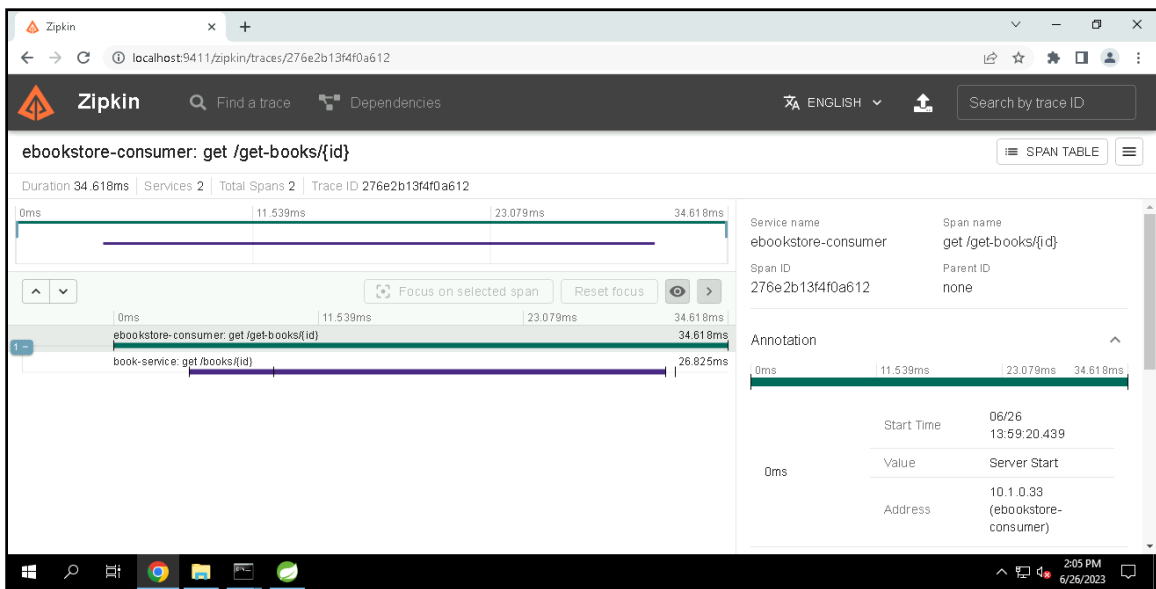
```
1 // 20240312122600
2 // http://localhost:9995/get-books/3
3
4 {
5   "id": 3,
6   "bookTitle": "101 UX Principles",
7   "bookPublisher": "Apress",
8   "isbn": "978-1-78883-736-1",
9   "page": 414,
10  "year": 2013
11 }
```

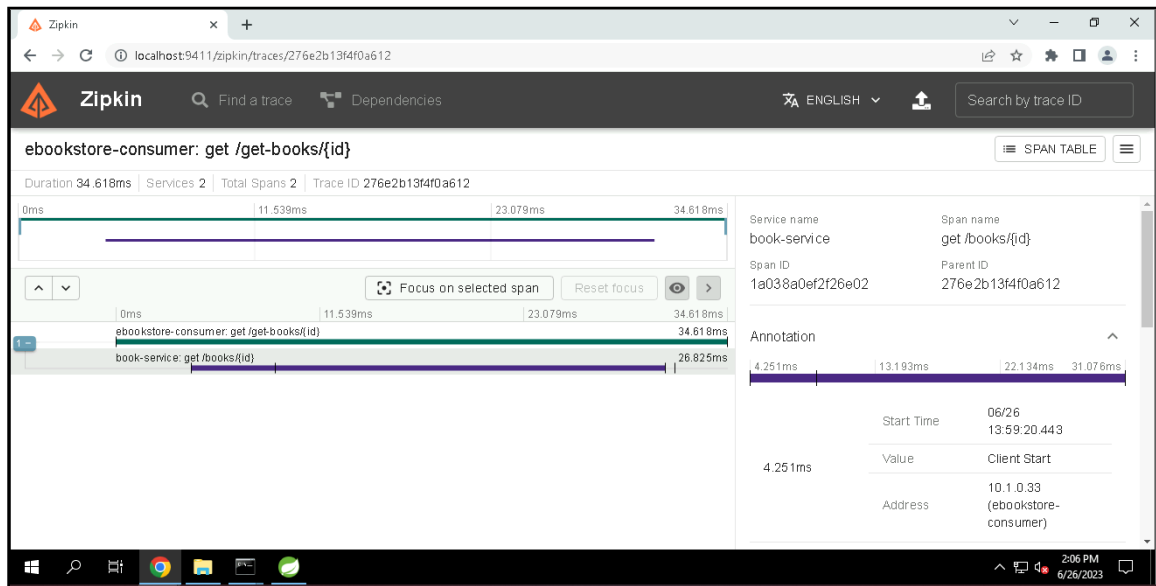
16. Review the debug logs on the console of both the applications.
17. Copy the traceId into the notepad.

18. Let's go to Zipkin (<http://localhost:9411>) webpage and find a trace using the copied traceId.

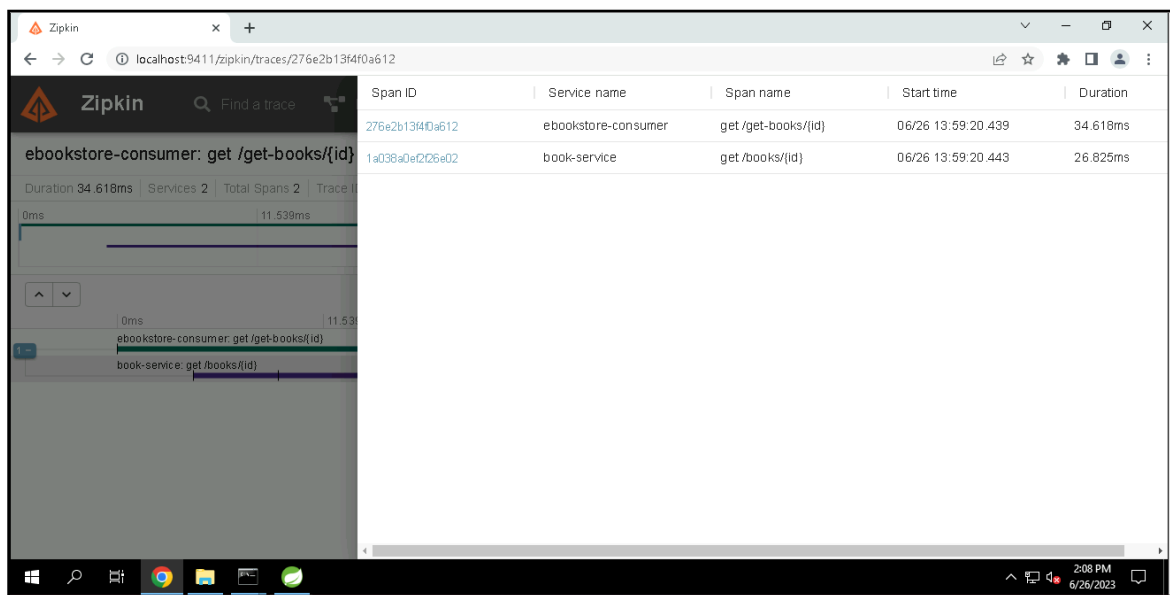


19. Click on SHOW button for the details about the Service i.e., Service name, Span name, Span ID and Parent ID.





20. Click on SPAN TABLE to see the SPAN details.



Problem Statement 24: Implementing an API Gateway with Netflix Zuul

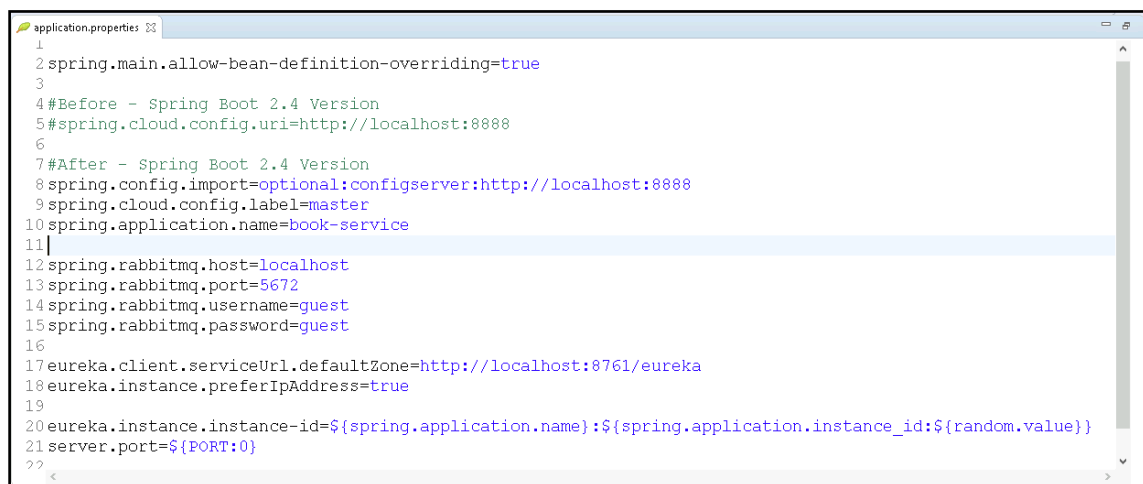
Let's look at how to set up an API gateway that serves as a single point of access for all clients. The ebookstore service will simply be proxied/routed through the API gateway.

Technology stack:

- Spring Cloud Netflix Zuul

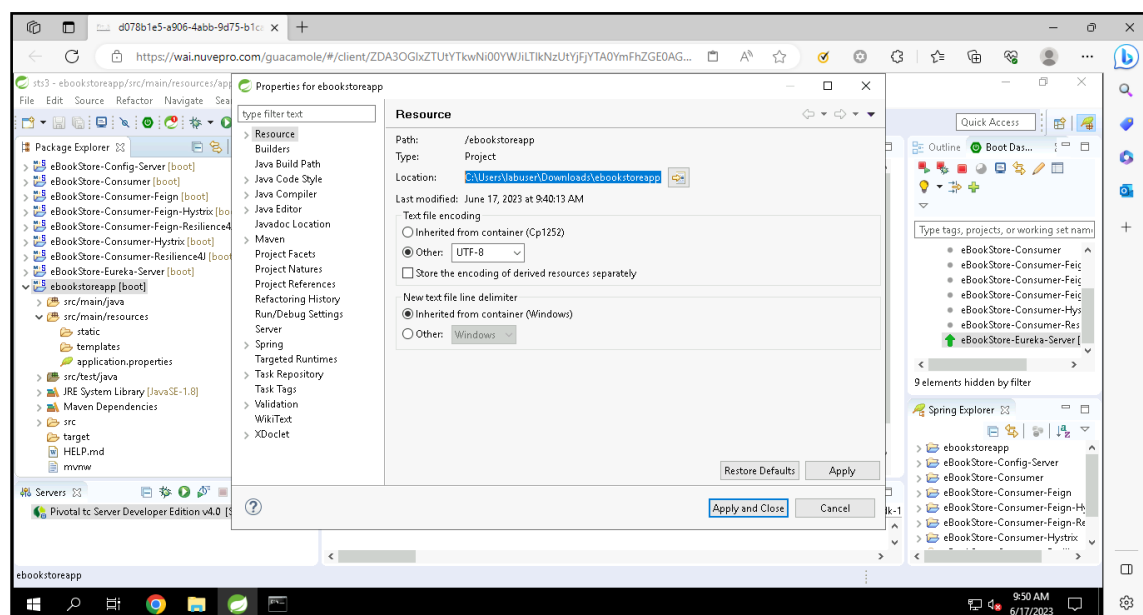
Implementation approach for API Gateway:

1. Refer the **ebookstoreapp** created in the problem statement – 4 and execute it with necessary changes.
2. In application.properties, add the instance-id and port properties:



```
1
2 spring.main.allow-bean-definition-overriding=true
3
4 #Before - Spring Boot 2.4 Version
5 #spring.cloud.config.uri=http://localhost:8888
6
7 #After - Spring Boot 2.4 Version
8 spring.config.import=optional:configserver:http://localhost:8888
9 spring.cloud.config.label=master
10 spring.application.name=book-service
11
12 spring.rabbitmq.host=localhost
13 spring.rabbitmq.port=5672
14 spring.rabbitmq.username=guest
15 spring.rabbitmq.password=guest
16
17 eureka.client.serviceUrl.defaultZone=http://localhost:8761/eureka
18 eureka.instance.preferIpAddress=true
19
20 eureka.instance.instance-id=${spring.application.name}:${spring.application.instance_id:${random.value}}
21 server.port=${PORT:0}
??
```

3. Open the command prompt and navigate to the project location:

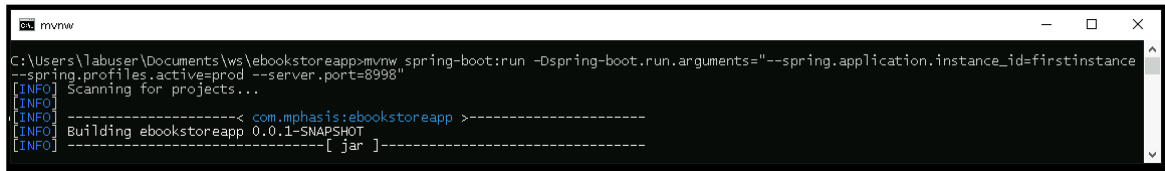


4. Ensure the RabbitMQ docker container, Config Server, and Eureka Server is running.

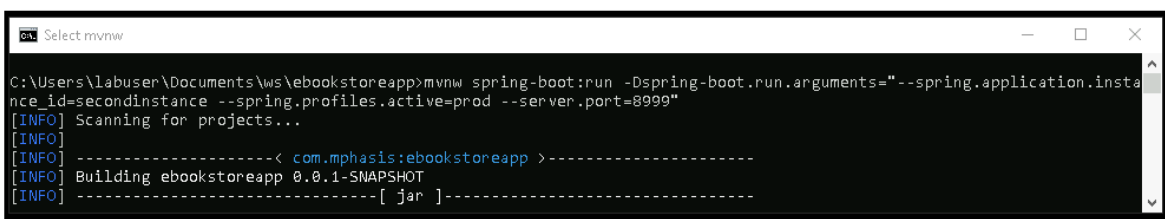
5. Execute the spring-boot ebookstoreapp with the different ports and instance ids.
6. Using the below mvnw commands:

```
mvnw spring-boot:run -Dspring-boot.run.arguments="--spring.application.instance_id=firstinstance  
--spring.profiles.active=prod --server.port=8998"
```

```
mvnw spring-boot:run -Dspring-boot.run.arguments="--spring.application.instance_id=secondinstance  
spring.profiles.active=prod --server.port=8999"
```

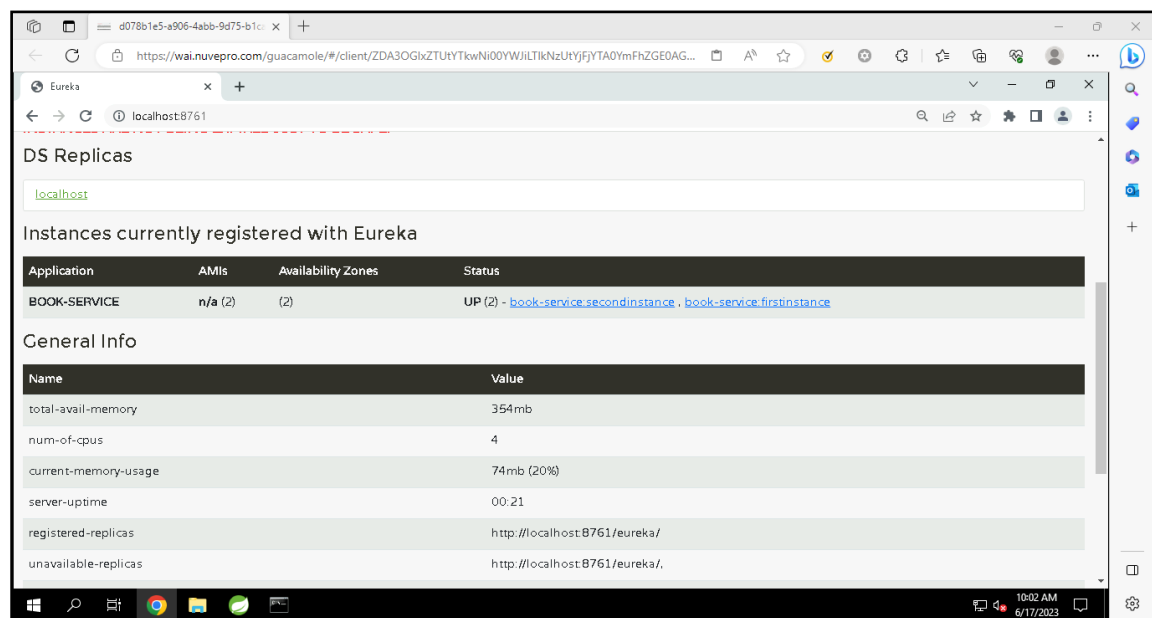


```
C:\Users\labuser\Documents\ws\ebookstoreapp>mvnw spring-boot:run -Dspring-boot.run.arguments="--spring.application.instance_id=firstinstance  
--spring.profiles.active=prod --server.port=8998"  
[INFO] Scanning for projects...  
[INFO] -----< com.mphasis:ebookstoreapp >-----  
[INFO] Building ebookstoreapp 0.0.1-SNAPSHOT  
[INFO] -----[ jar ]-----
```



```
C:\Users\labuser\Documents\ws\ebookstoreapp>mvnw spring-boot:run -Dspring-boot.run.arguments="--spring.application.instance_id=secondinstance  
--spring.profiles.active=prod --server.port=8999"  
[INFO] Scanning for projects...  
[INFO] -----< com.mphasis:ebookstoreapp >-----  
[INFO] Building ebookstoreapp 0.0.1-SNAPSHOT  
[INFO] -----[ jar ]-----
```

7. Check the running instances are registered with the Eureka Server.



The screenshot shows the Eureka Server UI at <https://wai.nuvepro.com/guacamole/#/client/ZDA3OGlxZTUyYTkxNi00YWJlTikzUyYjYTA0YmFhZGE0AG...>. The browser address bar shows localhost:8761.

DS Replicas

[localhost](#)

Instances currently registered with Eureka

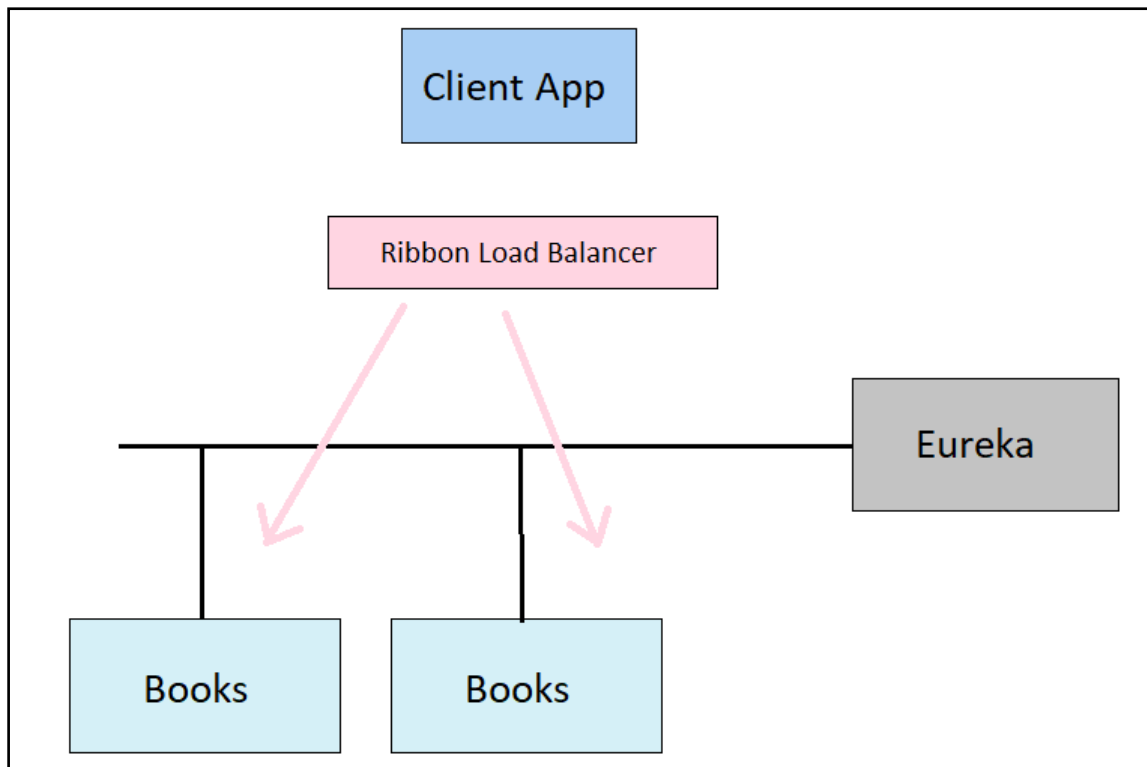
Application	AMIs	Availability Zones	Status
BOOK-SERVICE	n/a (2)	(2)	UP (2) - book-service.secondinstance , book-service.firstinstance

General Info

Name	Value
total-avail-memory	354mb
num-of-cpus	4
current-memory-usage	74mb (20%)
server-up-time	00:21
registered-replicas	http://localhost:8761/eureka/
unavailable-replicas	http://localhost:8761/eureka/

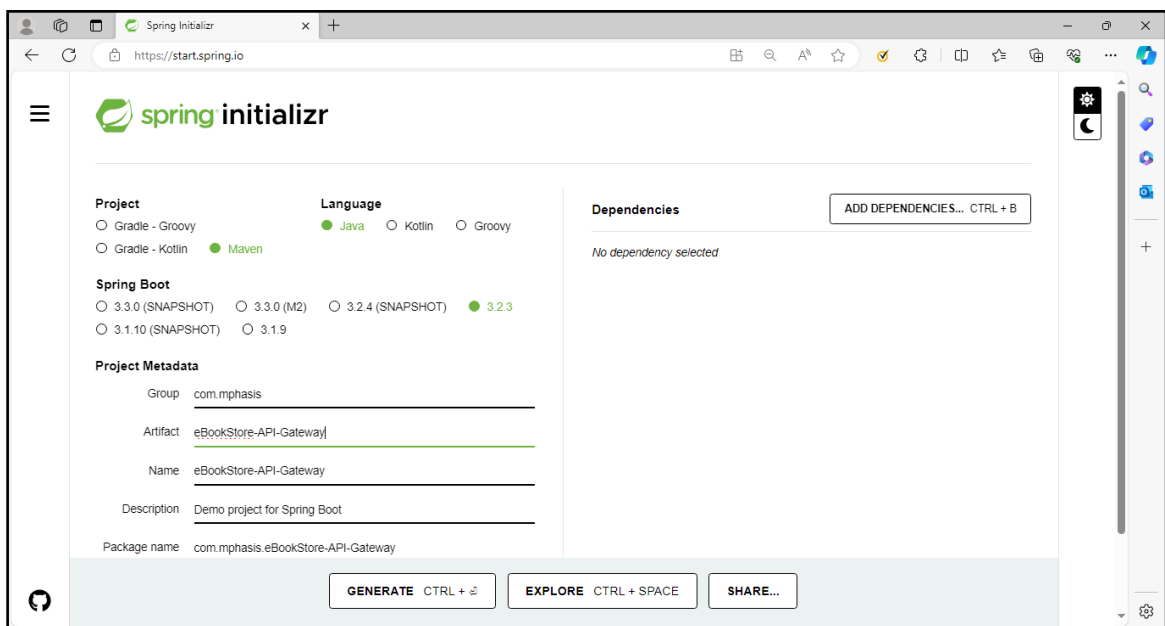
The taskbar at the bottom shows the Windows taskbar with icons for File Explorer, Google Chrome, and other applications. The system clock shows 10:02 AM on 6/17/2023.

8. Once your services are registered inside the Eureka Server. You will find multiple consumers are created using Ribbon Load Balancer. So, till now load balancing is happening at Client-Side.

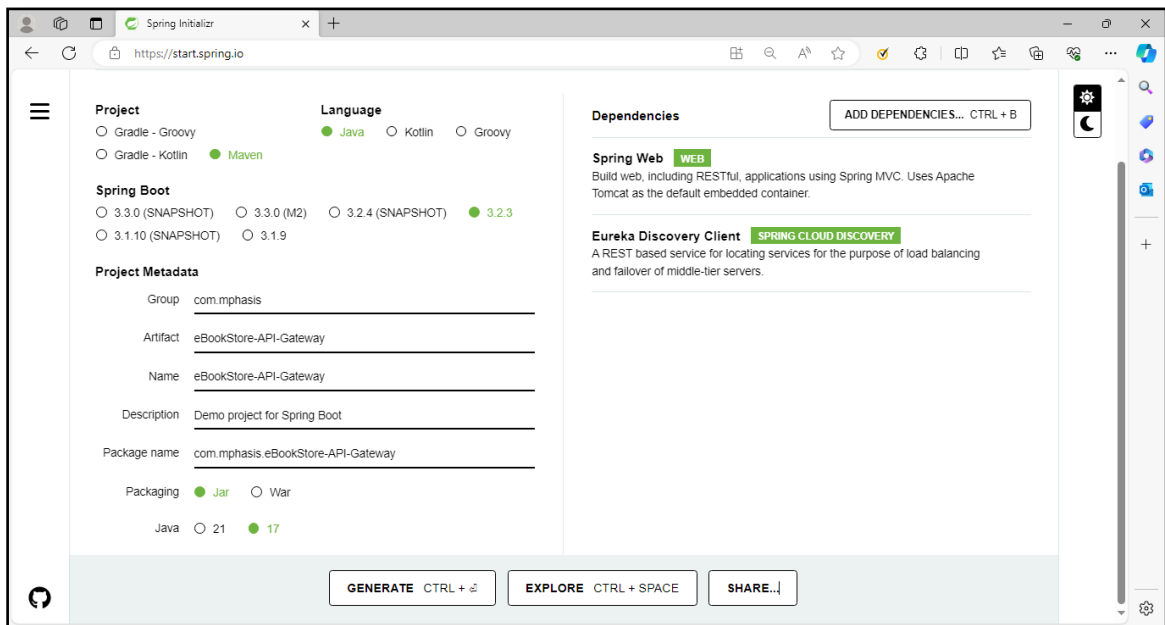


9. Now let implement an API gateway that acts as a single-entry point for all clients.
10. Create a new Project for **eBookStore-API-Gateway** using the **Spring Initializr** (start.spring.io).

Refer the below screenshots:



11. Select the below Dependencies:



The screenshot shows the Spring Initializr web form at <https://start.spring.io>. The form is configured with the following settings:

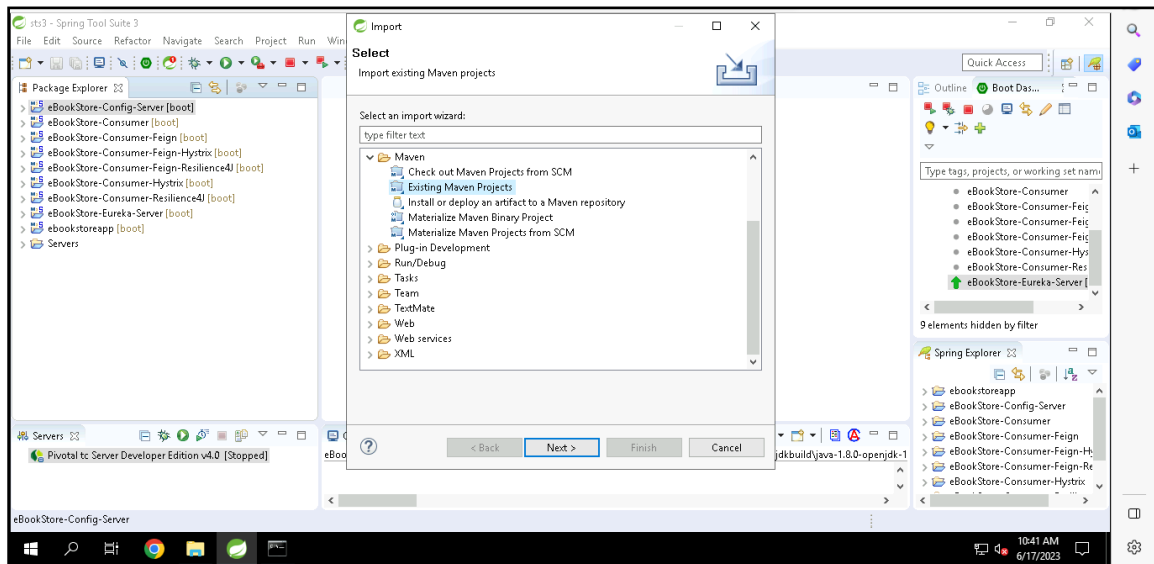
- Project:** ☐ Gradle - Groovy, ☒ Gradle - Kotlin, ☐ Groovy
- Language:** ☒ Java, ☐ Kotlin, ☐ Groovy
- Spring Boot:** ☐ 3.3.0 (SNAPSHOT), ☐ 3.3.0 (M2), ☐ 3.2.4 (SNAPSHOT), ☒ 3.2.3, ☐ 3.1.10 (SNAPSHOT), ☐ 3.1.9
- Project Metadata:**
 - Group:
 - Artifact:
 - Name:
 - Description:
 - Package name:
- Packaging:** ☒ Jar, ☐ War
- Java:** ☐ 21, ☒ 17

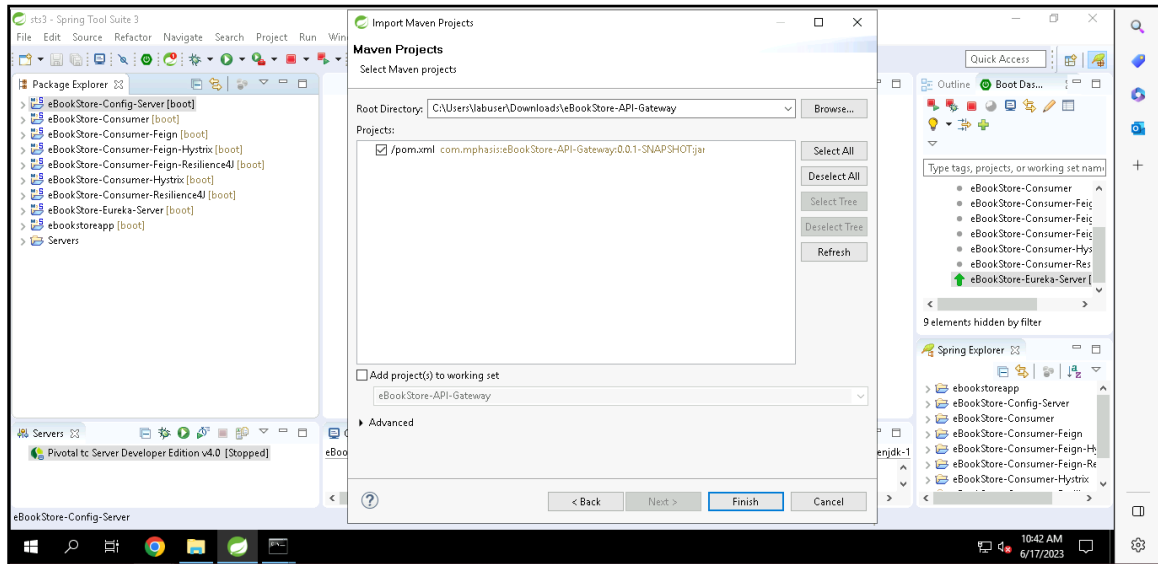
The **Dependencies** section on the right shows the following selected dependencies:

- Spring Web** (WEB): Build web, including RESTful, applications using Spring MVC. Uses Apache Tomcat as the default embedded container.
- Eureka Discovery Client** (SPRING CLOUD DISCOVERY): A REST based service for locating services for the purpose of load balancing and failover of middle-tier servers.

Buttons at the bottom: **GENERATE** (CTRL + G), **EXPLORE** (CTRL + SPACE), **SHARE...**

12. Let's import the eBookStore-API-Gateway maven project in STS.





13. Will take some time for the project to be imported and the maven dependencies to be downloaded once you click **Finish**.
14. Let's downgrade the version of Spring Boot and Spring Cloud in pom.xml for Hystrix implementation.
 - Spring Boot: 2.3.10.RELEASE
 - Spring Cloud: Hoxton.SR11

```
<parent>
  <groupId>org.springframework.boot</groupId>
  <artifactId>spring-boot-starter-parent</artifactId>
  <version>2.3.10.RELEASE</version>
  <relativePath/> <!-- lookup parent from repository -->
</parent>
<groupId>com.mphasis</groupId>
<artifactId>eBookStore-API-Gateway</artifactId>
<version>0.0.1-SNAPSHOT</version>
<name>eBookStore-API-Gateway</name>
<description>Demo project for Spring Boot - API Gateway</description>
<properties>
  <java.version>1.8</java.version>
  <spring-cloud.version>Hoxton.SR11</spring-cloud.version>
</properties>
```

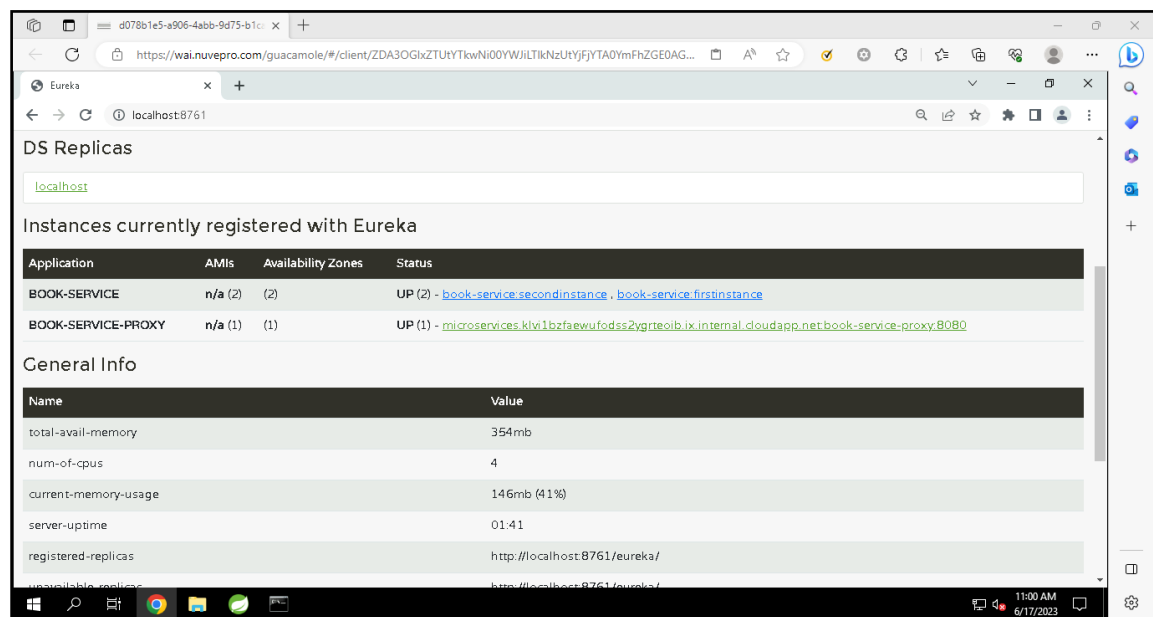
15. Add the **Spring Cloud Netflix Zuul** starter to the application.

```
<dependency>
  <groupId>org.springframework.cloud</groupId>
  <artifactId>spring-cloud-starter-netflix-zuul</artifactId>
</dependency>
```

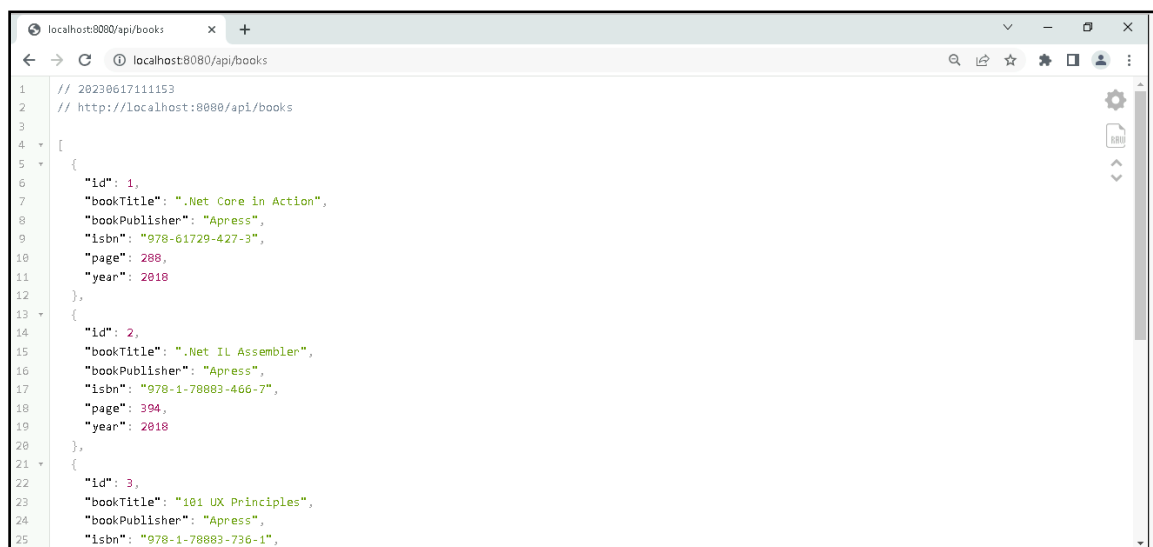
16. Add **@EnableDiscoveryClient** and **@EnableZuulProxy** in the **Application** class.
17. In application.properties files, add the routes, application name and eureka client service url.

```
application.properties
1
2 spring.application.name=book-service-proxy
3 server.port=8080
4 eureka.client.serviceUrl.defaultZone=http://localhost:8761/eureka
5
6 zuul.routes.books.service-id=book-service
7 zuul.routes.books.path=/api/**
8
```

18. Start the eBookStore-API-Gateway.
19. Check the proxy running instances is also registered with the Eureka Server.



20. Test the Proxy: <http://localhost:8080/api/books>



Problem Statement 25: Implementing Spring Cloud Gateway in Microservices

We used Zuul, a blocking API, in the previous problem statement. A blocked gateway API uses as many threads as there are incoming requests. So, this approach is more resource intensive. If no threads are available to process incoming request, the request will be queue.

In this problem statement, we will use Spring Cloud Gateway to implement API Gateway. The Spring Cloud Gateway is a non-blocking API. A thread is always available to process the incoming request while using non-blocking API. These requests are then handled asynchronously in the background, and the response is returned once completed. When using Spring Cloud Gateway, no incoming request is ever blocked.

Technology stack:

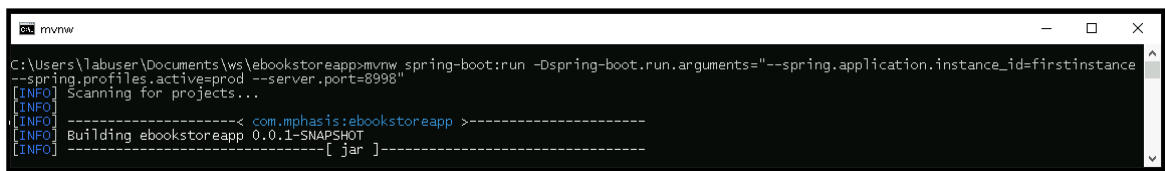
- Spring Cloud Routing - Gateway

Implementation approach for API Gateway:

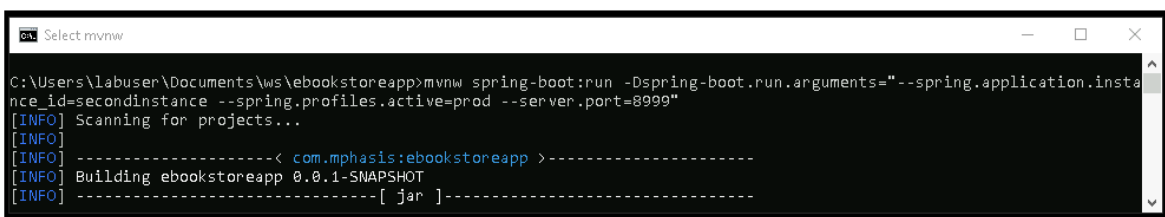
1. Ensure the RabbitMQ docker container, Config Server, and Eureka Server is running.
2. Once again, execute the spring-boot ebookstoreapp with the different ports and instance ids.
3. Using the below mvn commands:

```
mvnw spring-boot:run -Dspring-boot.run.arguments="--spring.application.instance_id=firstinstance  
--spring.profiles.active=prod --server.port=8998"
```

```
mvnw spring-boot:run -Dspring-boot.run.arguments="--spring.application.instance_id=secondinstance  
spring.profiles.active=prod --server.port=8999"
```

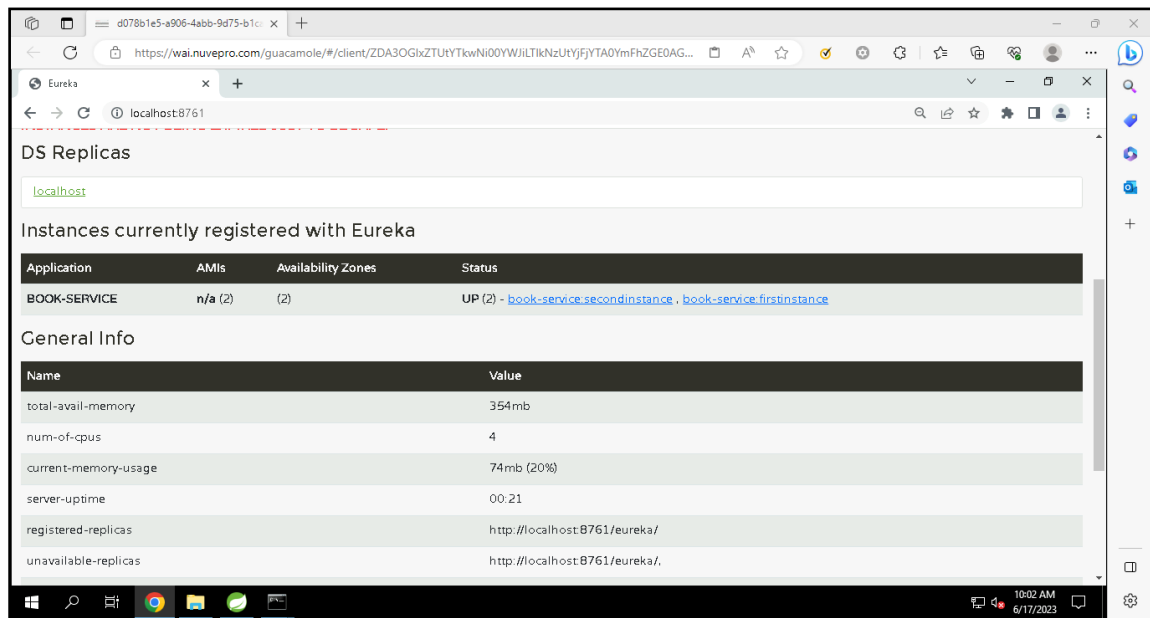


```
et mvnw
C:\Users\labuser\Documents\ws\ebookstoreapp>mvnw spring-boot:run -Dspring-boot.run.arguments="--spring.application.instance_id=firstinstance  
--spring.profiles.active=prod --server.port=8998"
[INFO] Scanning for projects...
[INFO] -----< com.mphasis:ebookstoreapp >-----
[INFO] Building ebookstoreapp 0.0.1-SNAPSHOT
[INFO] -----[ jar ]-----
```



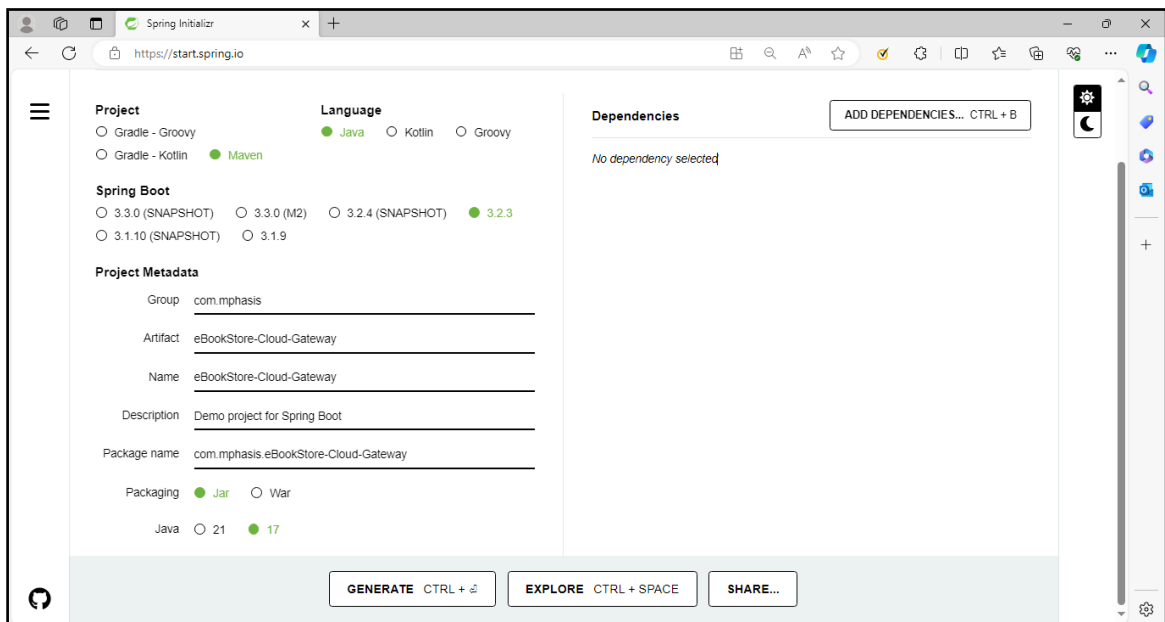
```
et Select mvnw
C:\Users\labuser\Documents\ws\ebookstoreapp>mvnw spring-boot:run -Dspring-boot.run.arguments="--spring.application.instance_id=secondinstance  
--spring.profiles.active=prod --server.port=8999"
[INFO] Scanning for projects...
[INFO] -----< com.mphasis:ebookstoreapp >-----
[INFO] Building ebookstoreapp 0.0.1-SNAPSHOT
[INFO] -----[ jar ]-----
```

4. Check the running instances are registered with the Eureka Server.

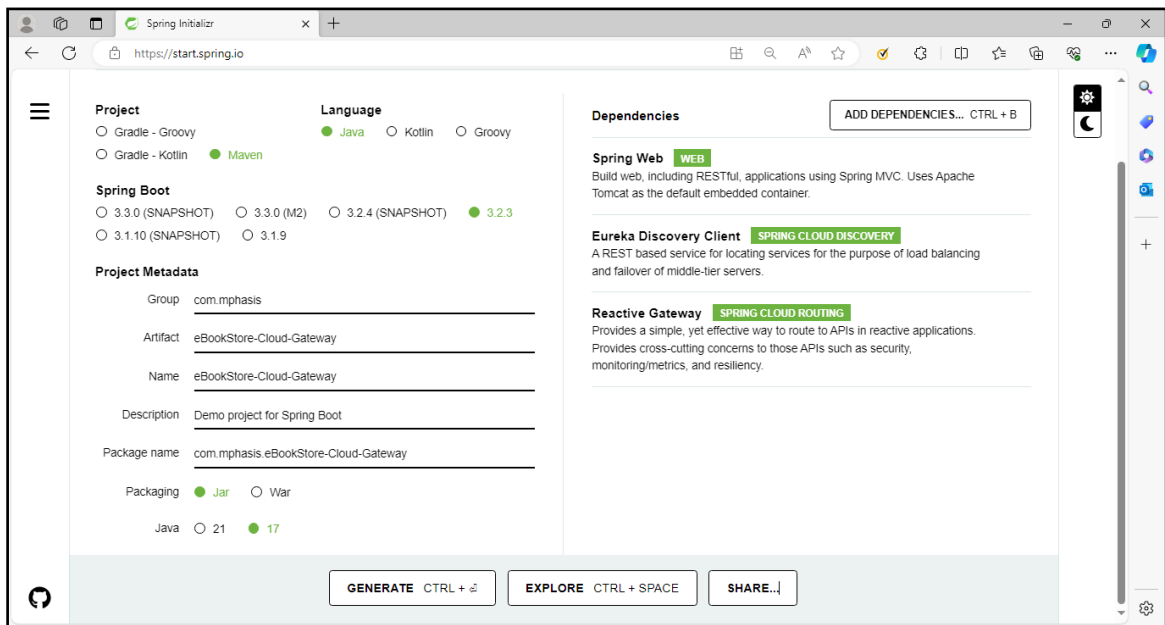


5. Now let implement an API gateway that acts as a single-entry point for a collection of microservices.
6. Create a new Project for **eBookStore-Spring-Cloud-Gateway** using the **Spring Initializr** (start.spring.io).

Refer the below screenshots:



7. Select the below Dependencies:



The screenshot shows the Spring Initializr web form at <https://start.spring.io>. The form is configured with the following settings:

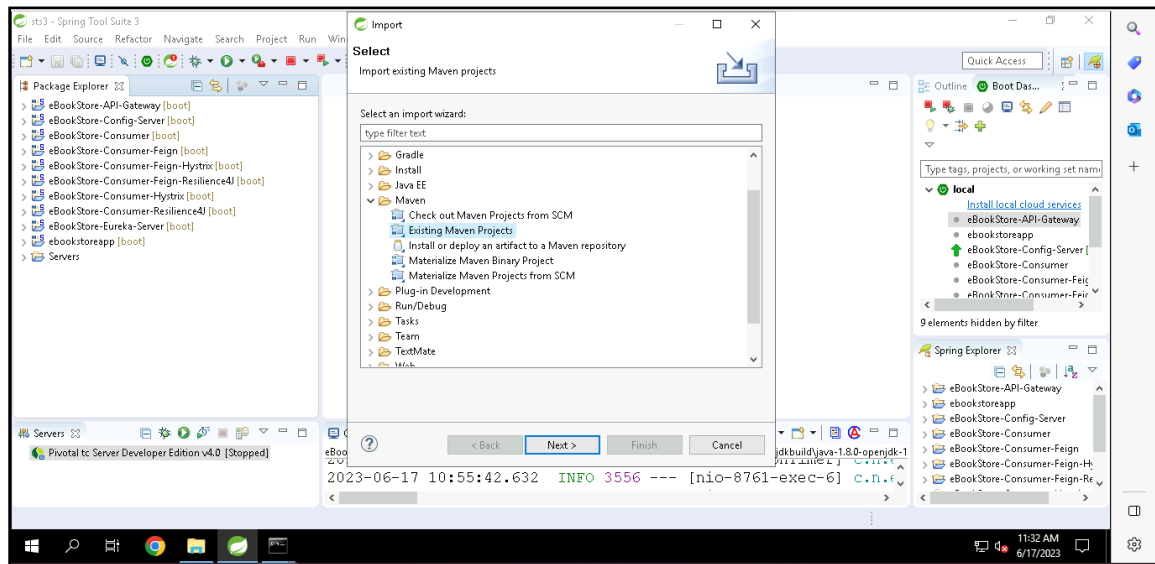
- Project:** ☐ Gradle - Groovy, ☒ Gradle - Kotlin, ☐ Groovy
- Language:** ☒ Java, ☐ Kotlin, ☐ Groovy
- Spring Boot:** ☐ 3.3.0 (SNAPSHOT), ☐ 3.3.0 (M2), ☐ 3.2.4 (SNAPSHOT), ☒ 3.2.3, ☐ 3.1.10 (SNAPSHOT), ☐ 3.1.9
- Project Metadata:**
 - Group:
 - Artifact:
 - Name:
 - Description:
 - Package name:
- Packaging:** ☒ Jar, ☐ War
- Java:** ☐ 21, ☒ 17

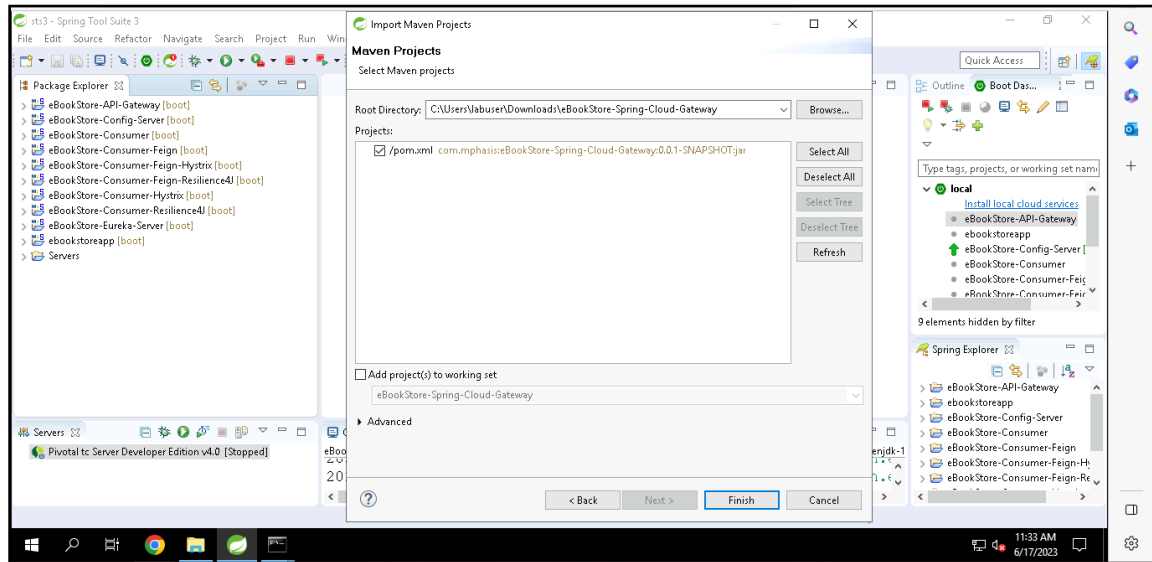
Dependencies:

- Spring Web** WEB: Build web, including RESTful, applications using Spring MVC. Uses Apache Tomcat as the default embedded container.
- Eureka Discovery Client** SPRING CLOUD DISCOVERY: A REST based service for locating services for the purpose of load balancing and failover of middle-tier servers.
- Reactive Gateway** SPRING CLOUD ROUTING: Provides a simple, yet effective way to route to APIs in reactive applications. Provides cross-cutting concerns to those APIs such as security, monitoring/metrics, and resiliency.

Buttons at the bottom: **GENERATE** (CTRL + G), **EXPLORE** (CTRL + SPACE), **SHARE...**

8. Let's import the eBookStore-Spring-Cloud-Gateway maven project in STS.



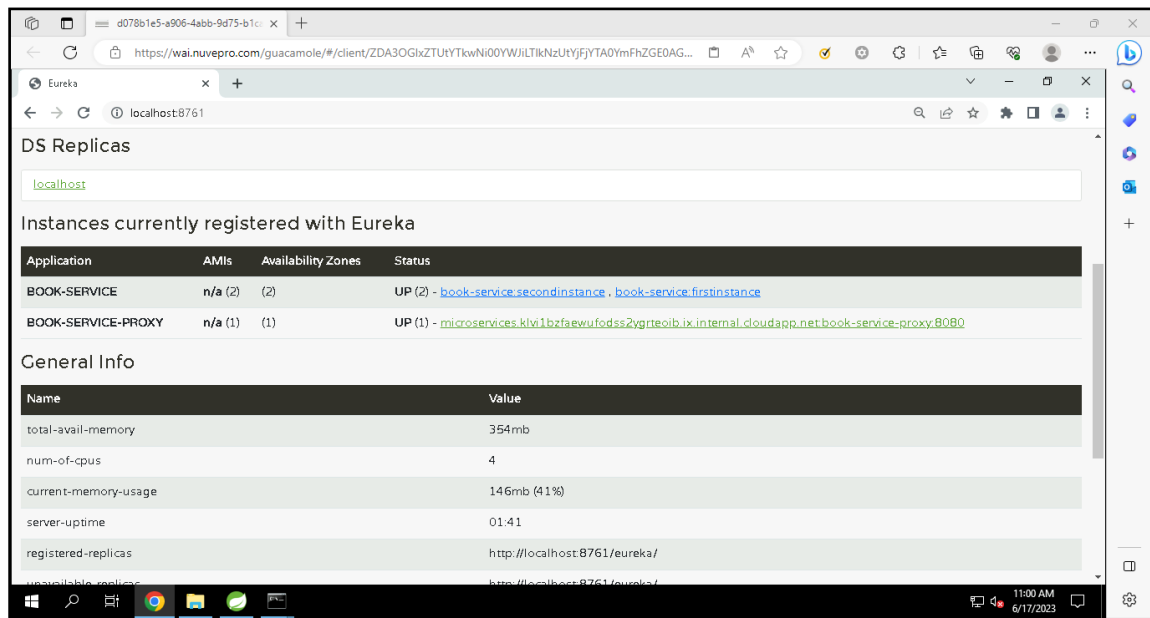


9. Will take some time for the project to be imported and the maven dependencies to be downloaded once you click **Finish**.
10. Add **@EnableDiscoveryClient** in the Application class.
11. In application.properties file, enable the automatic mapping of gateway routes and add the application name and eureka client serviceUrl.

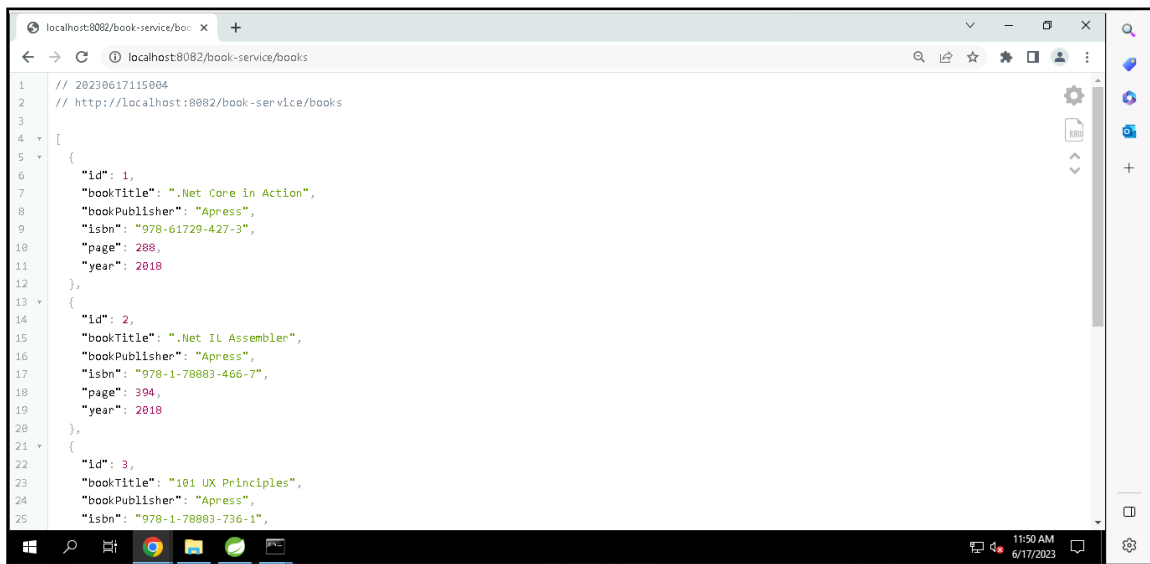
```
application.properties
1
2 spring.application.name=book-service-proxy
3 server.port=8082
4 eureka.client.serviceUrl.defaultZone=http://localhost:8761/eureka
5
6 spring.cloud.gateway.discovery.locator.enabled=true
7 spring.cloud.gateway.discovery.locator.lower-case-service-id=true
8
```

12. Start the eBookStore-Spring-Cloud-Gateway.

13. Check the proxy running instances is also registered with the Eureka Server.



14. Test the Proxy: <http://localhost:8082/book-service/books>

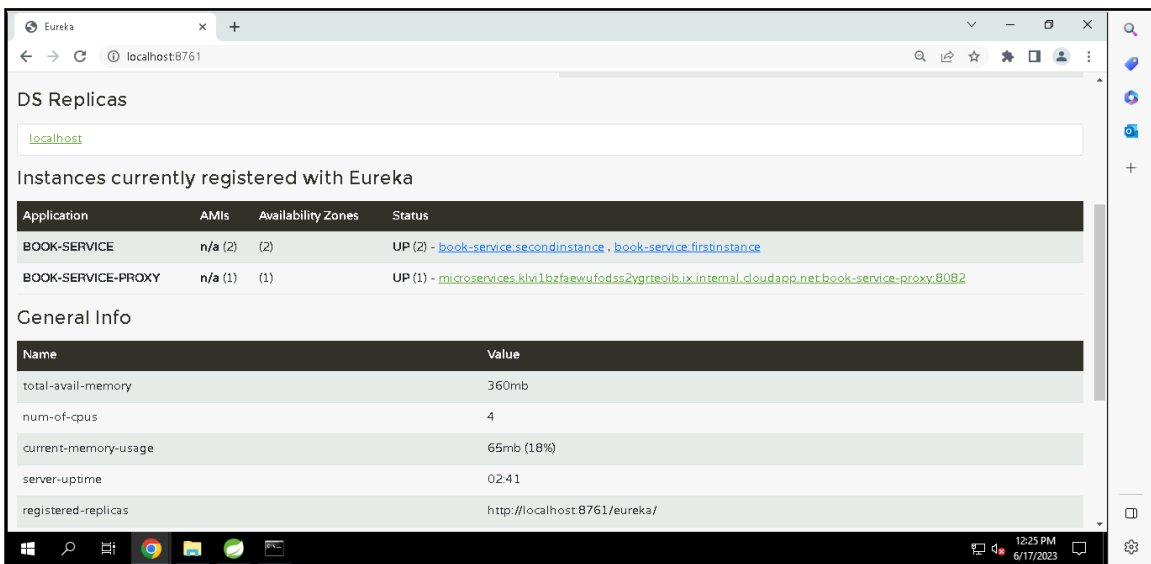


15. Now let's configure API Gateway Routes manually. In application.properties file, add the routes and build-in filters.

```
application.properties
1
2 spring.application.name=book-service-proxy
3 server.port=8082
4 eureka.client.serviceUrl.defaultZone=http://localhost:8761/eureka
5
6 #spring.cloud.gateway.discovery.locator.enabled=true
7 #spring.cloud.gateway.discovery.locator.lower-case-service-id=true
8
9 spring.cloud.gateway.routes[0].id=bookServiceModule
10 spring.cloud.gateway.routes[0].uri=lb://book-service
11 spring.cloud.gateway.routes[0].predicates[0]=Path=/books/**
12 spring.cloud.gateway.routes[0].predicates[1]=Method=GET
13 spring.cloud.gateway.routes[0].filters[0]=RemoveRequestHeader=Cookie
14
```

16. Re-start the eBookStore-Spring-Cloud-Gateway.

17. Check the proxy running instances is also registered with the Eureka Server.



The screenshot shows the Eureka Server web interface at localhost:8761. It displays the 'DS Replicas' section with 'localhost' selected. Below, it lists 'Instances currently registered with Eureka' in a table:

Application	AMIs	Availability Zones	Status
BOOK-SERVICE	n/a (2)	(2)	UP (2) - book-service.secondinstance , book-service.firstinstance
BOOK-SERVICE-PROXY	n/a (1)	(1)	UP (1) - microservices.klvi1bzfaewufodss2vgrteolb.ix.internal.cloudapp.net:book-service-proxy:8082

Below the table is the 'General Info' section with a table of system metrics:

Name	Value
total-avail-memory	360mb
num-of-cpus	4
current-memory-usage	65mb (18%)
server-upptime	02:41
registered-replicas	http://localhost:8761/eureka/

18. Test the Proxy: <http://localhost:8082/books/2>



The screenshot shows a web browser at localhost:8082/books/2. The response is a JSON object:

```
{
  "id": 2,
  "bookTitle": ".Net IL Assembler",
  "bookPublisher": "Apress",
  "isbn": "978-1-78883-466-7",
  "page": 394,
  "year": 2018
}
```